Redis의 수많은 자료 구조를 효율적으로 배우려면?

✅ Redis의 수많은 자료 구조를 실전에서 써먹을 수 있게 배우는 방법

'비전공자를 위한 Redis 입문/실전 (조회 성능 최적화편)' 강의에서는 가장 기본적이고 많이 쓰이는 **String 타입** 딱 한 가지를 배웠다. 하지만 Redis는 String 타입 이외에도 **List, Set, Hash, Sorted Set, Bitmap, Geospatial** 등 다양한 자료 구조를 가지고 있다.

하지만 각 자료 구조의 정의랑 특징이랑 명령어를 사전처럼 줄줄이 배우는 건 실전에서 도움이 하나도 안 된다. 그래서 **실전에서 자주 쓰일만한 사례를 기반으로 Redis의 다양한 자료구조를 하나씩 써보면서 익힐 것이다.** 이렇게 배우면 언제 어떻게 Redis를 활용해야 할 지도 익힐 수 있고, Redis의 명령어나 자료구조를 억지로 외우는 게 아니라 저절로 외워지는 경험을 할 수 있게 될 것이다.

그럼 본격적으로 수업으로 들어가보자.

@naver.com
@gmail.com
khb3025

📖

List 자료 구조 (RPUSH, LRANGE, LPOP 등)

✅ List (리스트)

Redis에서 **List(리스트)**는 **순서대로 데이터를 저장할 수 있는 자료구조**이다. 이런 특성 때문에 실제 서비스에서 **스택(stack)**과 **큐(queue)**로 자주 사용된다.

✅ 자주 사용하는 명령어

```
# RPUSH [key] [value] : 리스트 오른쪽에 데이터를 추가한다.
$ RPUSH my_list 1
$ RPUSH my_list 2

# 리스트가 잘 생성됐는 지 조회해보기
$ keys *

# LRANGE [key] [start index] [end index] : 인덱스 범위를 활용해 리스트 데이터 조회
$ LRANGE my_list 0 -1 # 리스트의 모든 데이터 조회


# LLEN [key] : 리스트에서 데이터의 총 개수를 조회
$ LLEN my_list

# LPOP [key] : 리스트 왼쪽(첫 부분)에서 데이터를 꺼내온다.
$ LPOP my_list # 1
$ LPOP my_list # 2
```

- **참고)** Redis에서 List는 데이터를 넣는 순간 알아서 생성된다. 그래서 따로 List를 생성하는 명령어는 존재하지 않는다.

✅ 기타 명령어

```
# LPUSH [key] [value] : 리스트 왼쪽에 데이터를 추가한다.
# RPOP [key] : 리스트 오른쪽에서 데이터를 꺼내온다.
```

🎯

[실습] 좋아요 수 폭증으로 인한 DB 부하를 Redis로 해결하기 - 1

✅ 문제 상황

'비전공자를 위한 Redis 입문/실전 (조회 성능 최적화편)' 강의에서는 **데이터 조회(Read) 작업**을 빠르게 만드는 가장 기초적이면서도 강력한 전략인 **Cache Aside 전략**을 배웠다. 하지만 **데이터 쓰기(Write) 작업**이 미친 듯이 발생하는 상황이라면 어떻게 해야할까?

예를 들어, 유명 인플루언서가 SNS에 새로운 게시글을 올린 상황을 상상해보자. 알림을 받은 수십만 명의 팔로워가 동시에 앱을 켜고 '좋아요' 버튼을 누를 것이다. 1초에 수천 번의 클릭이 발생할 것이고 그로 인해 1초에 수천 번의 INSERT 쿼리를 날림으로써 DB는 짧은 시간에 엄청난 부하를 받게 된다. 그로 인해 서버는 느려지고, 유저들은 좋아요 버튼을 눌렀는데 뒤늦게 반응을 하거나 작동을 하지 않는 경험을 하게 될 것이다.

```
INSERT INTO likes (user_id, post_id)
VALUES (?, ?)
```

이 상황을 재현해보면서 문제를 해결해나가보자.

✅ 테스트 환경 구성하기

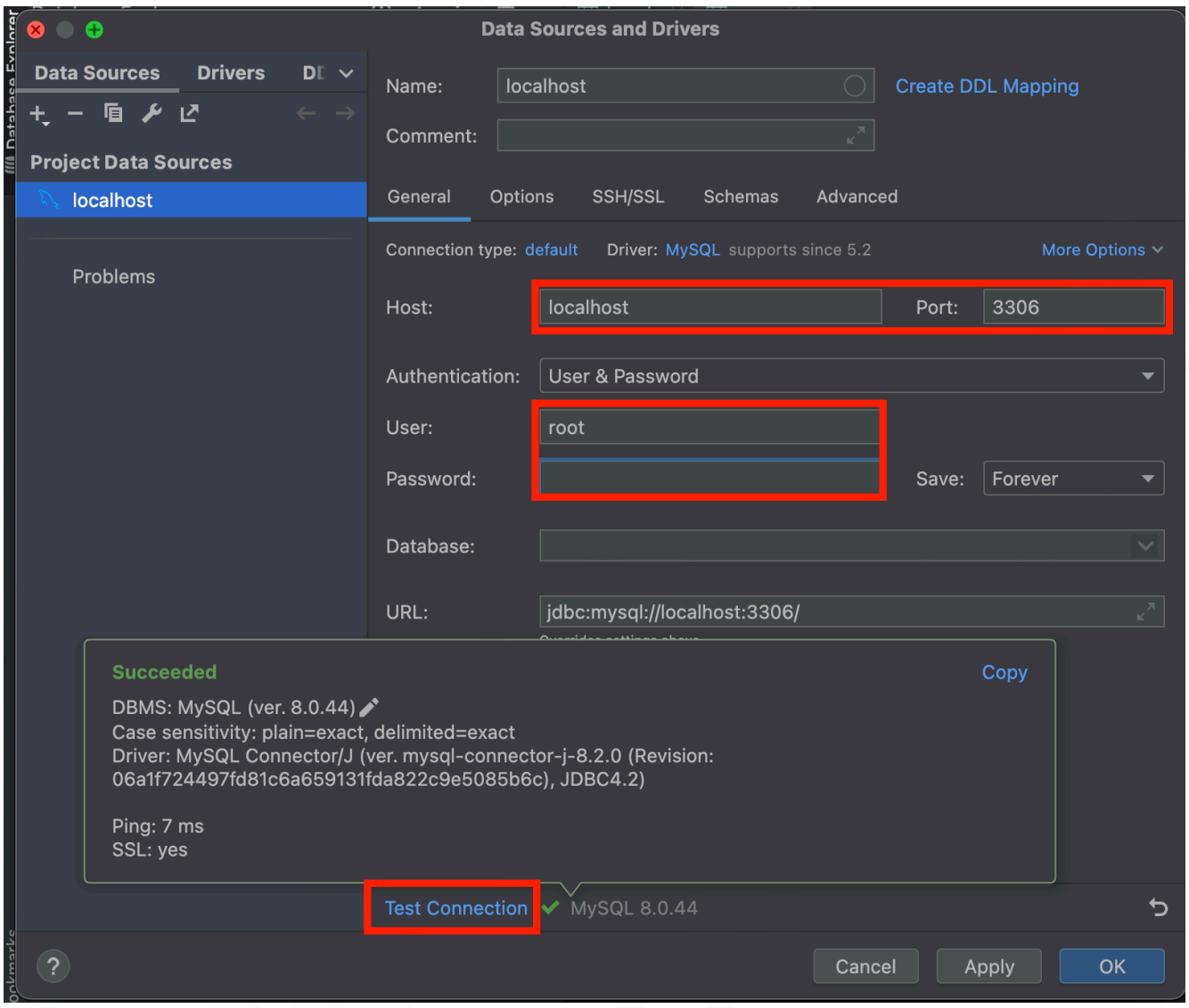
1. **Docker로 MySQL 띄우기**

   DB에 직접 수많은 쓰기 작업(좋아요 데이터 삽입)의 쿼리를 날리면 얼마나 느린지 눈으로 확인하려고 한다. 하지만 여러분이 쓰고 있는 노트북은 대부분 성능이 너무 좋다. 그래서 Docker를 활용해 DB가 쓸 수 있는 성능을 제한했다.

   ```
   $ docker run -d \
   -e MYSQL_DATABASE=mydb \
   -e MYSQL_ROOT_PASSWORD=password \
   -p 3306:3306 \
   --cpus="0.2" \
   --memory="256m" \
   mysql:8.0
   ```

2. **MySQL 실행 잘 됐는 지 Datagrip에 연결시켜보기**

khb3023@gmail.com



3. **Spring Boot 프로젝트 클론받기**

> 빠른 실습을 위해서 Spring Boot 프로젝트의 뼈대를 이미 만들어놨다.

GitHub - JSCODE-COURSE/redis-intermediate-project: 비전공자도 이해할 수 있는 Redis 중급/실전
비전공자도 이해할 수 있는 Redis 중급/실전. Contribute to JSCODE-COURSE/redis-intermediate-project development by creating an account on GitHub.
https://github.com/JSCODE-COURSE/redis-intermediate-project

JSCODE-COURSE/**redis-intermediate-project**
비전공자도 이해할 수 있는 Redis 중급/실전
1 Contributor · 0 Issues · 0 Stars · 0 Forks

- Spring Data JPA, Spring Data Redis 활용

  build.gradle

```
...

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-data-redis'
    compileOnly 'org.projectlombok:lombok'
    runtimeOnly 'com.mysql:mysql-connector-j'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-data-jpa-test'
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

...
```

- JPA의 설정으로 서버 실행 때마다 DB 초기화되도록 설정

application.yml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  data:
    redis:
      host: localhost
      port: 6379
  jpa:
    hibernate:
      ddl-auto: create
    show-sql: true
```

4. **좋아요 API 로직 살펴보기**

Like

```
@Entity(name = "likes")
@Getter
@NoArgsConstructor
public class Like {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private Long userId;

  private Long postId;

  public Like(Long userId, Long postId) {
    this.userId = userId;
    this.postId = postId;
  }
}
```

LikeRepository

```
public interface LikeRepository extends JpaRepository<Like, Long> {
}
```

LikeController

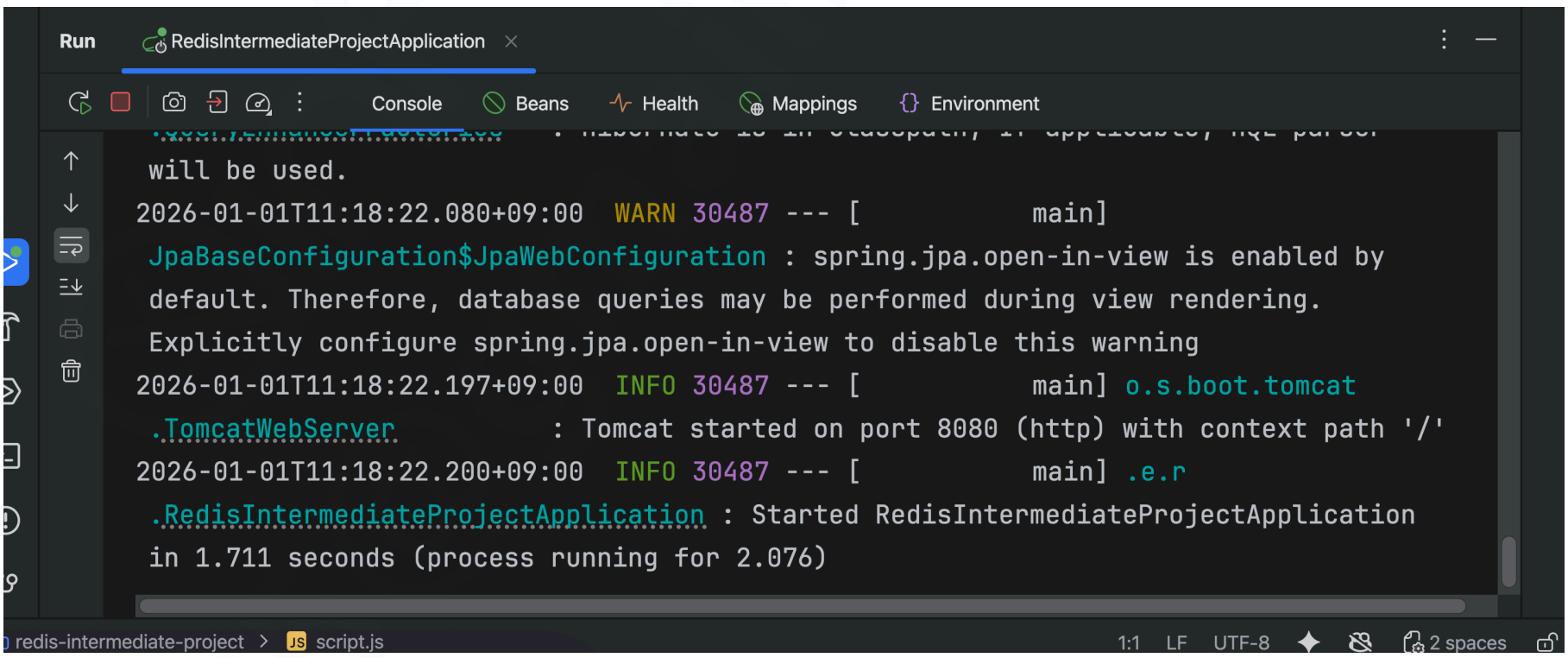
```
@RestController
@RequiredArgsConstructor
@RequestMapping("/likes")
public class LikeController {

    ...

    @PostMapping()
    public void likePost(
        @RequestBody LikePostRequestDto likePostRequestDto
```

```
    ) {
        likeService.likePost(likePostRequestDto);
    }
    ...
}
```

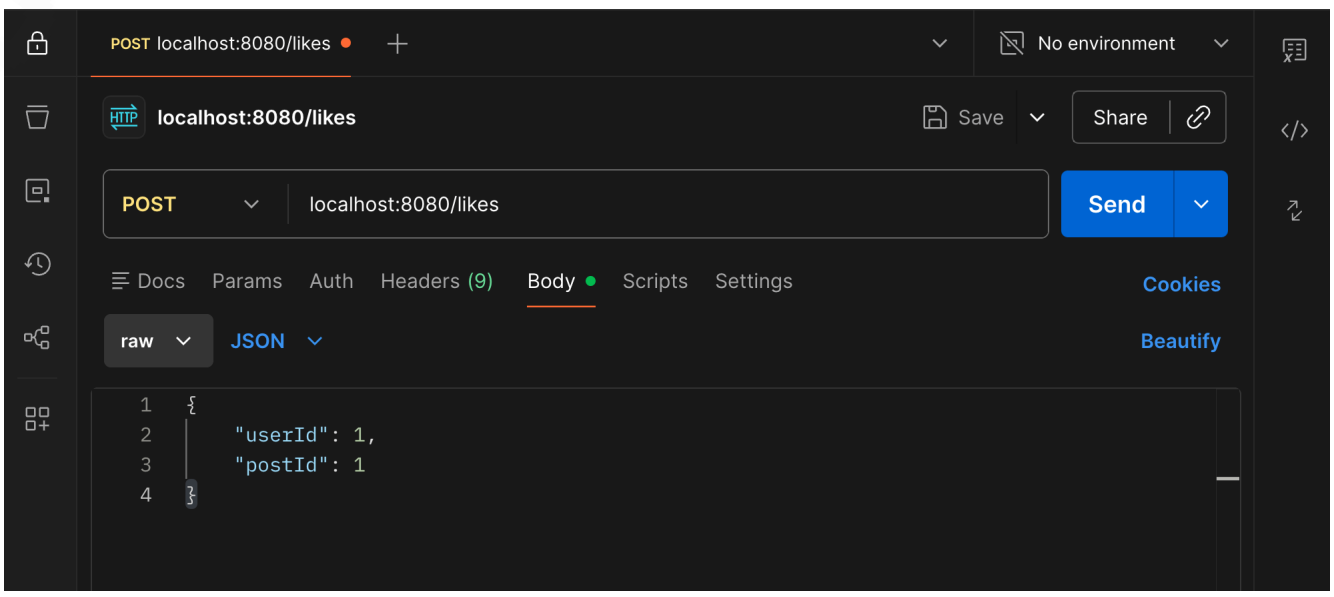
LikeService

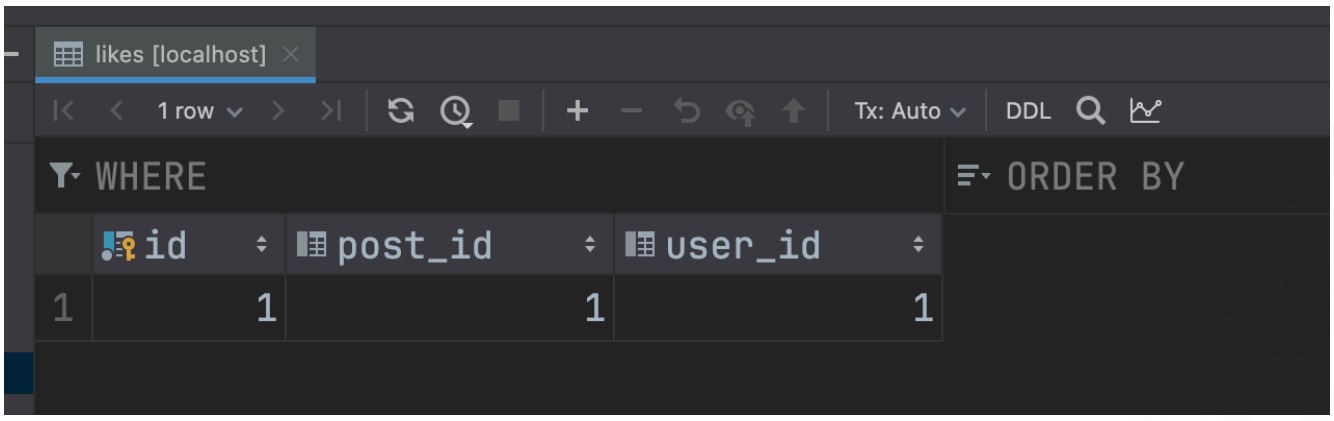
```
@Service
@RequiredArgsConstructor
public class LikeService {
    ...

    public void likePost(LikePostRequestDto likePostRequestDto) {
        Long userId = likePostRequestDto.getUserId();
        Long postId = likePostRequestDto.getPostId();
        Like like = new Like(userId, postId);
        likeRepository.save(like);
    }
        ...
}
```

5. **Spring Boot 서버 실행시키기**



5. **Postman으로 좋아요 API가 잘 작동하는 지 확인하기**





6. **k6 부하테스트 스크립트 실행시키기**

> k6가 설치되어 있다는 가정하에 진행한다.

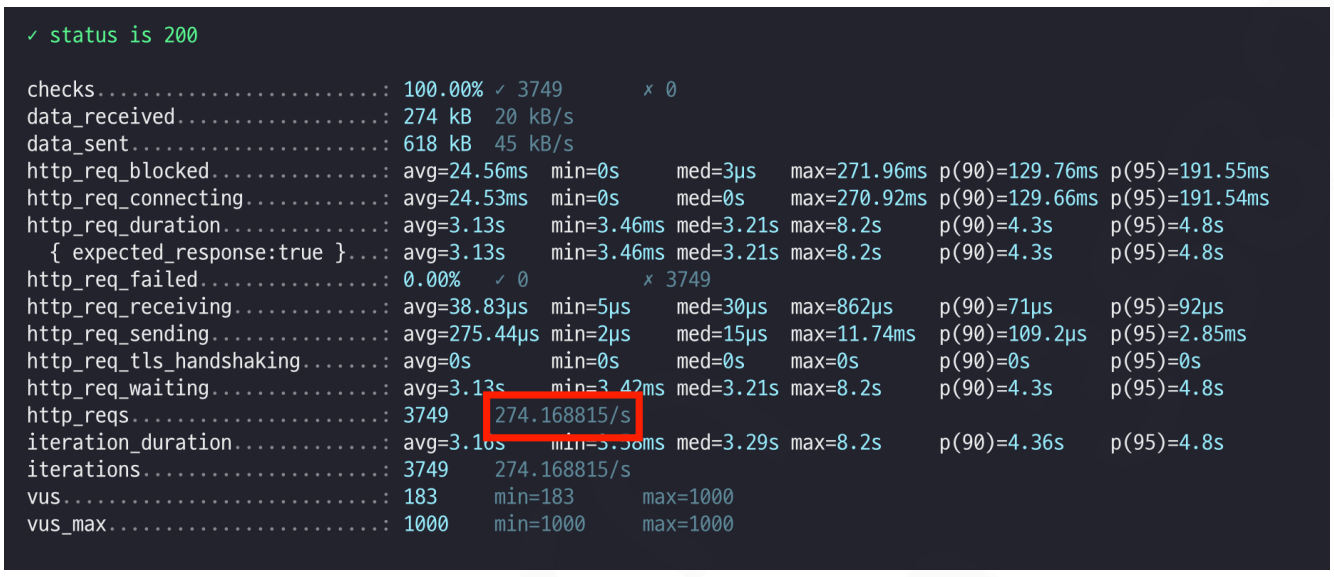
클론받은 Spring Boot 프로젝트의 루트 디렉토리에 보면 `scripts/script_1-1.js` 파일을 이미 만들어뒀다.

scripts/script_1-1.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { randomIntBetween } from 'https://jslib.k6.io/k6-utils/1.2.0/index.js';

export const options = {
  // 가상 유저(VUs) 1000명으로 설정
  vus: 1000,

  // 테스트를 10초 동안 진행
  duration: '10s',
};

export default function () {
    // 랜덤 데이터 생성
  // 유저는 1~10,000명, 게시글은 1~100개라고 가정
  const userId = randomIntBetween(1, 10000);
  const postId = randomIntBetween(1, 100);

  // 좋아요 API
  const url = 'http://localhost:8080/likes';
  const payload = JSON.stringify({
    userId: userId,
    postId: postId,
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
    },
  };

  // POST 요청 전송
  const res = http.post(url, payload, params);

  // 응답 상태 코드 확인 (200 OK가 왔는 지 체크)
  check(res, {
    'status is 200': (r) => r.status === 200,
  });
}
```

Spring Boot의 프로젝트 경로에서 아래 명령어를 실행시키자.

```
$ k6 run scripts/script_1-1.js
```



가상 유저를 1000명으로 설정해서 동시에 1000명이 좋아요 API를 요청하도록 설정했음에도 불구하고, DB가 들어오는 요청을 다 처리하지 못해 TPS는 274 밖에 안 나오는 걸 확인할 수 있다. 즉, 1초당 274개를 넘는 좋아요 요청이 들어오면, 처리하지 못하는 요청이 계속 쌓이면서 문제가 생길 수 있다.

이 문제를 해결하기 위해 DB 사양을 높이는 것도 방법이 될 수 있지만, 비싼 데이터베이스의 사양을 높이는 건 부담스럽기도 하고 데이터베이스 사양을 높이는 데에도 한계가 있다. 따라서 Redis를 활용해 이 문제를 해결해보자.

kkb3025@gmail.com

[실습] 좋아요 수 폭증으로 인한 DB 부하를 Redis로 해결하기 - 2

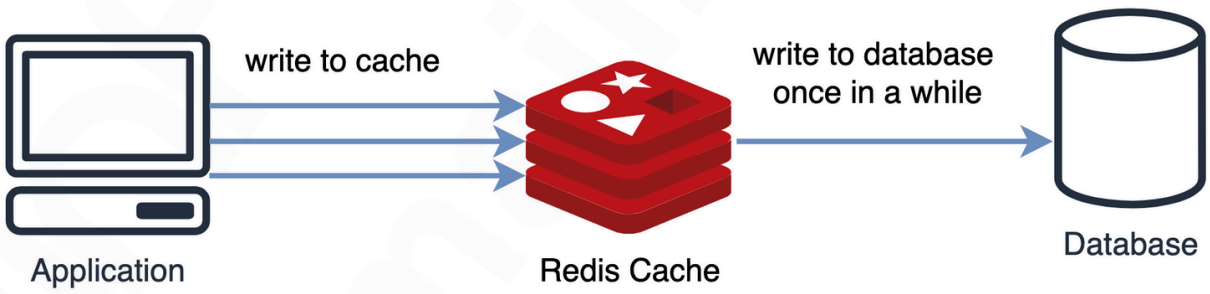
✅ 문제 원인

누군가 좋아요를 한 번 누를 때마다 DB에 계속해서 요청을 보낸다. 만약 1000명의 사용자가 글을 보고 좋아요를 누르면 1000번의 DB 쓰기 작업을 해야 한다. DB 부하의 원인은 크게 2가지이다.

1. 한 번의 요청이 일어날 때마다 매번 DB에 요청을 보내야 한다.
2. DB는 디스크에 데이터를 기록하기 때문에 비교적 속도가 느리다.

✅ 해결책

Write Back 전략을 활용하면 된다. Write Back 전략이란 클라이언트의 요청이 들어오면 먼저 Redis에 데이터를 쌓아두고 클라이언트에게 먼저 응답을 반환하고나서, Redis에 쌓인 데이터를 스케줄러가 주기적으로 DB에 한꺼번에 저장하는 방식을 뜻한다. 즉, Redis가 데이터를 쌓아두는 Queue의 역할을 한다.



✅ 실습

1. **Redis에 데이터를 쌓아두는 로직 추가하기**

   클라이언트이 요청이 들어오면 Redis에 데이터를 쌓아두고, 클라이언트에 먼저 빠르게 응답을 반환하도록 코드를 추가해보자.

LikeService

```
@Service
@RequiredArgsConstructor
public class LikeService {
    private final LikeRepository likeRepository;
    private final RedisTemplate<String, String> redisTemplate;

        ...

    public void likePostWithRedis(LikePostRequestDto likePostRequestDto) {
        // Redis에 넣을 value 값 생성
        // (원래는 데이터를 JSON으로 직렬화를 많이 하는 편이지만, 편의상 간단한 문자열 조합으로 대체)
        Long userId = likePostRequestDto.getUserId();
        Long postId = likePostRequestDto.getPostId();
        String value = userId + ":" + postId;
```

```
        // like_queue라는 List에 데이터를 오른쪽으로 Push해서 넣는다.
        // Redis 명령어에서 'RPUSH like_queue [value]'와 동일하게 작동한다.
        redisTemplate.opsForList().rightPush("like_queue", value);
    }
}
```

LikeController

```
@RestController
@RequiredArgsConstructor
@RequestMapping("/likes")
public class LikeController {

    private final LikeService likeService;

    ...

      @PostMapping("/redis")
    public void likePostWithRedis(
        @RequestBody LikePostRequestDto likePostRequestDto
    ) {
        likeService.likePostWithRedis(likePostRequestDto);
    }
}
```

2. **스케줄러가 주기적으로 DB에 한꺼번에 저장하도록 코드 작성하기**

   RedisIntermediateProjectApplication

```
@EnableScheduling
@SpringBootApplication
public class RedisIntermediateProjectApplication {

  public static void main(String[] args) {
    SpringApplication.run(RedisIntermediateProjectApplication.class, args);
  }
}
```

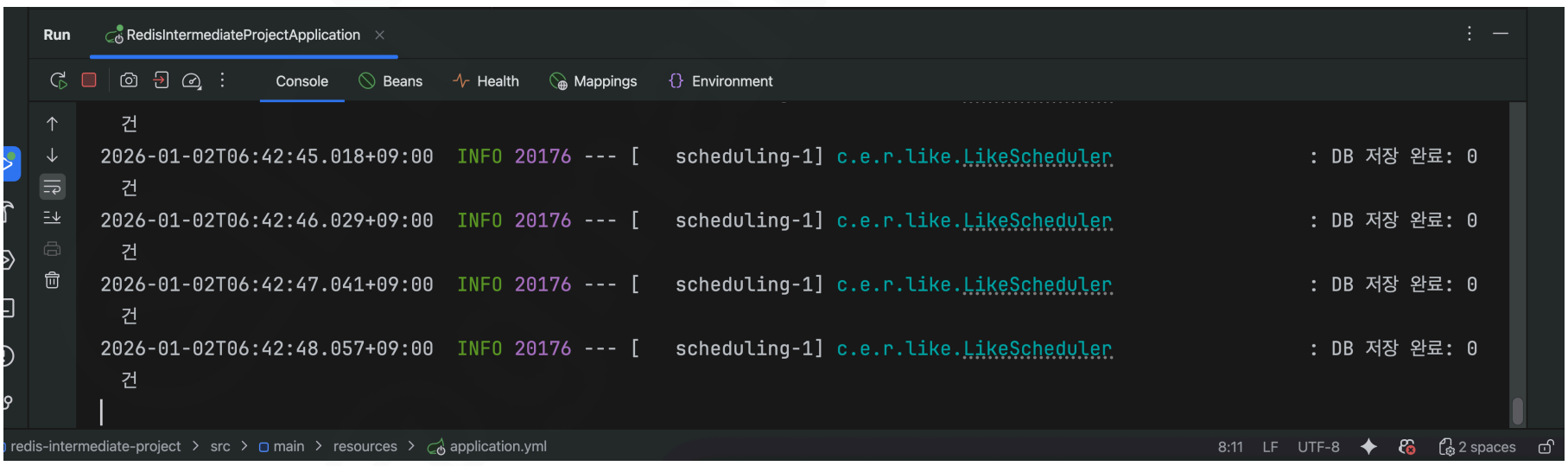
LikeScheduler

```
@Component
@RequiredArgsConstructor
@Slf4j
public class LikeScheduler {

  private final RedisTemplate<String, String> redisTemplate;
  private final LikeRepository likeRepository;

  // 1초마다 실행 (실무에서는 데이터 양에 따라 조절)
  @Scheduled(fixedDelay = 1000)
  public void saveLikesToDb() {
    // 1. Redis에서 데이터를 1000개 꺼내오기
    // (큐에 있는 데이터를 하나씩 꺼내서 리스트에 담기)
```

```
    List<Like> likesToSave = new ArrayList<>();
    while (true) {
      // like_queue라는 List에서 왼쪽(첫 부분)에서 데이터를 꺼내온다.
        // Redis 명령어에서 'LPOP like_queue'와 동일하게 작동한다.
      String value = redisTemplate.opsForList().leftPop("like_queue");
      if (value == null) break;

      String[] split = value.split(":");
      Long userId = Long.parseLong(split[0]);
      Long postId = Long.parseLong(split[1]);

      likesToSave.add(new Like(userId, postId));

      if (likesToSave.size() >= 1000) break;
    }

        likeRepository.saveAll(likesToSave);
    log.info("DB 저장 완료: {} 건", likesToSave.size());
      }
    }
```

참고) `saveAll()` 대신에 `jdbcTemplate`의 `batchUpdate` 쿼리를 사용하면 더 효율화를 시킬 수 있다. 하지만 이번 실습에서는 간단하게 테스트를 하기 위해 `saveAll()`을 사용했다.

3. **Spring Boot 서버 재실행시키기**



4. **부하테스트 스크립트 살펴보기**

   이전에 실행시켰던 스크립트(`script_1-1.js`)와 코드가 대부분 똑같고, 새로 만든 API에 맞게 주소만 변경해주었다.

   scripts/script_1-2.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { randomIntBetween } from 'https://jslib.k6.io/k6-utils/1.2.0/index.js';

export const options = {
  // 가상 유저(VUs) 1000명으로 설정
  vus: 1000,

  // 테스트를 10초 동안 진행
  duration: '10s',
};

export default function () {
```

```
// 랜덤 데이터 생성
// 유저는 1~10,000명, 게시글은 1~100개라고 가정
const userId = randomIntBetween(1, 10000);
const postId = randomIntBetween(1, 100);

// 좋아요 API
const url = 'http://localhost:8080/likes/redis';
const payload = JSON.stringify({
  userId: userId,
  postId: postId,
});

const params = {
  headers: {
    'Content-Type': 'application/json',
  },
};

  // POST 요청 전송
  const res = http.post(url, payload, params);

  // 응답 상태 코드 확인 (200 OK가 왔는 지 체크)
  check(res, {
    'status is 200': (r) => r.status === 200,
  });
}
```

4. **k6 부하테스트 스크립트 실행시키기**

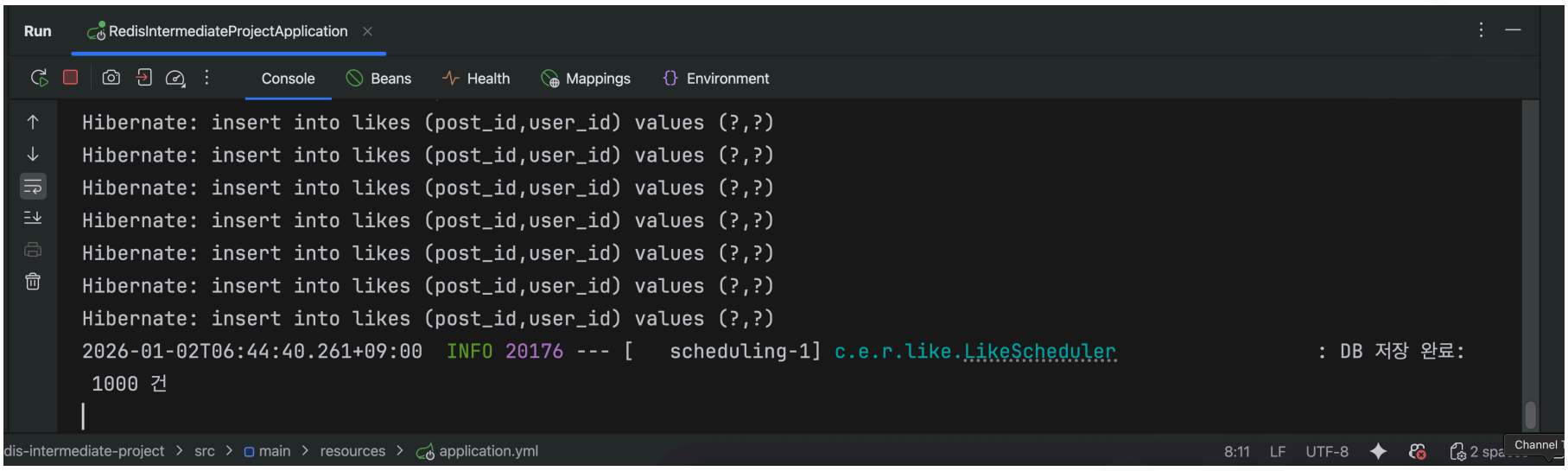
   Spring Boot의 프로젝트 경로에서 아래 명령어를 실행시키자.

```
$ k6 run scripts/script_1-2.js
```

```
✓ status is 200

checks.........................: 100.00% ✓ 303256       ✗ 0
data_received..................: 22 MB   2.2 MB/s
data_sent......................: 52 MB   5.2 MB/s
http_req_blocked...............: avg=770.75µs min=0s      med=1µs     max=1.12s    p(90)=1µs     p(95)=2µs
http_req_connecting............: avg=769.39µs min=0s      med=0s      max=1.11s    p(90)=0s      p(95)=0s
http_req_duration..............: avg=32.18ms  min=161µs   med=29.48ms max=218.89ms p(90)=44.19ms p(95)=54.94ms
  { expected_response:true }...: avg=32.18ms  min=161µs   med=29.48ms max=218.89ms p(90)=44.19ms p(95)=54.94ms
http_req_failed................: 0.00%   ✓ 0            ✗ 303256
http_req_receiving.............: avg=9.51µs   min=2µs     med=5µs     max=3.45ms   p(90)=14µs    p(95)=20µs
http_req_sending...............: avg=5.14µs   min=1µs     med=2µs     max=4.25ms   p(90)=7µs     p(95)=10µs
http_req_tls_handshaking.......: avg=0s       min=0s      med=0s      max=0s       p(90)=0s      p(95)=0s
http_req_waiting...............: avg=32.17ms  min=155µs   med=29.47ms max=216.7ms  p(90)=44.17ms p(95)=54.93ms
http_reqs......................: 303256  30247.579394/s
iteration_duration.............: avg=32.99ms  min=178.5µs med=29.52ms max=1.18s    p(90)=44.24ms p(95)=55.08ms
iterations.....................: 303256  30247.579394/s
vus............................: 1000    min=1000       max=1000
vus_max........................: 1000    min=1000       max=1000
```

부하 테스트를 진행해보면 1초에 약 30,000개의 요청을 처리하는 걸 확인할 수 있다. DB에 바로 저장하는 것이 아닌 Redis에 데이터를 먼저 쌓아두는 형식이라 TPS(1초에 처리할 수 있는 요청의 수)가 확 늘었다.

그러고 Redis에 쌓아둔 데이터는 스케쥴러를 활용해 1000개씩 DB에 업데이트를 시키는 방식으로 처리한다.



이런 구조로 인해 갑자기 폭증하는 트래픽을 DB가 직접적으로 받지 않고, Redis에 폭증한 트래픽을 쌓아두고 DB가 스케쥴러에 의해 차근차근 데이터를 처리하게끔 작동한다. 이로 인해 한 번에 몰려드는 트래픽으로부터 DB의 부하를 방지할 수 있으면서도, 클라이언트한테는 빠르게 응답할 수 있는 이점을 동시에 가져갈 수 있다.

> 이로써 좋아요 수 폭증으로 인한 DB 부하를 Redis로 해결했다.

@naver.com
khb3025

📖

String 자료 구조 (SET, GET, DEL, SETNX)

✅ String 자료 구조의 SETNX

```
# key, value 구조로 데이터 저장하기
$ SET hobby soccer

# key로 value 조회하기
$ GET hobby

# 기존에 hobby라는 key가 존재하더라도, 해당 key에 value를 덮어씌워서 데이터를 저장한다
$ SET hobby baseball

# value가 바뀌었는 지 확인해보기
$ GET hobby

# 지정한 키가 없을 때에만 데이터를 저장
# SET [key] [value] NX
$ SET hobby golf NX

# value가 바뀌었는 지 확인해보기
$ GET hobby

# key로 데이터 삭제하기
$ DEL hobby

# 지정한 키가 없을 때에만 데이터를 저장
$ SET hobby golf NX

# key, value가 잘 저장됐는 지 확인해보기
$ GET hobby
```

요약) SET과 NX 옵션을 함께 사용하면 기존에 key가 없을 때만 데이터를 저장한다.

khb302@gmail.com

🎯

[실습] 재고를 차감할 때 동시성 이슈로 인한 오차가 발생하지 않도록 만들기 - 1

✅ 문제 상황

100명의 사용자가 하나의 상품을 1개씩 구매하면, 이 상품의 재고는 정확하게 100개가 차감되는 게 당연하다. 그런데 실제 코드를 실행시켜보면 오차가 발생하는 경우가 많다. 한 마디로, 재고가 100개가 차감되어야 하는데 95개, 92개 이런식으로 차감되는 상황이 발생한다는 뜻이다. 정말 이런 일이 일어나는 지 실습을 해보자.

✅ 실습 진행하기

1. **이전 스케줄러 로직 비활성화 해놓기**

   콘솔창에 로그가 계속 찍히는 걸 방지하기 위해, 이전 실습에서 활성화 시켜둔 스케줄러 로직을 비활성화 해두자.

   RedisIntermediateProjectApplication

```
@EnableScheduling
@SpringBootApplication
public class RedisIntermediateProjectApplication {

  public static void main(String[] args) {
    SpringApplication.run(RedisIntermediateProjectApplication.class, args);
  }
}
```

2. **재고 차감 API 살펴보기**

   > 효율적인 실습을 위해 재고 차감 API를 미리 만들어뒀다.

   Stock

```
@Entity(name = "stocks")
@Getter
@NoArgsConstructor
public class Stock {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private Long quantity;

  public Stock(Long quantity) {
    this.quantity = quantity;
  }

  public void decrease(Long quantity) {
    if (this.quantity - quantity < 0) {
      throw new RuntimeException("재고는 0개 미만이 될 수 없습니다.");
    }
    this.quantity -= 1;
```

```
  }
}
```

StockController

```
@RestController
@RequestMapping("/stocks")
@RequiredArgsConstructor
public class StockController {

  private final StockService stockService;

  @PostMapping("/{id}/decrease")
  public void decrease(
      @PathVariable Long id
  ) {
    stockService.decrease(id);
  }
}
```

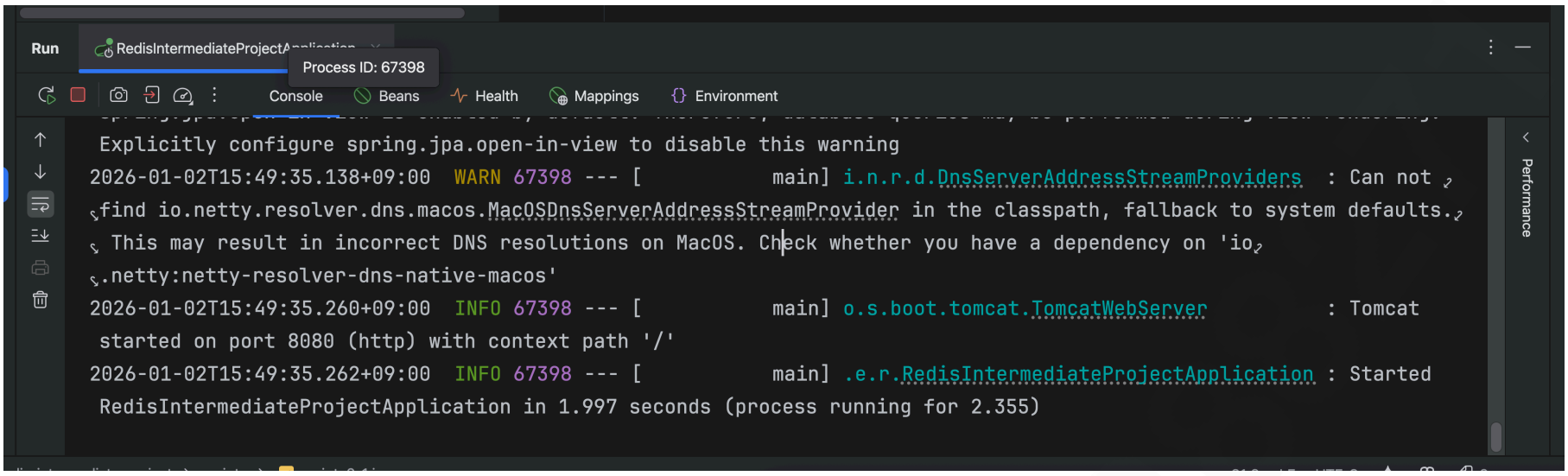
StockService

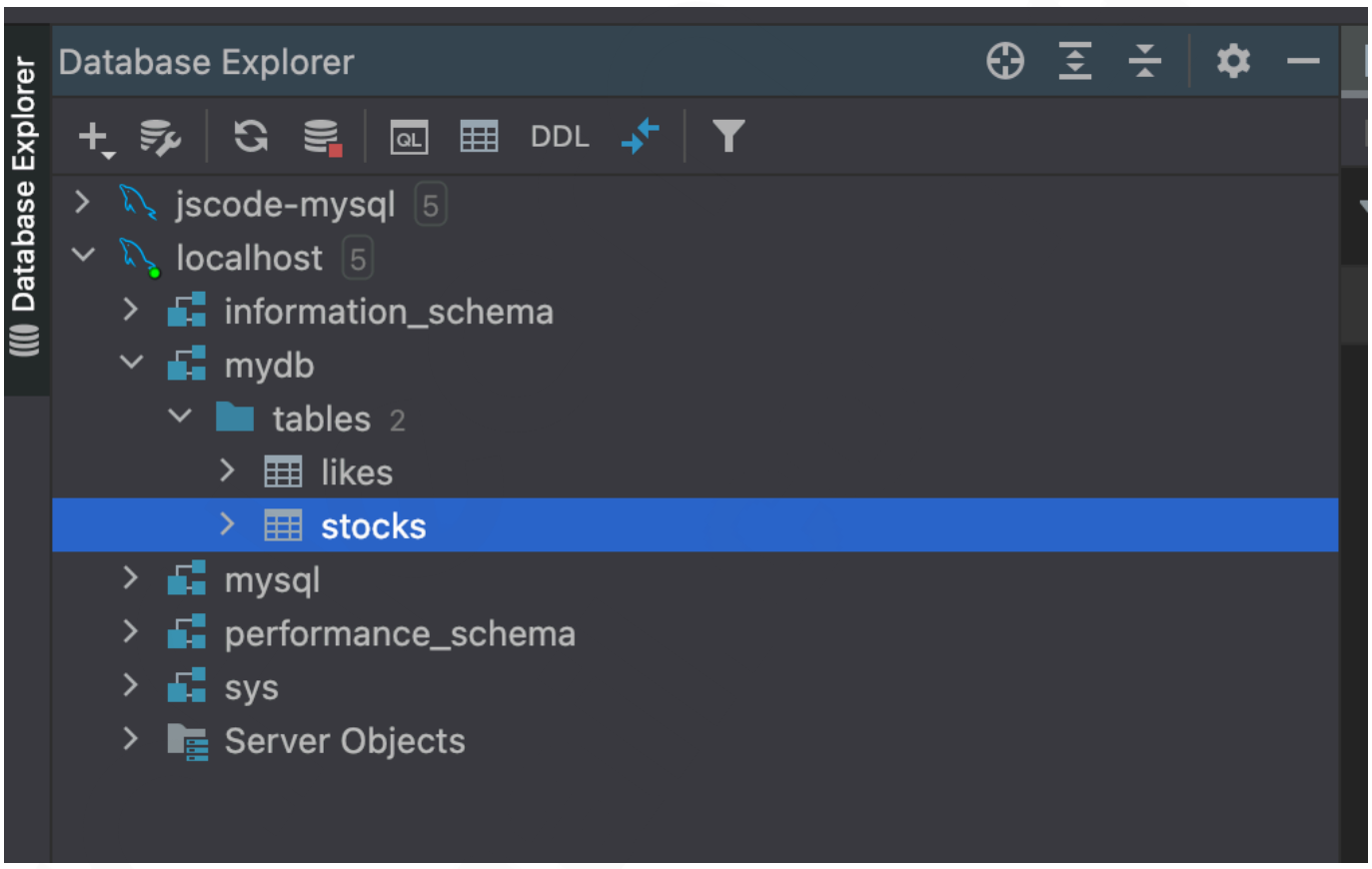
```
@Service
@RequiredArgsConstructor
public class StockService {

  private final StockRepository stockRepository;

  @Transactional
  public void decrease(Long id) {
    Stock stock = stockRepository.findById(id).orElseThrow();
    stock.decrease(id);
    stockRepository.save(stock);
  }
}
```

StockRepository

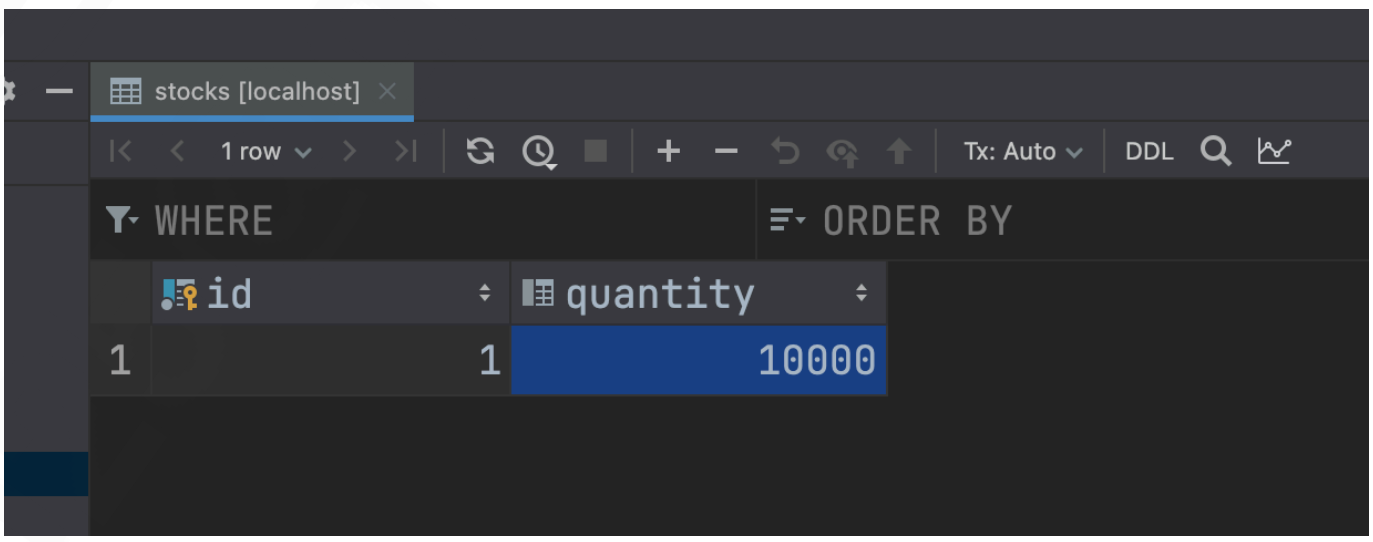
```
public interface StockRepository extends JpaRepository<Stock, Long> {
}
```

3. **Spring Boot 실행시켜서 DB 동기화시키기**





4. **테스트를 위해 DB에 데이터 넣어주기**



2. **부하 테스트 스크립트 살펴보기**

scripts/script_2-1.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  // 가상 유저(VUs) 100명으로 설정
  vus: 100,
```

```
  // 테스트를 10초 동안 진행
  duration: '10s',
};

export default function () {
  // 재고 차감 API
  const url = 'http://localhost:8080/stocks/1/decrease';

  const params = {
    headers: {
      'Content-Type': 'application/json',
    },
  };

  // POST 요청 전송
  const res = http.post(url, null, params);

  // 1초 간격으로 요청 발송
  sleep(1);

  // 응답 상태 코드 확인 (200 OK가 왔는 지 체크)
  check(res, {
    'status is 200': (r) => r.status === 200,
  });
}
```

5. **부하 테스트 진행하기**

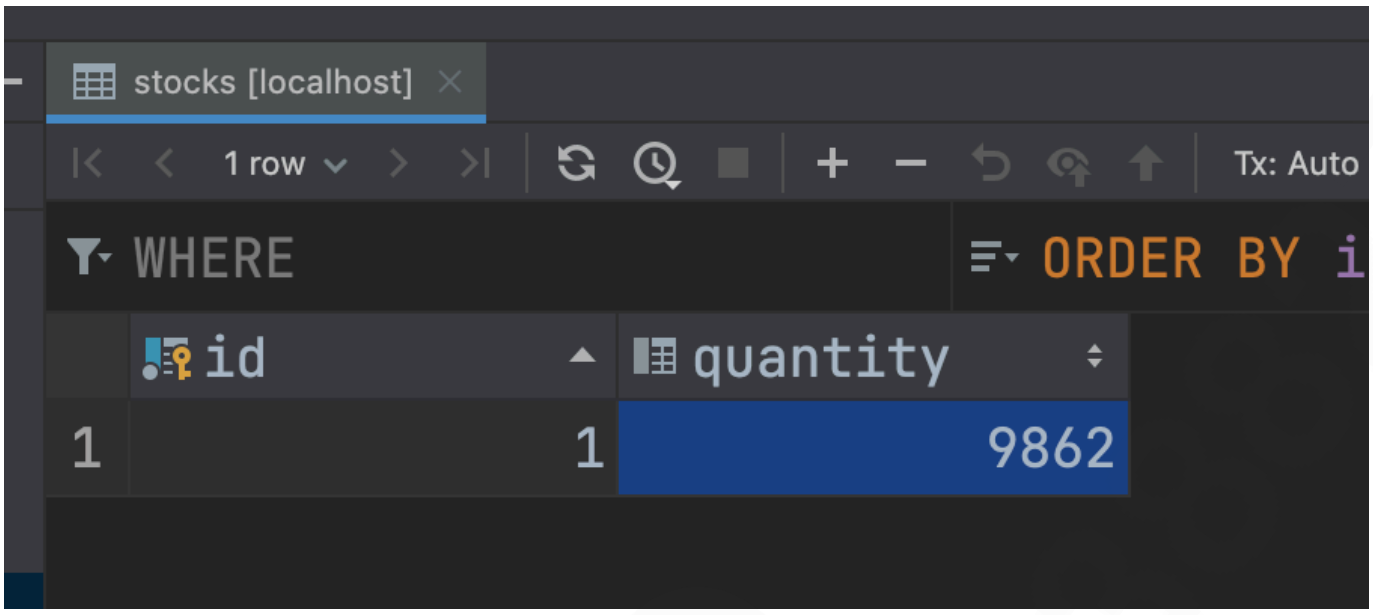
```
$ k6 run scripts/script_2-1.js
```

```
✓ status is 200

checks.........................: 100.00% ✓ 1000      ✗ 0
data_received..................: 73 kB   6.7 kB/s
data_sent......................: 149 kB  14 kB/s
http_req_blocked...............: avg=195.23µs min=1µs    med=5µs     max=3.42ms   p(90)=123.6µs  p(95)=1.98ms
http_req_connecting............: avg=138.08µs min=0s     med=0s      max=1.81ms   p(90)=47.5µs   p(95)=1.5ms
http_req_duration..............: avg=71.15ms  min=3.27ms med=12.49ms max=778.13ms p(90)=112.88ms p(95)=484.92ms
  { expected_response:true }...: avg=71.15ms  min=3.27ms med=12.49ms max=778.13ms p(90)=112.88ms p(95)=484.92ms
http_req_failed................: 0.00%   ✓ 0         ✗ 1000
http_req_receiving.............: avg=31.44µs  min=9µs    med=26µs    max=152µs    p(90)=56µs     p(95)=71µs
http_req_sending...............: avg=22.93µs  min=4µs    med=17µs    max=452µs    p(90)=36.1µs   p(95)=44µs
http_req_tls_handshaking.......: avg=0s       min=0s     med=0s      max=0s       p(90)=0s       p(95)=0s
http_req_waiting...............: avg=71.1ms   min=3.25ms med=12.45ms max=777.81ms p(90)=112.79ms p(95)=484.88ms
http_reqs......................: 1000    91.593024/s
iteration_duration.............: avg=1.07s    min=1s     med=1.01s   max=1.77s    p(90)=1.11s    p(95)=1.48s
iterations.....................: 1000    91.593024/s
vus............................: 100     min=100     max=100
vus_max........................: 100     min=100     max=100
```

1000번의 요청을 처리했음을 알 수 있고, 91 TPS 정도의 성능이 나오는 걸 알 수 있다.

6. **DB 확인하기**

1000번의 요청을 처리했기 때문에 10,000개의 재고에서 1,000개의 재고가 차감돼서 남아있는 재고가 9,000개여야 정상이다. 확인해보자.



DB를 확인해봤더니 재고가 138개 밖에 차감되지 않았다. 왜 이런 일이 발생한걸까? 다음 강의에서 이 문제의 원인에 대해 살펴보자.

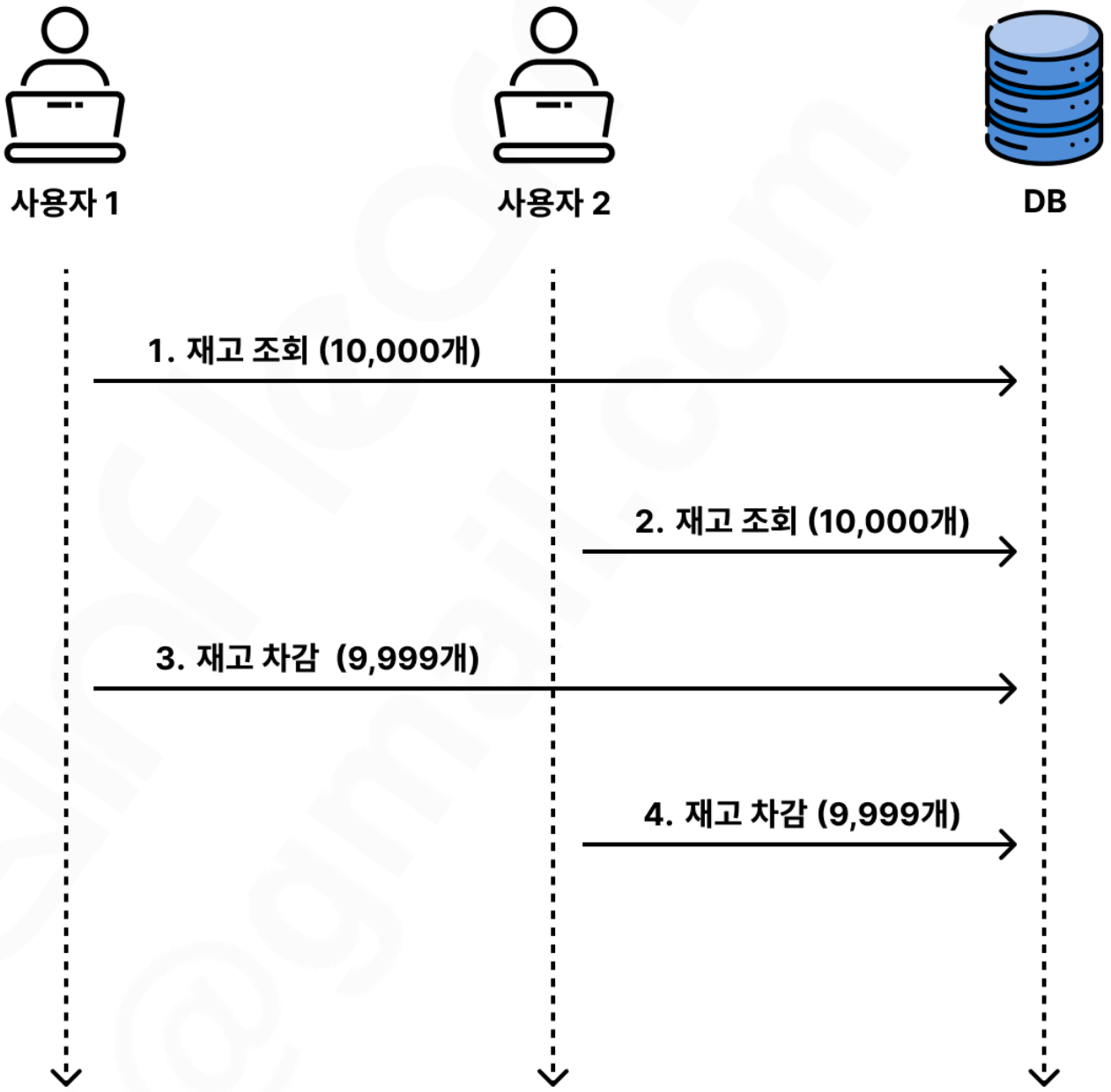
@gmail.com
khb3025

🎯

[실습] 재고를 차감할 때 동시성 이슈로 인한 오차가 발생하지 않도록 만들기 - 2

✅ 원인 파악

이전 강의에서 실제 테스트를 해보니 원래 차감되어야 하는 개수보다 적게 차감이 됐다. 한 마디로 재고를 차감할 때 오차가 발생했다. 실제 서비스에서는 이런 문제가 절대 발생하면 안 된다. 이 문제를 해결하기 위해 문제의 원인을 살펴보자.



재고 차감 API에 요청을 보낸 사용자가 두 명 있다고 가정하자. 그런데 이 사용자들이 거의 동시에 요청을 보낸 것이다. DB가 들어온 요청을 하나씩 순서대로 처리하는 방식이 아닌, 병렬적으로 동시에 여러 요청을 처리할 수 있게끔 설계가 되어있다.

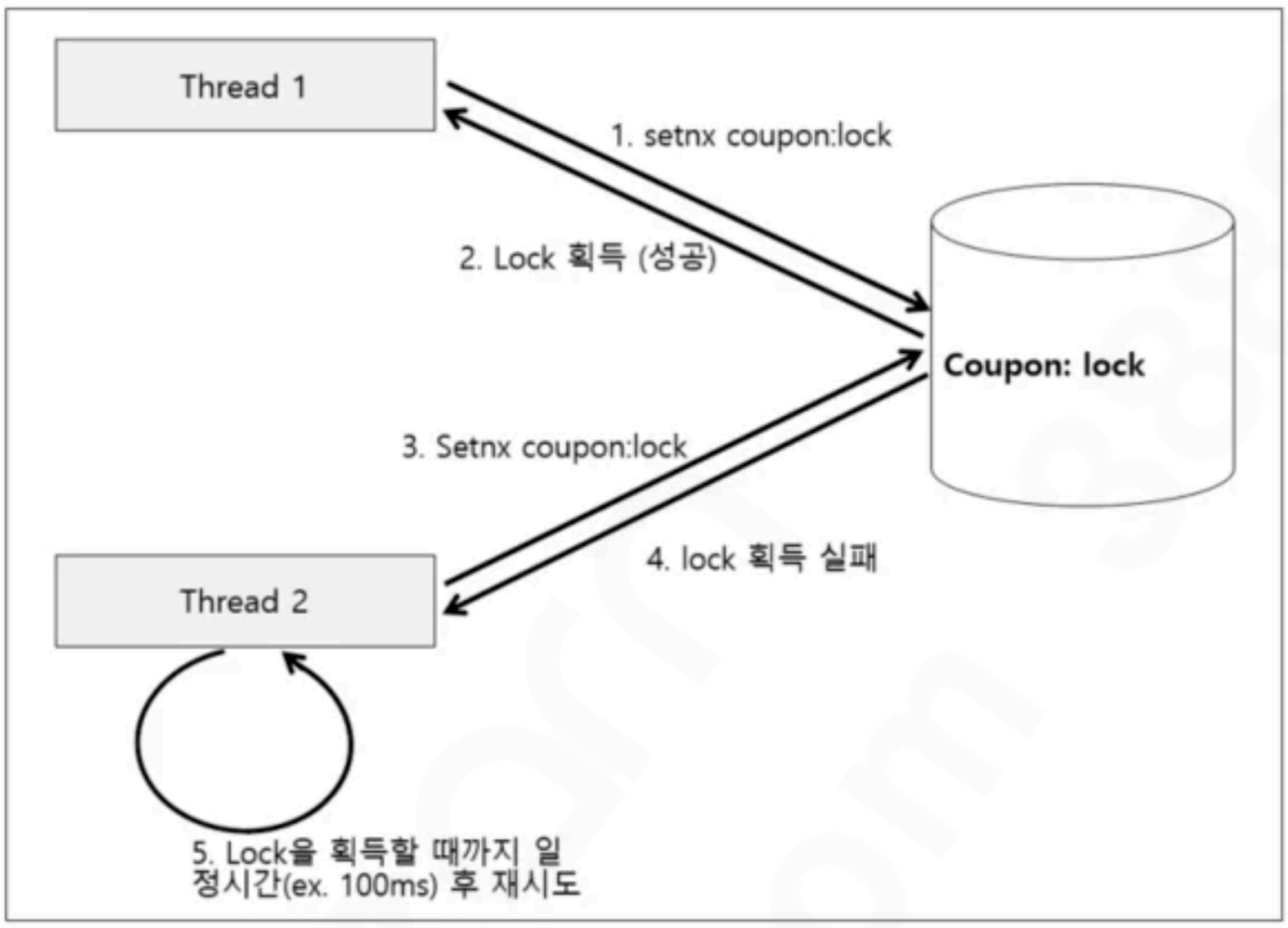
그래서 사용자 1이 보낸 요청을 처리하기 위해 (재고 조회 → 재고 차감)의 로직을 수행하는 동안, 사용자 2가 보낸 요청을 처리하기 위해 (재고 조회 → 재고 차감)의 로직을 병렬적으로 수행한다. 문제는 이 때 발생한다.

사용자 1이 재고를 차감하기 전에 현재 재고가 얼마나 있는 지 조회(10,000개)한다. 그러고 사용자 2도 거의 동시에 요청을 보냈기 때문에 현재 재고가 얼마나 있는 지 조회(10,000개)를 한다. 그런 뒤에 각각 사용자가 현재 재고를 기준으로 1씩 차감한 뒤에 DB에 반영한다. 하지만 각 사용자가 조회한 재고의 개수는 둘 다 10,000개이기 때문에, 두 사용자 전부 DB에 재고를 UPDATE 할 때 9,999개로 업데이트(`UPDATE stocks SET quantity = 9999 WHERE id = 1`)한다.

이로 인해 2명의 사용자가 재고 차감 API에 요청을 보냈기 때문에 재고가 9,998개가 돼야함에도 불구하고, 재고가 9,999개가 됐던 것이다.

> 이런 문제를 보고 '거의 동시에 요청을 보냈을 때 발생하는 문제'라고 해서 '**동시성 이슈**'라고 부른다.

✅ 해결책



이 문제를 해결하는 방법은 간단하다. 들어온 요청을 병렬적으로 처리하게 하지 말고, 하나씩 순서대로 처리하게 하면 된다. 즉, 첫 번째 사용자의 요청을 다 처리한 뒤에 두 번째 사용자의 요청을 처리하는 식으로 설계를 하는 것이다.

들어온 요청을 병렬적으로 처리하지 않고, 하나씩 순서대로 처리하게끔 만드는 방법에는 여러가지가 있는 데 그 중에서 **Redis로 Lock을 구현하는 방식**으로 해결해볼 것이다. 여기서 Lock이라 함은 특정 데이터에 대해 동시에 접근하지 못하도록 '잠금'을 거는 장치를 의미한다.

비유를 하자면, 한 화장실에 여러 명이 동시에 들어갈 수 없게 만드는 것과 같다. 누군가 화장실을 사용하고 있으면 문을 잠그고, 다른 사람들은 문이 열릴 때까지 밖에서 기다려야 한다. 화장실 사용이 끝나고 문을 열면 그 다음 사람이 들어갈 수 있다.

재고 차감도 마찬가지다. 어떤 사용자가 재고 차감 API 요청을 처리하고 있다면, 그 처리가 끝날 때까지 다른 사용자는 해당 재고 데이터에 접근하지 못하도록 막아야 한다. 이때 사용하는 '문 잠금 장치' 역할을 하는 것이 바로 Redis의 Lock이다. 즉, **Redis의 Lock**은 **여러 요청이 동시에 하나의 자원을 수정하려 할 때, 이를 순서대로 처리하도록 강제하는 장치**이다.

> 다음 강의에서는 Redis로 Lock을 구현해 동시성 이슈를 해결해보자.

khb3023

🎯

[실습] 재고를 차감할 때 동시성 이슈로 인한 오차가 발생하지 않도록 만들기 - 3

👨🏻‍🏫 이번 강의에서는 Redis로 Lock을 구현해 동시성 이슈를 해결해보자.

✅ 문제 해결

1. **코드 작성하기**

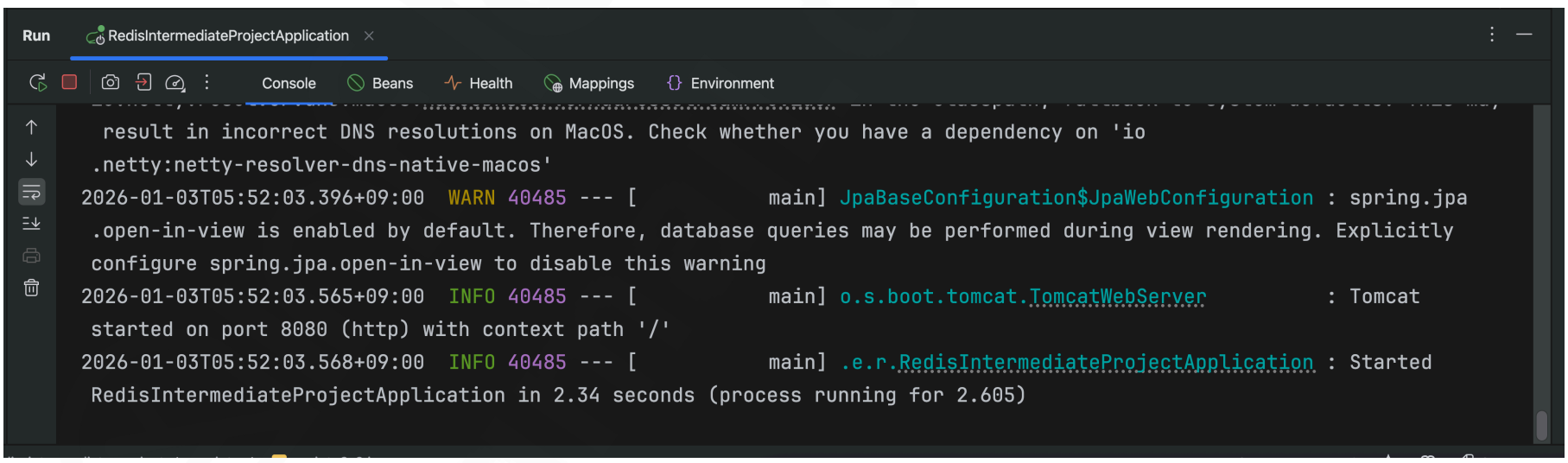
   StockLockService

```
@Component
@RequiredArgsConstructor
public class StockLockService {

  private final RedisTemplate<String, String> redisTemplate;
  private final StockService stockService;

  public void decrease(Long id) throws InterruptedException {
    // Redis 명령어에서 'SET [key] [value] NX EX [seconds]'와 동일
    // key : stock_lock:{id}
    // value : lock
    // timeout : 3초 (락을 획득하고 3초 뒤에는 자동으로 소멸 -> 데드락 방지)

    String key = "stock_lock:" + id;

    // 락 획득 시도 (락 획득에 실패하면 100ms 대기 후 재시도)
    while (!tryLock(key)) {
      Thread.sleep(100);
    }

    try {
      stockService.decrease(id);
    } finally {
      // 락 해제
      redisTemplate.delete(key);
    }
  }

  // 락 획득 시도 (성공시 true, 실패시 false를 return)
  // (Redis 명령어에서 'SET [key] [value] NX EX [seconds]'와 동일)
  // (key : stock_lock:{id})
  // (value : lock)
  // (timeout : 3초 => 락을 획득하고 3초 뒤에는 자동으로 소멸함으로써 데드락 방지)
  private boolean tryLock(String key) {
    return redisTemplate
        .opsForValue()
        .setIfAbsent(key, "lock", Duration.ofMillis(3_000));
  }
}
```

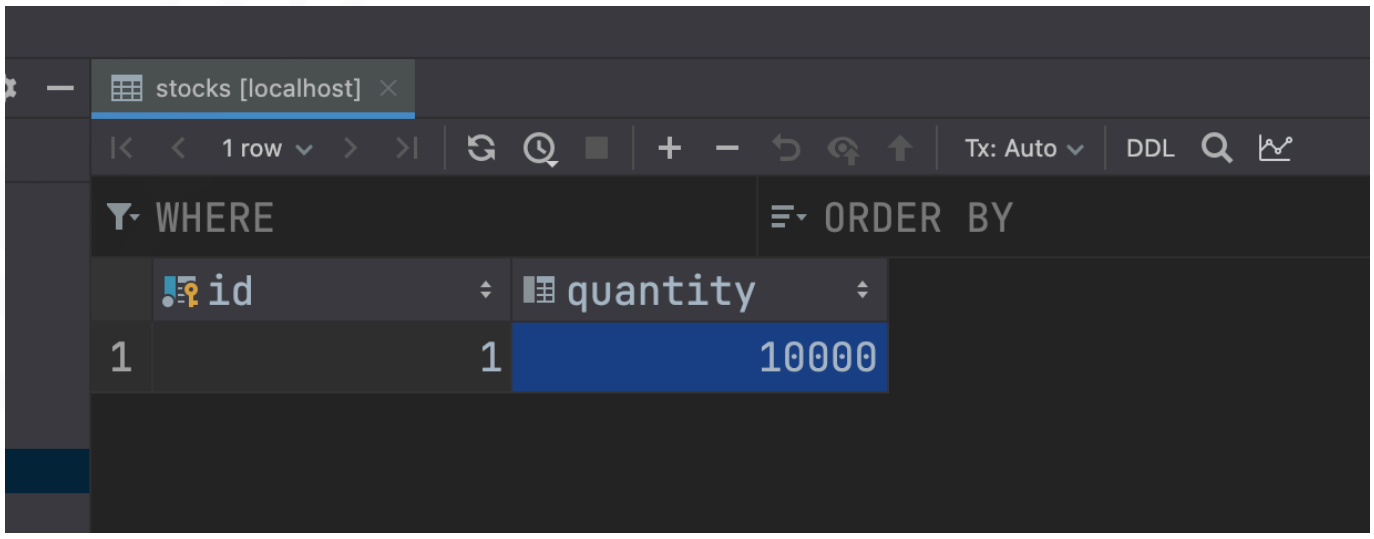
StockController

```
@RestController
@RequestMapping("/stocks")
@RequiredArgsConstructor
public class StockController {

  private final StockService stockService;
  private final StockLockService stockLockService;

  ...

  @PostMapping("/{id}/decrease/redis")
  public void decreaseWithRedis(
      @PathVariable Long id
  ) throws InterruptedException {
    stockLockService.decrease(id);
  }
}
```

2. **서버 실행시키기**



4. **테스트를 위해 DB에 데이터 넣어주기**



2. **부하 테스트 스크립트 살펴보기**

scripts/script_2-2.js

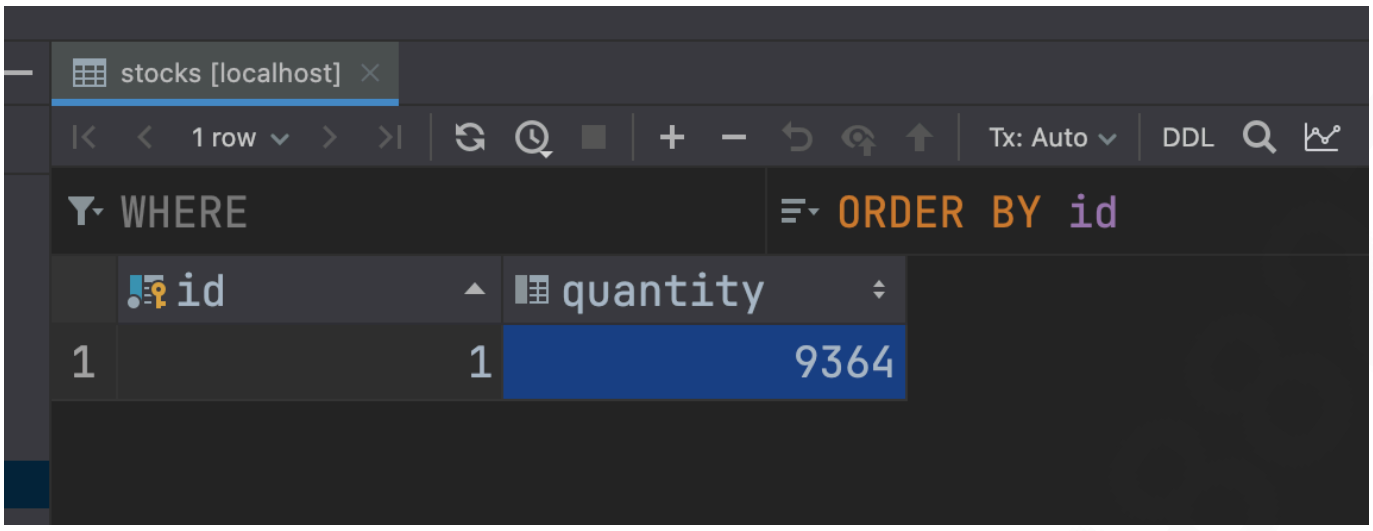
```
import http from 'k6/http';
import { check, sleep } from 'k6';
```

```
export const options = {
  // 가상 유저(VUs) 100명으로 설정
  vus: 100,

  // 테스트를 10초 동안 진행
  duration: '10s',
};

export default function () {
  // 재고 차감 API
  const url = 'http://localhost:8080/stocks/1/decrease/redis';

  const params = {
    headers: {
      'Content-Type': 'application/json',
    },
  };

  // POST 요청 전송
  const res = http.post(url, null, params);

  // 1초 간격으로 요청 발송
  sleep(1);

  // 응답 상태 코드 확인 (200 OK가 왔는 지 체크)
  check(res, {
    'status is 200': (r) => r.status === 200,
  });
}
```

3. **부하테스트 진행하기**

```
$ k6 run scripts/script_2-2.js
```

```
✓ status is 200

checks.........................: 100.00% ✓ 636       ✗ 0
data_received..................: 46 kB   4.1 kB/s
data_sent......................: 99 kB   8.7 kB/s
http_req_blocked...............: avg=356.56µs min=1µs    med=8µs     max=4.08ms p(90)=2.01ms  p(95)=2.53ms
http_req_connecting............: avg=275.81µs min=0s     med=0s      max=2.85ms p(90)=1.35ms  p(95)=2.36ms
http_req_duration..............: avg=663.31ms min=3.38ms med=12.06ms max=6.14s  p(90)=3.26s   p(95)=4.39s
  { expected_response:true }...: avg=663.31ms min=3.38ms med=12.06ms max=6.14s  p(90)=3.26s   p(95)=4.39s
http_req_failed................: 0.00%   ✓ 0         ✗ 636
http_req_receiving.............: avg=75.64µs  min=14µs   med=62µs    max=3.84ms p(90)=110.5µs p(95)=135µs
http_req_sending...............: avg=33.25µs  min=3µs    med=30µs    max=319µs  p(90)=55µs    p(95)=64.24µs
http_req_tls_handshaking.......: avg=0s       min=0s     med=0s      max=0s     p(90)=0s      p(95)=0s
http_req_waiting...............: avg=663.2ms  min=3.3ms  med=11.91ms max=6.14s  p(90)=3.26s   p(95)=4.39s
http_reqs......................: 636     55.948889/s
iteration_duration.............: avg=1.66s    min=1s     med=1.01s   max=7.14s  p(90)=4.26s   p(95)=5.4s
iterations.....................: 636     55.948889/s
vus............................: 14      min=14      max=100
vus_max........................: 100     min=100     max=100
```



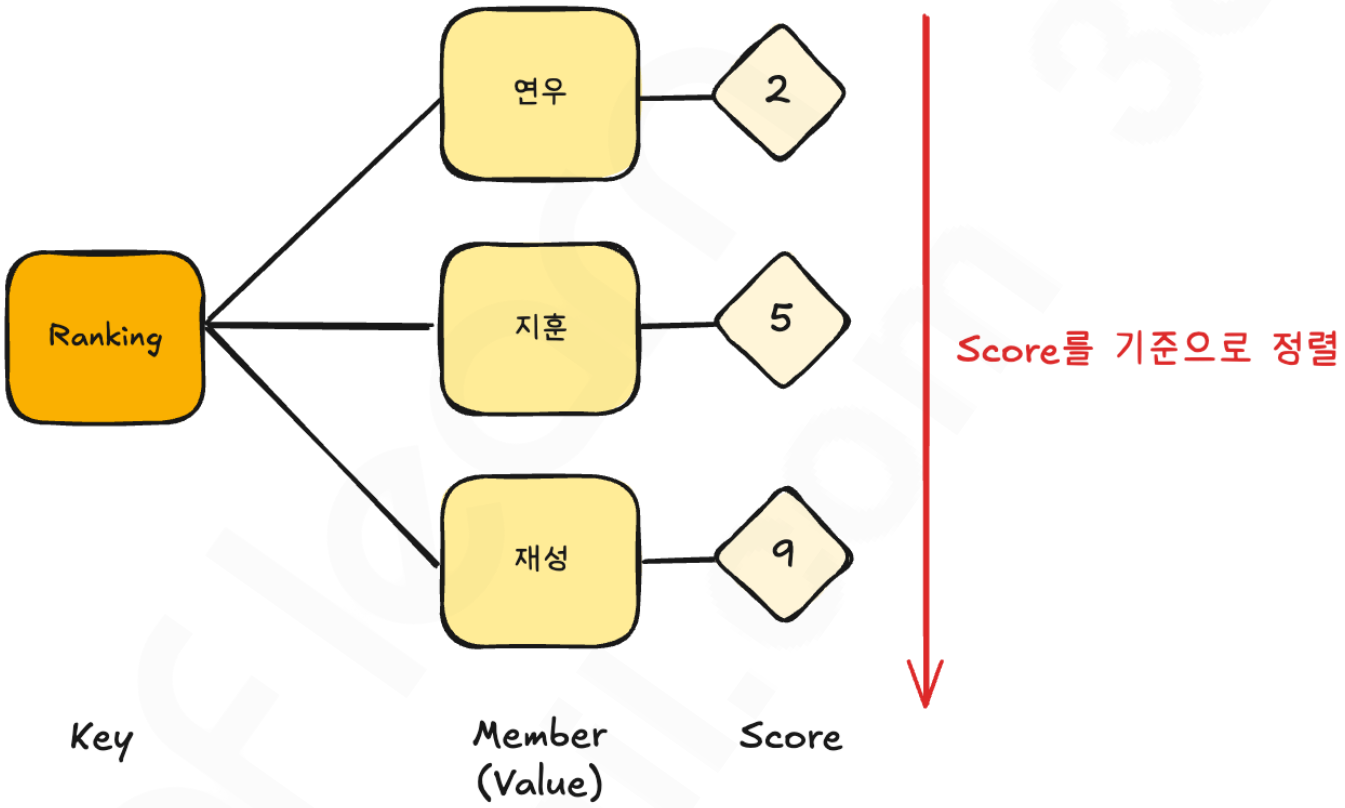
들어오는 요청을 병렬적으로 처리하던 걸 Redis의 Lock을 활용해 순서대로 차례차례 처리하게 만들어서 기존에 성능보다 TPS가 떨어지긴 했다. 하지만 재고 차감 요청의 횟수(636회)에 맞게 오차 없이 정확하게 재고가 차감됐다. 즉, Redis로 Lock을 구현해 동시성 이슈를 해결했다.

@naver.com
khb3025

Sorted Set 자료 구조 (ZADD, ZRANGE, ZINCRBY)

✅ Sorted Set (ZSet)

Redis에서 **Sorted Set(ZSet)**은 **score(점수)를 기준으로 자동 정렬되는 중복 없는 자료구조**이다. 이런 특성 때문에 실제 서비스에서 랭킹 시스템, 실시간 인기 데이터 관리처럼 순서와 가중치가 중요한 경우에 주로 활용된다.



참고) Redis의 Sorted Set에서는 key-value의 구조에서 value를 member라 부른다.

✅ Sorted Set 생성 및 데이터 저장

```
# Sorted Set에 데이터 저장
# ZADD [key] [score] [member]
$ ZADD ranking:2025 5 jihoon # ranking:2025에서 jihoon의 score는 5
$ ZADD ranking:2025 9 jaeseong # ranking:2025에서 jaeseong의 score는 9
$ ZADD ranking:2025 2 yeonwoo # ranking:2025에서 yeonwoo의 score는 2
```

✅ Sorted Set으로부터 데이터 조회

```
# member + score 조회
# ZRANGE [key] [start index] [end index] WITHSCORES
$ ZRANGE ranking:2025 0 1 WITHSCORES # 0번 인덱스부터 1번 인덱스까지의 데이터를 오름차순으로 조회
$ ZRANGE ranking:2025 0 -1 WITHSCORES # 0번 인덱스(처음)부터 -1번 인덱스(끝)까지의 데이터를 조회
$ ZRANGE ranking:2025 0 -1 WITHSCORES REV # REV : 역순(내림차순)으로 조회

# member 조회
# ZRANGE [key] [start index] [end index]
$ ZRANGE ranking:2025 0 1 # 0번 인덱스부터 1번 인덱스까지의 데이터를 오름차순으로 조회
```

```
$ ZRANGE ranking:2025 0 -1 # 0번 인덱스(처음)부터 -1번 인덱스(끝)까지의 데이터를 조회
$ ZRANGE ranking:2025 0 -1 REV # REV : 역순(내림차순)으로 조회
```

✅ Sorted Set 점수 증가

```
# Sorted Set의 Score 증가 (+ 만약 key가 없을 경우 Sorted Set 새로 생성)
# ZINCRBY [key] [increment] [member]
$ ZINCRBY ranking:2026 3 jaeseong

# Sorted Set이 새로 생성됐는 지 확인하기
$ ZRANGE ranking:2026 0 -1 WITHSCORES

# score 2 추가
$ ZINCRBY ranking:2026 2 jaeseong

# score 2가 정상적으로 추가됐는 지 확인하기
$ ZRANGE ranking:2026 0 -1 WITHSCORES
```

khb3025@gmail.com

🎯

[실습] 많이 검색한 키워드 Top 10을 보여주는 '인기 검색어 기능' 만들기 - 1

✅ 인기 검색어 기능 구현

실제 서비스를 구현하다보면 특정 기준으로 **순위별 데이터를 보여줘야 하는 경우**가 많다. 예를 들어, 인기 게시글이라던지, 인기 검색어라던지, 인기순으로 영화 데이터를 보여줘야 된다던지, 상품을 판매순으로 보여줘야 되는 것들이 전부 순위별 데이터를 보여줘야 하는 경우이다.

다양한 예시 중에서 '인기 검색어 기능'의 예를 가지고 RDB 기반으로 구현된 코드를 먼저 살펴보자. 그런 다음 Redis로 직접 구현하는 방식을 직접 경험해보자.

✅ 실습 진행하기

> 효율적인 실습을 위해 검색 API, 인기 검색어 Top 10 조회 API를 미리 만들어뒀다.

1. **검색 API, 인기 검색어 조회 API 살펴보기**

SearchKeyword

```
@Entity(name = "search_keywords")
@Getter
@NoArgsConstructor
public class SearchKeyword {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String keyword;

    private Long count;

    public SearchKeyword(String keyword) {
        this.keyword = keyword;
        this.count = 1L;
    }

    public void increaseCount() {
        this.count++;
    }
}
```

SearchController

```
@RestController
@RequiredArgsConstructor
@RequestMapping("/search")
public class SearchController {

    private final SearchService searchService;
```

```
    @GetMapping()
    public void search(@RequestParam String keyword) {
        searchService.search(keyword);
    }

    @GetMapping("/top10")
    public List<String> getTop10Keywords() {
        return searchService.getTop10Keywords();
    }
}
```

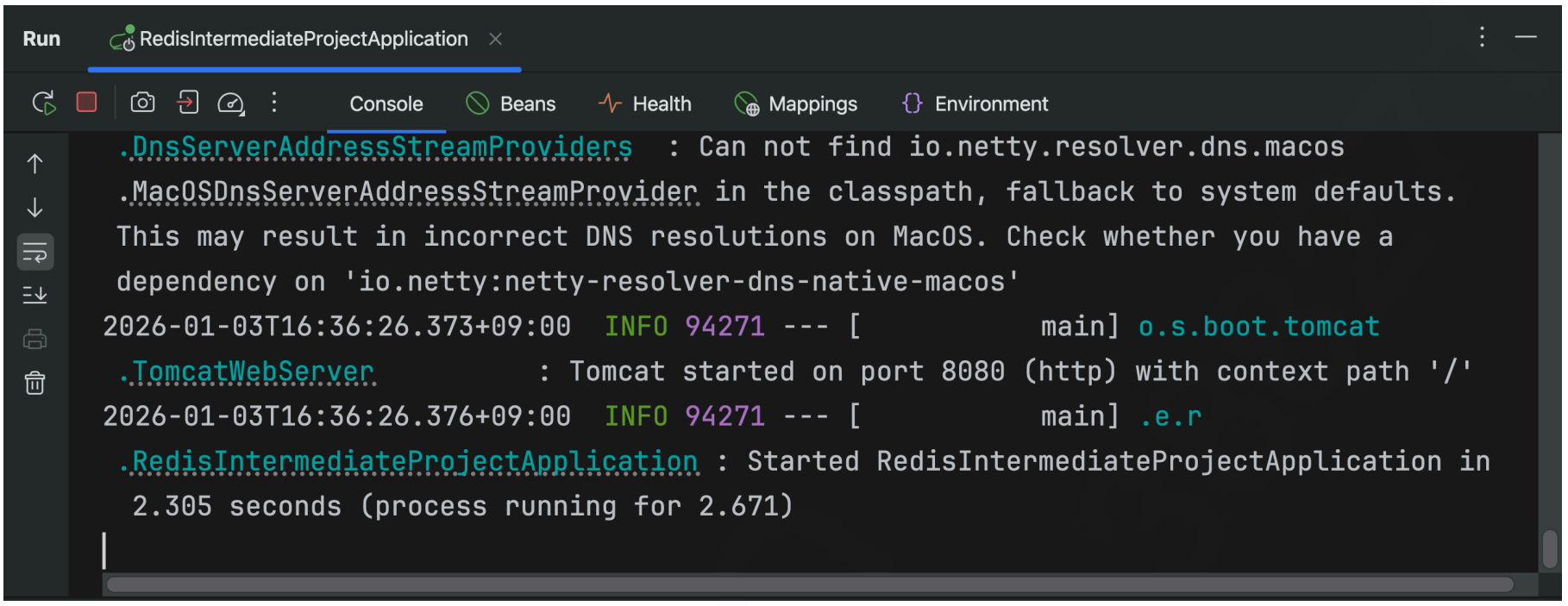
StockService

```
@Service
@RequiredArgsConstructor
public class SearchService {

    private final SearchRepository searchRepository;

    @Transactional
    public void search(String keyword) {
        SearchKeyword searchKeyword = searchRepository.findByKeyword(keyword)
                .orElse(new SearchKeyword(keyword));
        searchKeyword.increaseCount();
        searchRepository.save(searchKeyword);
    }

    @Transactional(readOnly = true)
    public List<String> getTop10Keywords() {
        return searchRepository.findTop10ByOrderByCountDesc().stream()
                .map(SearchKeyword::getKeyword)
                .toList();
    }
}
```

StockRepository

```
public interface SearchRepository extends JpaRepository<SearchKeyword, Long> {
    Optional<SearchKeyword> findByKeyword(String word);
    List<SearchKeyword> findTop10ByOrderByCountDesc();
}
```

3. **Spring Boot 실행시키기**



2. **부하 테스트 스크립트 살펴보기**

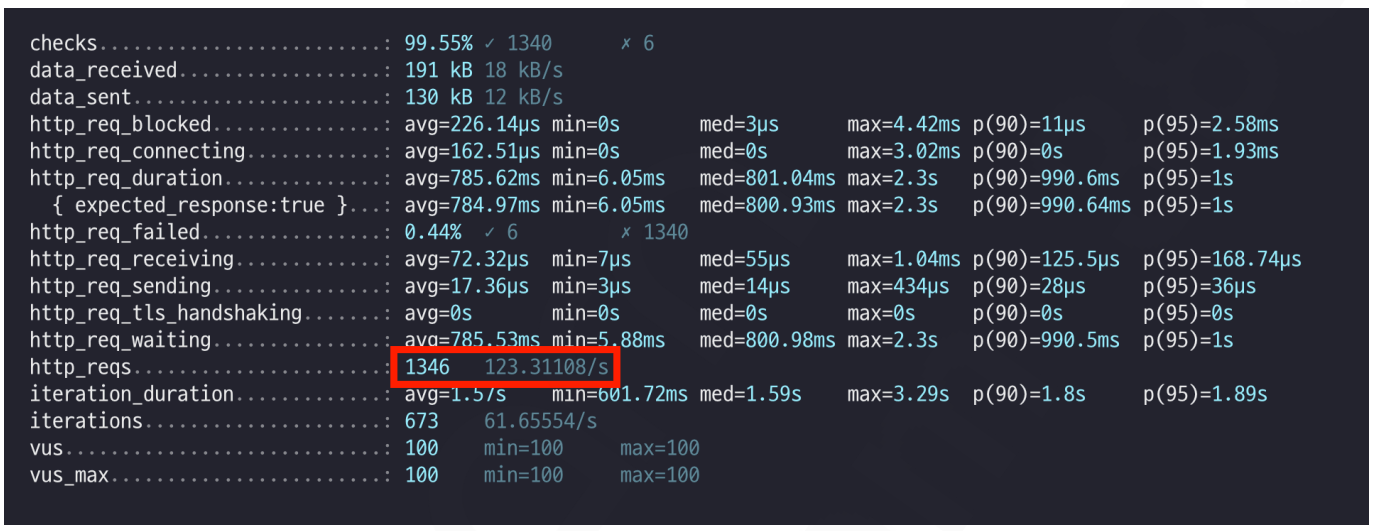
scripts/script_3-1.js

```
import http from 'k6/http';
import {check} from 'k6';
import {randomItem} from 'https://jslib.k6.io/k6-utils/1.2.0/index.js';

export const options = {
  // 가상 유저(VUs) 100명으로 설정
  vus: 100,

  // 테스트를 10초 동안 진행
  duration: '10s',
};

// 검색어 리스트 정의
const keywords = [
  'spring', 'java', 'redis', 'mysql', 'jpa',
  'k6', 'performance', 'test', 'load', 'stress',
  'docker', 'kubernetes', 'aws', 'cloud', 'microservices',
  'python', 'javascript', 'react', 'vue', 'angular'
];

export default function () {
  // 1. 검색 API 호출
  // 랜덤한 키워드 선택
  const keyword = randomItem(keywords);

  // 검색 요청
  const searchRes = http.get(`http://localhost:8080/search?keyword=${keyword}`);

  check(searchRes, {
    'search status is 200': (r) => r.status === 200,
  });

  // 2. 인기 검색어 조회 API 호출
  const top10Res = http.get('http://localhost:8080/search/top10');

  check(top10Res, {
    'top10 status is 200': (r) => r.status === 200,
```

```
  });
}
```

3. **부하 테스트 진행하기**

```
$ k6 run scripts/script_3-1.js
```



검색 API와 인기 검색어 조회 API 둘 다 총 합해서 1346번 요청을 보냈음을 알 수 있고, 두 API 평균 TPS는 123인 것을 알 수 있다.

RDB를 기반으로 인기 검색어 Top 10을 조회하려면 count(검색 횟수)를 기준으로 매번 정렬을 해서 조회해야 하기 때문에 성능 측면에서 비효율이 발생한다. 따라서 성능을 개선하기 위해 RDB 대신에 Redis를 활용해 구현해보자.

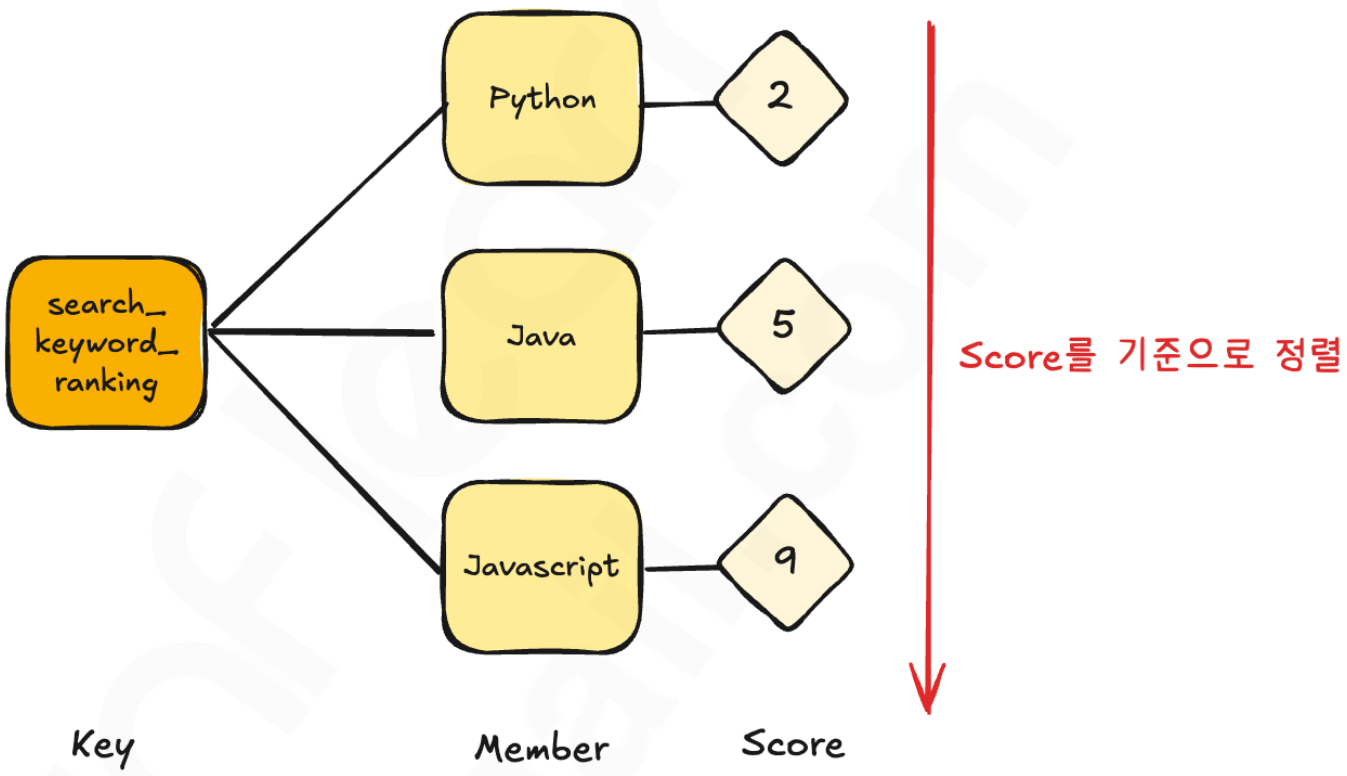
참고) RDB의 인덱스를 활용해서 성능을 개선하는 것도 하나의 방법이지만, 여기서는 극도로 성능을 끌어올려야 한다는 가정하에 Redis를 도입하는 상황이라고 가정하자.

@gmail.com
khb3025

🎯 [실습] 많이 검색한 키워드 Top 10을 보여주는 '인기 검색어 기능' 만들기 - 2

👨🏻‍🏫 이번에는 검색 API와 인기 검색어 조회 API를 Redis를 활용해 구현해보자.

✅ 구현 방법



- 인기 검색어 조회 API에서는 인기순으로 검색어를 정렬해야 하기 때문에, **자동 정렬을 해주는 자료구조인 Sorted Set**을 활용한다.
- **검색을 할 때마다 score를 1 증가**시킴으로써 인기 검색어를 구분한다.

✅ 실습

1. **코드 작성하기**

SearchService

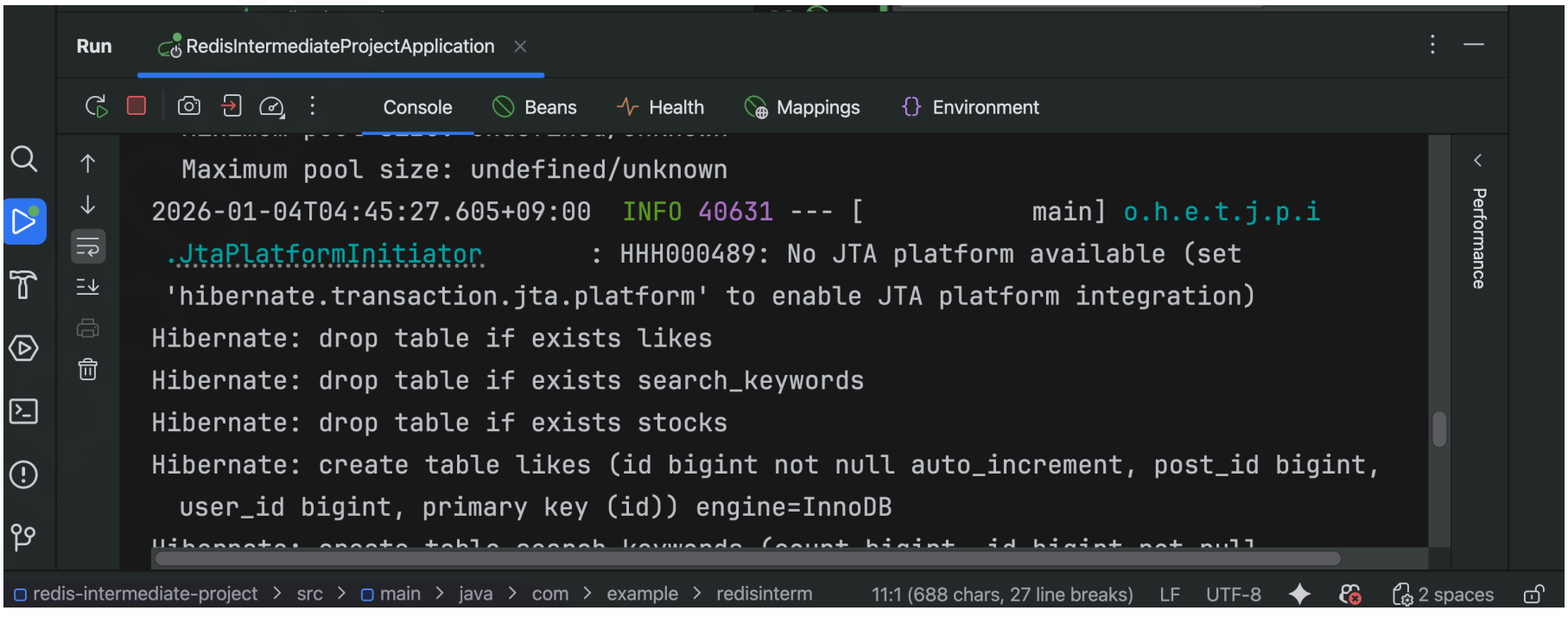
```
@Service
@RequiredArgsConstructor
public class SearchService {

    private final SearchRepository searchRepository;
    private final RedisTemplate<String, String> redisTemplate;

    ...


    public void searchWithRedis(String keyword) {
      // Redis 명령어에서 'ZINCRBY search_keyword_ranking 1 [keyword]'와 동일
      // ZINCRBY 명령어를 사용하여 해당 keyword의 score를 1 증가시킴
      // (만약 해당 키워드가 존재하지 않으면 새로 추가하고 score를 1로 설정함)
      redisTemplate.opsForZSet()
```

```
            .incrementScore("search_keyword_ranking", keyword, 1.0);
    }

    public List<String> getTop10KeywordsWithRedis() {
      // Redis 명령어에서 'ZRANGE [key] [start index] [end index] REV'와 동일
      // REV 옵션을 사용하여 Score가 높은 순(내림차순)으로 0~9번 인덱스 데이터를 조회
      Set<String> topKeywords = redisTemplate.opsForZSet()
          .reverseRange("search_keyword_ranking", 0, 9);

      // Set을 List로 변환하여 반환 (null일 경우 빈 리스트 반환)
      return new ArrayList<>(topKeywords);
    }
}
```

SearchController

```
@RestController
@RequiredArgsConstructor
@RequestMapping("/search")
public class SearchController {

    private final SearchService searchService;

    ...

    @GetMapping("/redis")
    public void searchWithRedis(@RequestParam String keyword) {
      searchService.searchWithRedis(keyword);
    }

    @GetMapping("/top10/redis")
    public List<String> getTop10KeywordsWithRedis() {
      return searchService.getTop10KeywordsWithRedis();
    }
}
```

2. **서버 실행시키기**



3. **부하테스트 스크립트 살펴보기**

   이전에 실행시켰던 스크립트(`script_3-1.js`)와 코드가 대부분 똑같고, 새로 만든 API에 맞게 주소만 변경해주었다.

   scripts/script_3-2.js

```
import http from 'k6/http';
import {check} from 'k6';
import {randomItem} from 'https://jslib.k6.io/k6-utils/1.2.0/index.js';

export const options = {
  // 가상 유저(VUs) 100명으로 설정
  vus: 100,

  // 테스트를 10초 동안 진행
  duration: '10s',
};

// 검색어 리스트 정의
const keywords = [
  'spring', 'java', 'redis', 'mysql', 'jpa',
  'k6', 'performance', 'test', 'load', 'stress',
  'docker', 'kubernetes', 'aws', 'cloud', 'microservices',
  'python', 'javascript', 'react', 'vue', 'angular'
];

export default function () {
  // 1. 검색 API 호출
  // 랜덤한 키워드 선택
  const keyword = randomItem(keywords);

  // 검색 요청
  const searchRes = http.get(`http://localhost:8080/search/redis?keyword=${keyword}`);

  check(searchRes, {
    'search status is 200': (r) => r.status === 200,
  });

  // 2. 인기 검색어 조회 API 호출
  const top10Res = http.get('http://localhost:8080/search/top10/redis');

  check(top10Res, {
    'top10 status is 200': (r) => r.status === 200,
  });
}
```

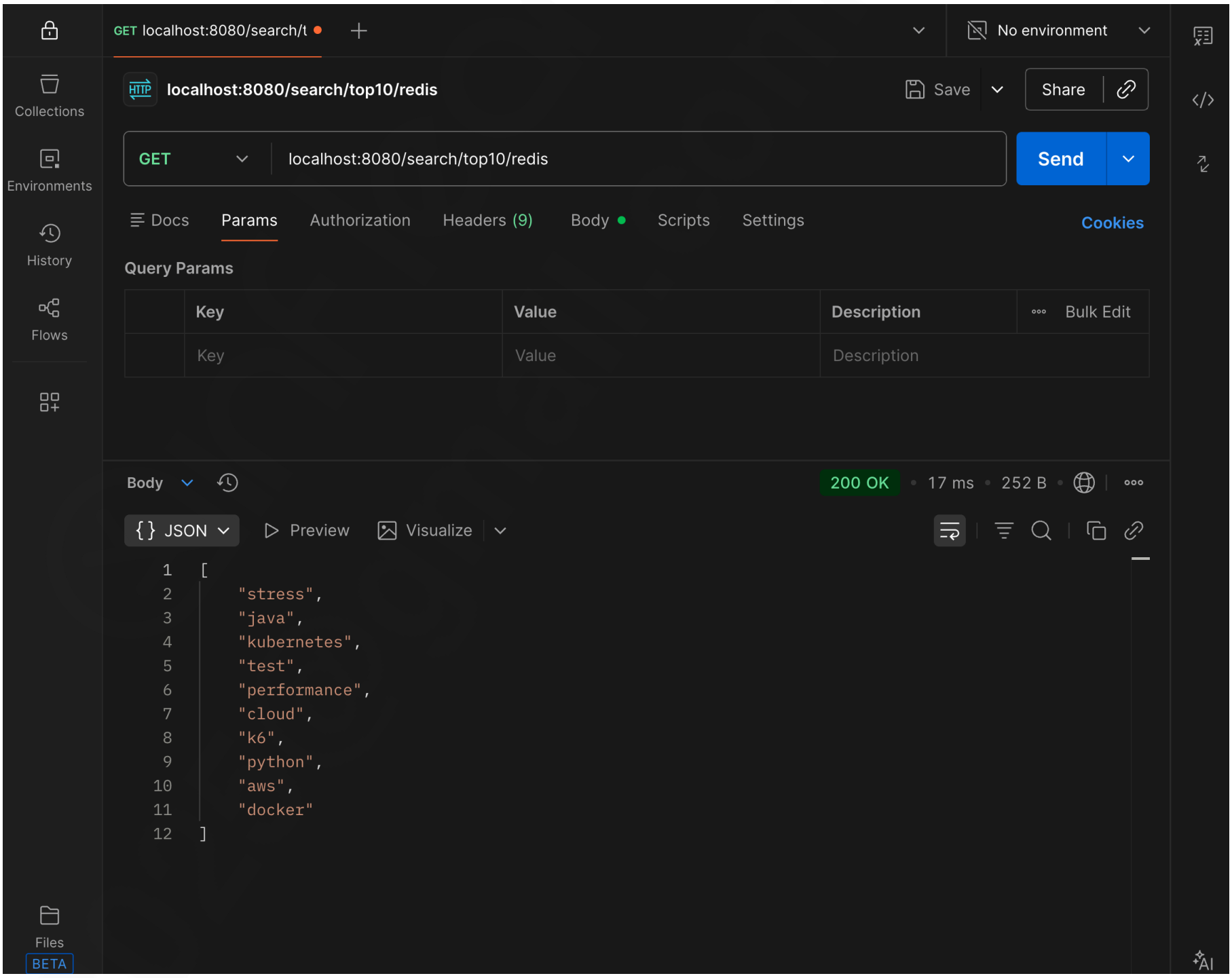
3. **부하테스트 진행하기**

```
$ k6 run scripts/script_3-2.js
```

```
checks.........................: 100.00% ✓ 404324       ✗ 0
data_received..................: 58 MB   5.8 MB/s
data_sent......................: 41 MB   4.1 MB/s
http_req_blocked...............: avg=2.49µs  min=0s      med=1µs    max=3.87ms  p(90)=1µs    p(95)=2µs
http_req_connecting............: avg=1.46µs  min=0s      med=0s     max=2.93ms  p(90)=0s     p(95)=0s
http_req_duration..............: avg=2.44ms  min=170µs   med=2.31ms max=53.51ms p(90)=3.35ms p(95)=3.82ms
  { expected_response:true }...: avg=2.44ms  min=170µs   med=2.31ms max=53.51ms p(90)=3.35ms p(95)=3.82ms
http_req_failed................: 0.00%   ✓ 0           ✗ 404324
http_req_receiving.............: avg=14.37µs min=2µs     med=7µs    max=6.11ms  p(90)=31µs   p(95)=49µs
http_req_sending...............: avg=3.55µs  min=1µs     med=2µs    max=5.41ms  p(90)=5µs    p(95)=6µs
http_req_tls_handshaking.......: avg=0s      min=0s      med=0s     max=0s      p(90)=0s     p(95)=0s
http_req_waiting...............: avg=2.42ms  min=164µs   med=2.29ms max=53.47ms p(90)=3.33ms p(95)=3.79ms
http_reqs......................: 404324  40422.456076/s
iteration_duration.............: avg=4.94ms  min=677.5µs med=4.76ms max=85.39ms p(90)=6.56ms p(95)=7.29ms
iterations.....................: 202162  20211.228038/s
vus............................: 100     min=100       max=100
vus_max........................: 100     min=100       max=100
```

부하테스트를 해보니 검색 API와 인기 검색어 조회 API의 평균 TPS가 40422로 성능이 눈에 띄게 향상된 걸 확인할 수 있다.

5. 인기 검색어 조회 API가 정말 잘 작동하는 지 테스트해보기



✅ 정리

순위별 데이터를 조회해야 하는데 대용량 트래픽을 감당해야 하는 상황이라면, Redis의 Sorted Set을 활용하는 걸 고려해보자.

Sorted Set 자료 구조 (ZCARD, ZREMRANGEBYSCORE, EXPIRE)

API 요청 횟수 제한 기능을 구현하기 위해 필요한, Sorted Set 자료 구조의 명령어를 추가로 더 배워보자.

✅ Sorted Set의 데이터 개수 조회

```
# Sorted Set 생성 및 데이터 삽입
$ ZADD ranking 1 apple
$ ZADD ranking 5 banana
$ ZADD ranking 3 melon

# Sorted Set에 들어있는 데이터 개수 조회
# ZCARD [key]
$ ZCARD ranking
```

✅ Sorted Set의 score 기준으로 데이터 삭제

```
# score 범위에 해당하는 데이터 삭제
# ZREMRANGEBYSCORE [key] [min score] [max score]
$ ZREMRANGEBYSCORE ranking 0 3 # score가 0 이상 3이하인 데이터 삭제

# 삭제 됐는 지 확인
$ ZRANGE ranking 0 -1 WITHSCORES # 0번 인덱스(처음)부터 -1번 인덱스(끝)까지 데이터 조회
```

✅ Sorted Set에 만료기간(TTL) 설정

```
# Redis에 저장되어 있는 전체 key 확인
$ keys *

# 만료 시간 설정 (단위 : 초)
# EXPIRE [key] [seconds]
$ EXPIRE ranking 10 # 10초 후에 자동 삭제

# 남은 만료 시간 확인
# TTL [key]
$ TTL ranking

# 만료시간이 지난 뒤 삭제 됐는 지 확인해보기
$ keys *
```

참고) `EXPIRE`, `TTL` 명령어는 Sorted Set에만 쓰이는 명령어는 아니고, Redis의 모든 자료 구조에서 통용적으로 쓰이는 명령어이다.

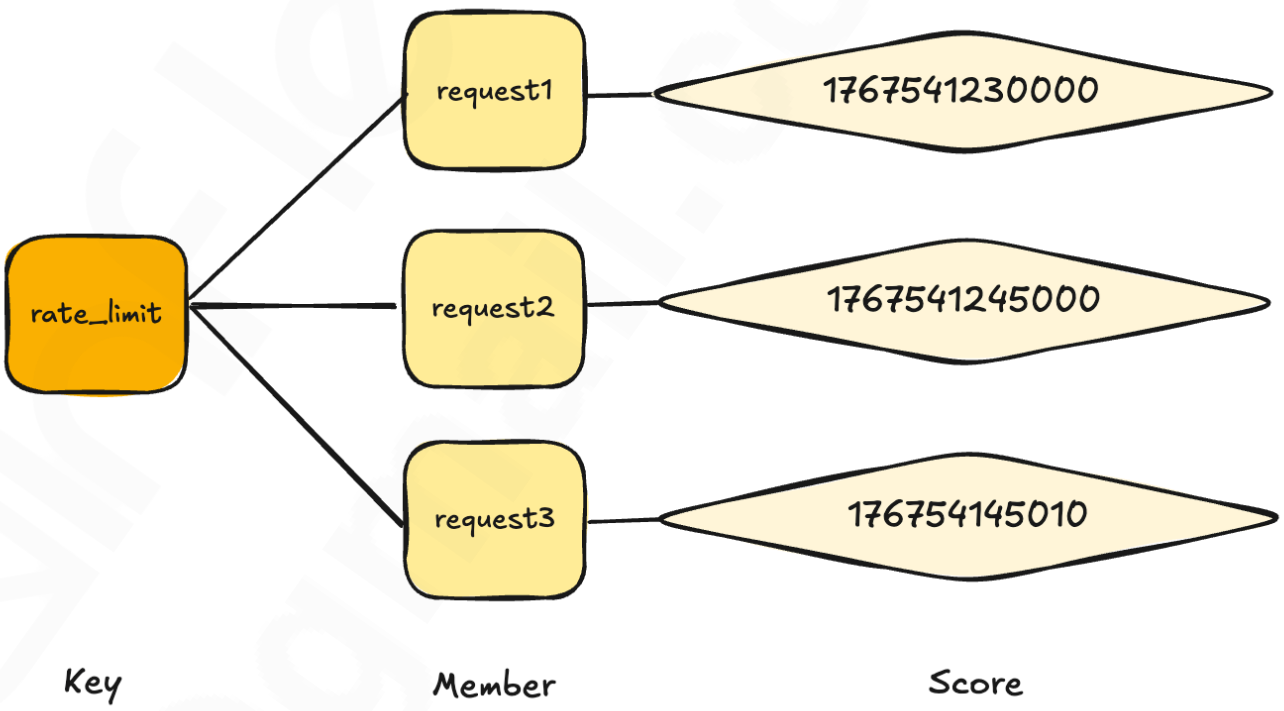
[실습] 악의적인 요청을 방지하기 위해, 10초당 API 요청 횟수를 5회로 제한하기 - 1

✅ 문제 상황

클론 받은 프로젝트의 API에는 요청 횟수에 대한 제한(Rate Limit)이 전혀 걸려있지 않다. 이 상태에서는 특정 사용자가 악의적으로 아주 짧은 시간 안에 수십, 수백 번의 요청을 보내면 서버 부하가 급격히 증가할 수 있다. 따라서 실무에서는 악의적으로 사용자가 서버에 과부하를 걸 수 없도록 API 요청 횟수를 제한한다. 따라서 이번 실습에서는 10초당 API 요청 횟수를 5회로 제한하는 기능을 Redis를 활용해 구현해볼 것이다.

✅ 구현 방법

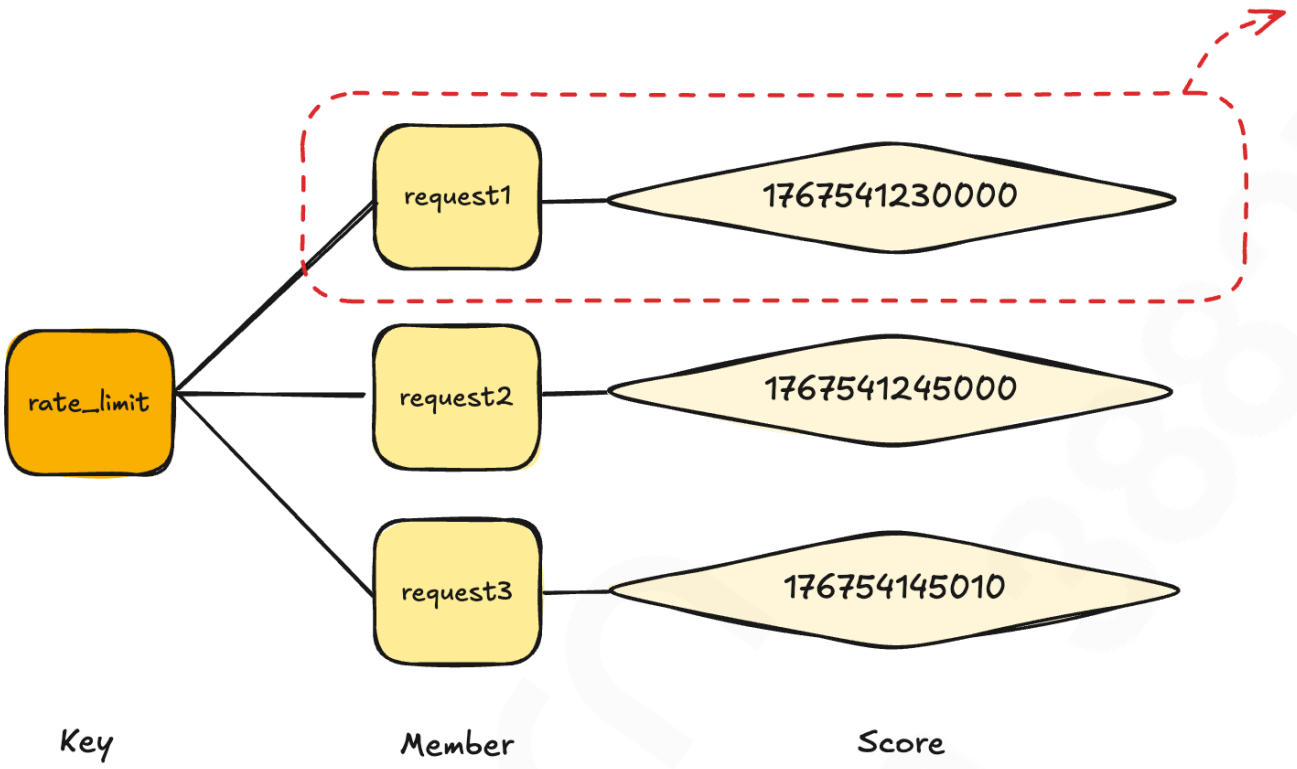
1. **요청이 들어올 때마다 Sorted Set에서 Member에는 각 요청을 구분할 수 있는 고유의 값을 넣고, Score에는 요청을 보낸 시간을 unixtime으로 넣는다.**



2. **'현재 시간(unixtime)'에서 'Score에 저장되어 있는 요청을 보낸 시간(unixtime)'을 빼서 10,000(10초)을 넘는 값은 전부 삭제한다.**

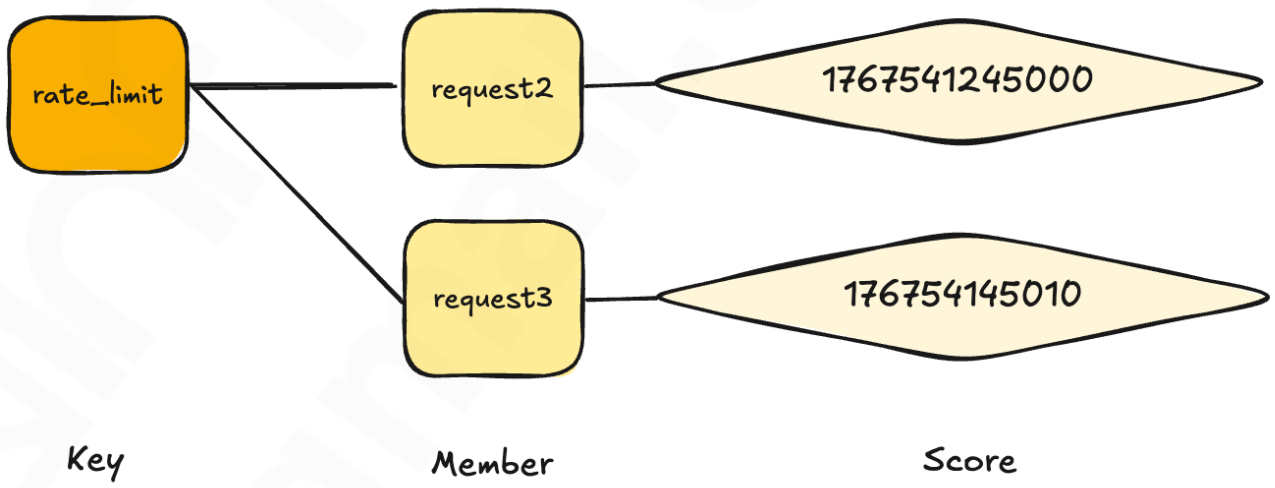
최근 10초에서 발생한 API 요청 횟수를 카운팅해서 제한하는 방식이기 때문에, 현재 시간을 기준으로 10초가 넘어간 요청은 삭제해준다.

khb3023@gmail.com



3. **Sorted Set에 남아있는 Member의 총 개수를 세알린다.**

   현재 시간을 기준으로 10초가 넘어간 요청은 삭제했기 때문에, 남아있는 요청은 10초 이내의 요청들이다. 따라서 남아있는 요청의 개수가 곧 최근 10초 동안 사용자가 보낸 요청의 개수이다.



4. **사용자가 보낸 요청의 개수로 API 요청을 허용하거나 제한한다.**

khb3025

🎯 [실습] 악의적인 요청을 방지하기 위해, 10초당 API 요청 횟수를 5회로 제한하기 - 2

✅ 실습하기

1. **코드 작성하기**

   RateLimiter

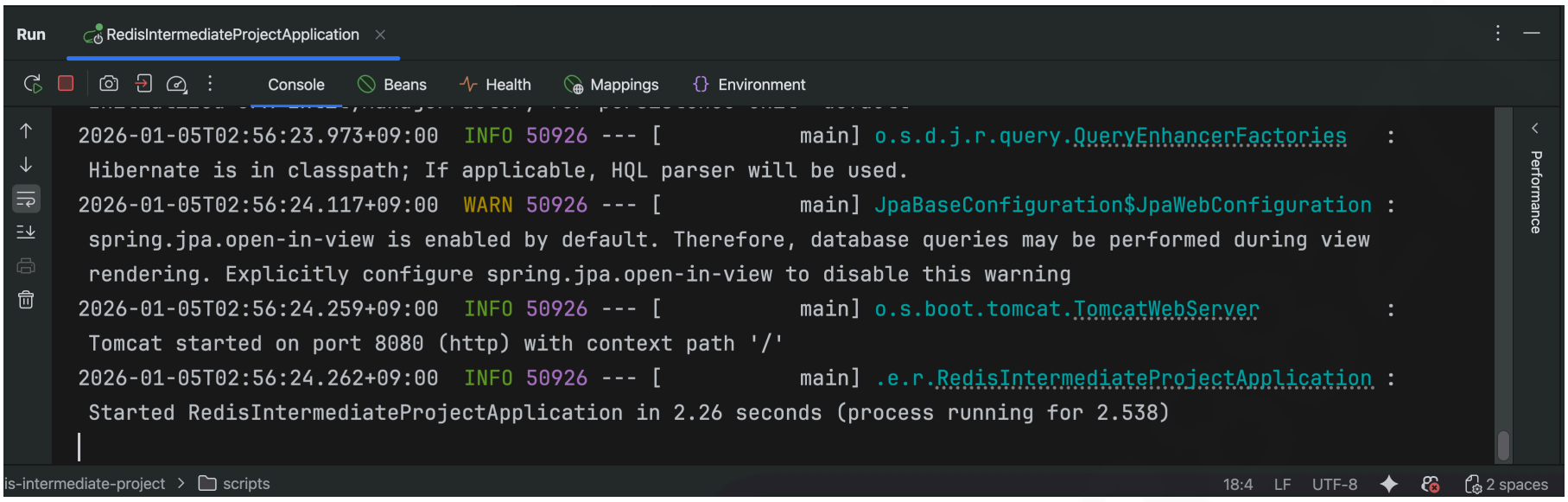
```
@Component
public class RateLimiter {
  private static final long RATE_LIMIT_TIME_MS = 10_000; // 요청을 카운팅하는 시간 단위
  private static final long MAX_REQUESTS = 5; // 10초 동안 최대 5개의 요청만 허용

  private final RedisTemplate<String, String> redisTemplate;

  public RateLimiter(RedisTemplate<String, String> redisTemplate) {
    this.redisTemplate = redisTemplate;
  }

  public boolean allow(String userId) {
    long now = System.currentTimeMillis();

    // user 단위로 API 요청 횟수를 제한하기 위해, userId를 기반으로 Key를 생성
    // (IP 단위로 API 요청 횟수를 제한하고 싶다면, userId 대신 IP를 파라미터로 받으면 된다.)
    String redisKey = "rate_limit:" + userId;

    // 서로 다른 요청을 구분하기 위해 UUID로 requestId 값을 생성
    String requestId = UUID.randomUUID().toString();

    // 1. Soted Set에서(redisKey)에서 score가 0 이상이고 (now - 10_000) 이하인 모든 member를 삭
제
    // = 최근 10초 내에 저장한 member만 남겨두고, 나머지 member는 전부 삭제
    redisTemplate
        .opsForZSet()
        .removeRangeByScore(redisKey, 0, now - RATE_LIMIT_TIME_MS);

    // 2. 현재 들어온 API 요청을 Sorted Set에 저장.
    // member를 UUID 기반으로 생성된 고유의 requestId로 저정하고,
    // score를 현재 시간값(ex. 1735972805123)으로 저장한다.
    // SortedSet에 저장하는 것이기 때문에 score를 기준으로 자동 정렬돼서 저장된다.
    redisTemplate
        .opsForZSet()
        .add(redisKey, requestId, now);

    // 3. Sorted Set에 저장되어 있는 데이터 개수 조회
    // = 최근 10초 내에 요청을 보낸 횟수
    Long count = redisTemplate
        .opsForZSet()
        .size(redisKey);

    // 4. 만료 시간(TTL)을 10초로 설정함으로써
    // Redis 공간을 불필요하게 많이 점유하는 것을 방지
```

```
    redisTemplate.expire(
        redisKey,
        10_000,
        TimeUnit.MILLISECONDS
    );

    // 최대 요청 수보다 작거나 같으면 true를 리턴
    // 최대 요청 수를 초과하면 false를 리턴
    return count <= MAX_REQUESTS;
  }
}
```

RateLimitFilter

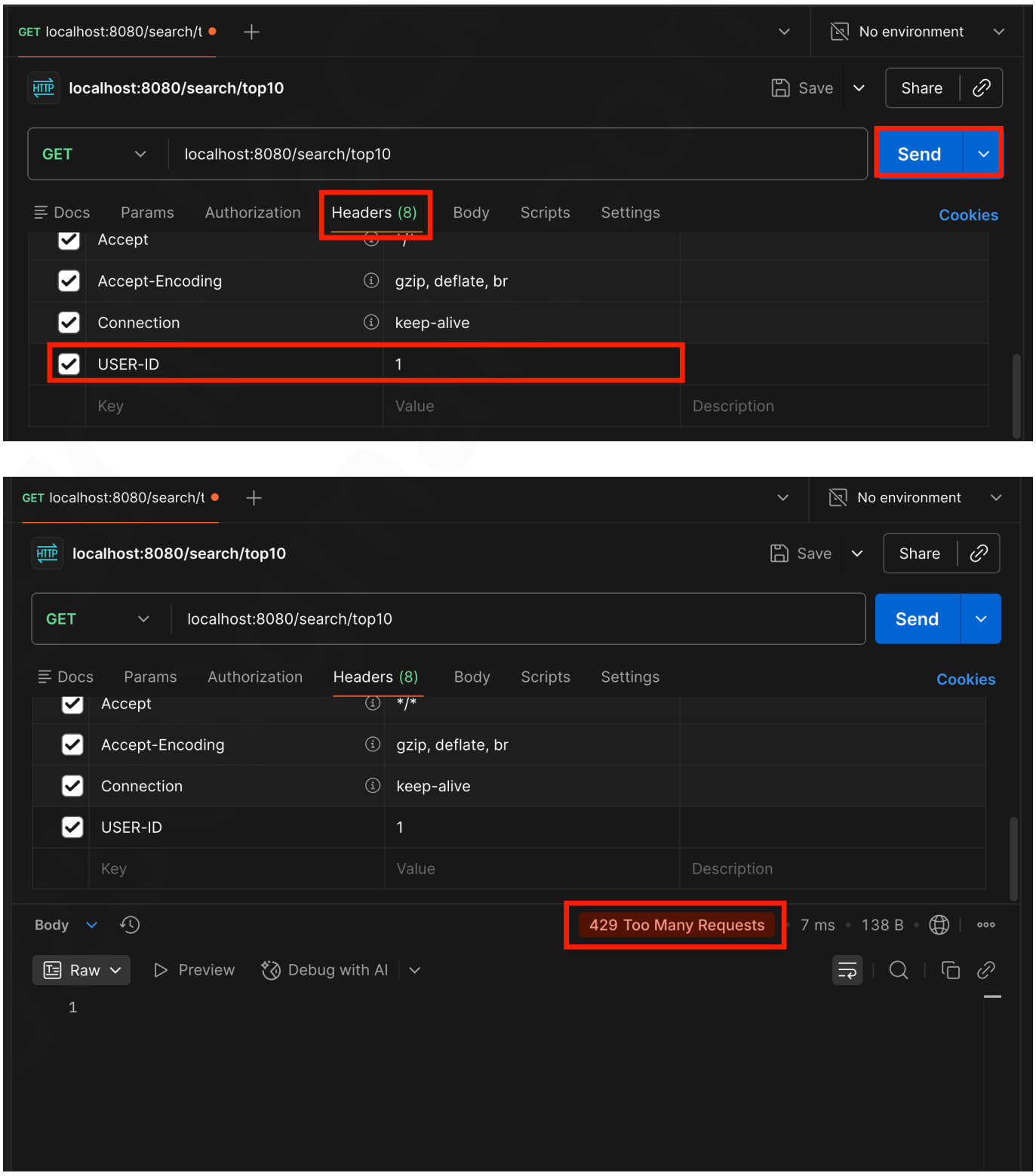
```
// OncePerRequestFilter : HTTP 요청이 들어올 때마다 단 한 번 실행되는 필터 로직
@Component
public class RateLimitFilter extends OncePerRequestFilter {
  private final RateLimiter rateLimiter;

  public RateLimitFilter(RateLimiter rateLimiter) {
    this.rateLimiter = rateLimiter;
  }

  @Override
  protected void doFilterInternal(
      HttpServletRequest request,
      HttpServletResponse response,
      FilterChain filterChain
  ) throws ServletException, IOException {

    // 어떤 사용자인지 식별하기 위한 키
    // (실무에서는 IP 값을 활용해 식별하는 편)
    String userId = request.getHeader("USER-ID");

    // userId가 null일 땐 RateLimiter가 작동하지 않게 바로 return하기
    if (userId == null) {
      return;
    }

    // rateLimiter.allow()가 false라는 건 API 요청 횟수를 초과했다는 뜻이다.
    // API 요청 횟수가 초과할 경우 429(TOO MANY REQUESTS)로 응답하게 된다.
    if (!rateLimiter.allow(userId)) {
      response.setStatus(429);
      return;
    }

    filterChain.doFilter(request, response);
  }
}
```

2. **서버 실행시키기**



3. **Postman으로 요청 보내보기**

   API 요청 횟수를 잘 제한시켜주는 지, Postman을 활용해 10초 안에 5회가 넘는 요청을 보내보자.



정상적으로 API 요청 횟수 제한 기능이 잘 작동하는 걸 확인할 수 있다.

📖

Geospatial 자료 구조 (GEOADD, GEOPOS, GEODIST, GEOSEARCH)

✅ Geospatial 자료 구조

Redis에서 **Geospatial**은 **위치 데이터(경도, 위도)를 저장하고 검색하는 데 최적화된 자료 구조**이다. 이 자료 구조는 **특정 위치 기준으로 주어진 거리 내의 데이터를 찾아야 할 때 자주 사용하는 자료 구조**이다.

예를 들어, 배달의 민족과 같은 서비스에서는 사용자의 현재 위치(경도, 위도)를 기준으로 반경 3km 이내에 있는 음식점을 빠르게 조회해야 한다. 이때 Redis의 Geospatial을 사용하면 각 음식점의 위치를 미리 저장해두고, 사용자 위치를 중심으로 가까운 음식점을 거리순으로 즉시 검색할 수 있다.

또 다른 예로는 카카오 택시 서비스가 있다. 승객이 호출을 하면, 시스템은 승객의 위치를 기준으로 가장 가까운 택시를 실시간으로 찾아서 알림을 보낼 수 있어야 한다. 이 경우에도 Redis Geospatial을 활용하면 택시들의 위치 정보를 저장해 두었다가 특정 반경 내의 기사들을 빠르게 조회할 수 있다.

✅ Geospatial 데이터 저장

```
# 경도, 위도 데이터를 저장
# GEOADD [key] [위도] [경도] [member]
$ GEOADD cafe 127.0276 37.4979 a_cafe
$ GEOADD cafe 127.0294 37.4981 b_cafe
$ GEOADD cafe 127.0330 37.4950 c_cafe
```

✅ Geospatial 데이터 조회

```
# 데이터가 잘 저장됐는 지 key로 확인하기
$ keys *

# 모든 member 조회하기
$ ZRANGE cafe 0 -1 # 0번 인덱스(처음)부터 -1번 인덱스(끝)까지의 데이터를 조회

# 저장된 특정 member의 경도, 위도 좌표를 조회하는 명령어
# GEOPOS [key] [member...]
$ GEOPOS cafe a_cafe b_cafe c_cafe
```

** 특징 : Geospatial 자료 구조에 저장된 데이터는, 내부적으로 Sorted Set 방식으로 저장된다. 그래서 Sorted Set 명령어를 입력해도 작동한다.

✅ Geospatial 거리 조회

```
# 두 데이터 사이의 직선 거리를 조회
# GEOPOS [key] [member1] [member2] [단위]
$ GEODIST cafe a_cafe b_cafe m

# 특정 위치를 기준으로 원하는 반경 내에 있는 데이터 조회
# (+ 거리 가까운 순으로 조회 = ASC)
```

```
# (+ 거리 포함해서 출력 = WITHDIST)
# GEOSEARCH [key] FROMLONLAT [위도] [경도] BYRADIUS [반경] [단위] [ASC/DESC] WITHDIST
$ GEOSEARCH cafe FROMLONLAT 127.0280 37.4975 BYRADIUS 1000 m ASC WITHDIST
```

** 참고) GEOSEARCH 명령어는 위의 기능 이외에도 다양한 검색 옵션을 제공하니 필요 시 찾아보도록 하자.

@naver.com
khb302

🎯

[실습] 내 위치를 기준으로 반경 1km에 위치한 카페 찾기 - 1

✅ 상황

실무에서 '**내 위치를 기준으로 반경 1km에 위치한 카페 찾는 기능**'을 구현해야 한다고 가정하자. 만약 이 기능을 RDB인 MySQL로 구현해야 한다면 SQL문이 굉장히 복잡해지고 거리 계산을 일일이 다 해야되기 때문에 성능도 좋지 않다.

그래서 특정 위치 기준으로 주어진 거리 내의 데이터를 찾아야 할 때 자주 사용하는 Redis의 Geospatial 자료 구조를 활용해 구현해볼 것이다. Geospatial은 거리 계산에 최적화된 자료 구조이면서 거리 계산에 관련된 기능을 기본적으로 제공해주기 때문에 성능이 보장되면서도 쉽게 구현할 수 있다.

✅ 실습

1. **코드 살펴보기**

> 실습의 편의를 위해 일부 코드를 작성해두었다.

CafeController

```
@RestController
@RequestMapping("/cafe")
@RequiredArgsConstructor
public class CafeController {

    private final CafeService cafeService;

    // Redis에 카페 데이터 추가하는 API
    @PostMapping
    public void addCafe(@RequestBody AddCafeRequestDto addCafeRequestDto) {
        cafeService.addCafe(addCafeRequestDto);
    }

    // 특정 위치를 기준으로 지정된 반경 내에 있는 카페를 조회해주는 API
    @GetMapping("/nearby")
    public List<Object> findCafesNearby(@RequestParam double longitude,
                                        @RequestParam double latitude,
                                        @RequestParam double distance) {
        return cafeService.findCafesNearby(longitude, latitude, distance);
    }
}
```

AddCafeRequestDto

```
@AllArgsConstructor
@Getter
public class AddCafeRequestDto {
    private String name;
    private double longitude;
```

```
    private double latitude;
}
```

CafeService

```
@Service
@RequiredArgsConstructor
public class CafeService {

    public void addCafe(AddCafeRequestDto addCafeRequestDto) {
        return;
    }

    public List<Object> findCafesNearby(double longitude, double latitude, double distance
Km) {
        return null;
    }
}
```

2. **코드 작성하기**

   CafeService

```
@Service
@RequiredArgsConstructor
public class CafeService {

    private final RedisTemplate<String, String> redisTemplate;
    private static final String GEOSPATIAL_KEY = "cafe";

    public void addCafe(AddCafeRequestDto addCafeRequestDto) {
        // Redis GeoSpatial 자료구조에 카페 데이터 저장
        // Redis 명령어에서 'GEOADD [key] [위도] [경도] [member]'와 동일
        redisTemplate.opsForGeo()
            .add(GEOSPATIAL_KEY, new Point(addCafeRequestDto.getLongitude(), addCafeReques
tDto.getLatitude()), addCafeRequestDto.getName());
    }

    public List<Object> findCafesNearby(double longitude, double latitude, double distance
Km) {
        // Redis GeoSpatial 자료구조에서 특정 위치를 기준으로 지정된 반경 내에 있는 카페 조회 (오름
차순)
        // Redis 명령어에서 'GEOSEARCH [key] FROMLONLAT [경도] [위도] BYRADIUS [거리] [단위] A
SC WITHDIST'와 동일
        GeoSearchCommandArgs args = GeoSearchCommandArgs
            .newGeoSearchArgs()
            .includeDistance()
            .sortAscending(); // ASC WITHDIST

        return redisTemplate.opsForGeo().search( // GEOSEARCH
                GEOSPATIAL_KEY, // [key]
                fromCoordinate(new Point(longitude, latitude)), // FROMLONLAT [경도] 위도]
                new Distance(distanceKm, Metrics.KILOMETERS), // BYRADIUS [거리] [단위]
                args // ASC WITHDIST
        ).getContent().stream()
                .map(geoResult -> new CafeDistance(
```

```
                        geoResult.getContent().getName(), geoResult.getDistance().getValue()
// '카페 이름'과 '거리'를 반환
                    ))

                .collect(Collectors.toList());
    }
}
```


다음 강의에서 작성한 코드가 잘 작동하는 지 테스트해보자.

@naver.com
khb3025

🎯

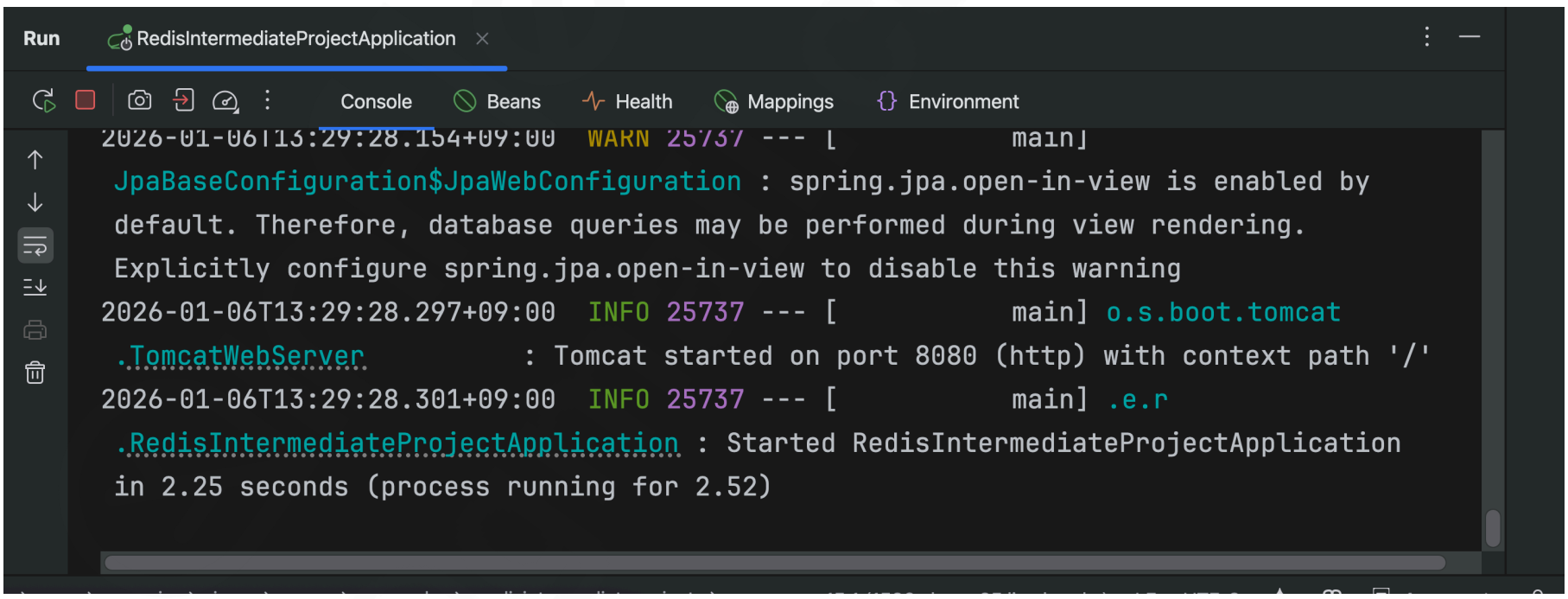
[실습] 내 위치를 기준으로 반경 1km에 위치한 카페 찾기 - 2

✅ 실습

1. **테스트를 위해 Redis에 생성되어 있는 Key 삭제하기**

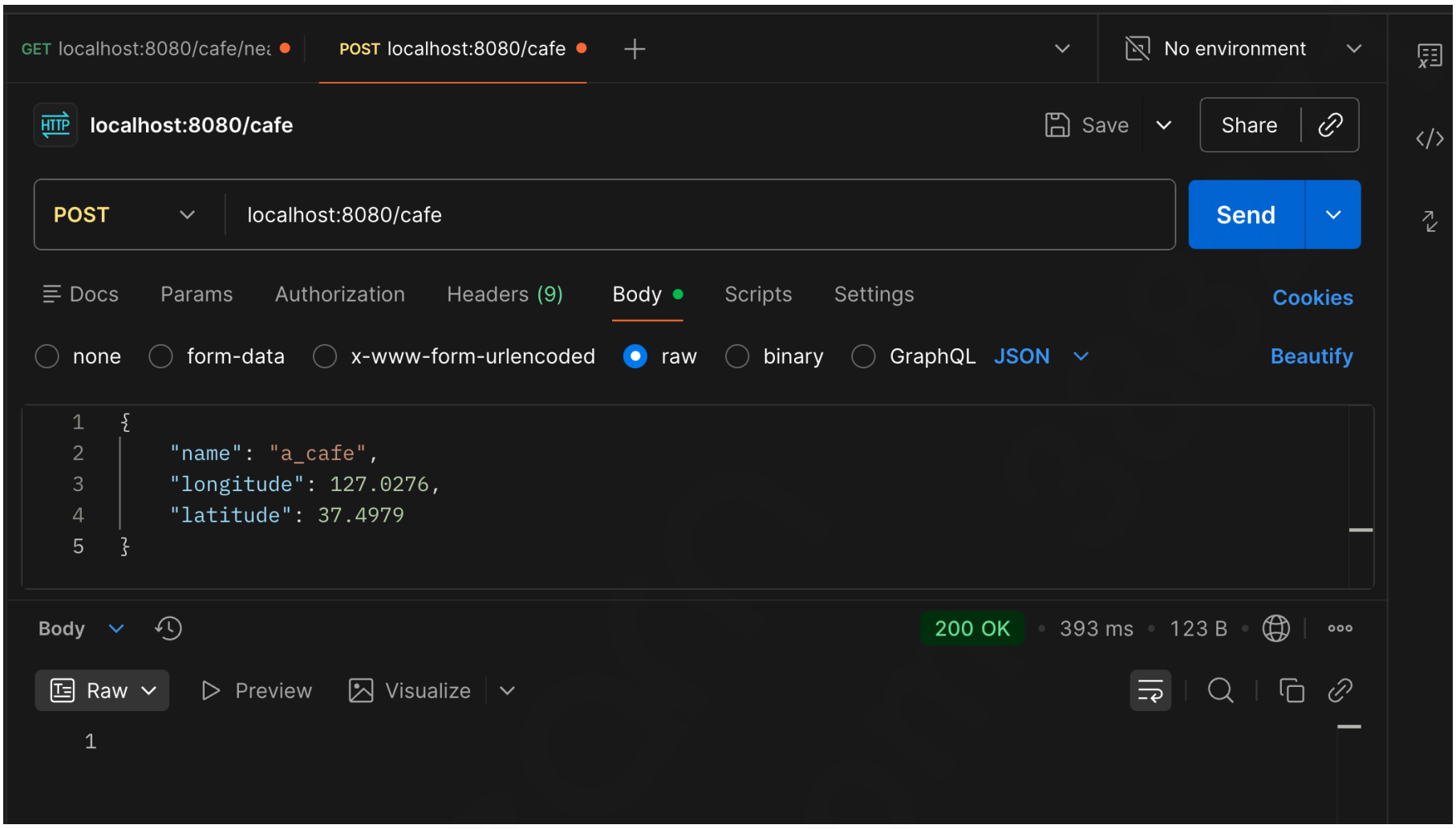
```
# 모든 key 조회
$ keys *

# 특정 key 삭제
# del [key]
$ del cafe
```

2. **서버 실행시키기**



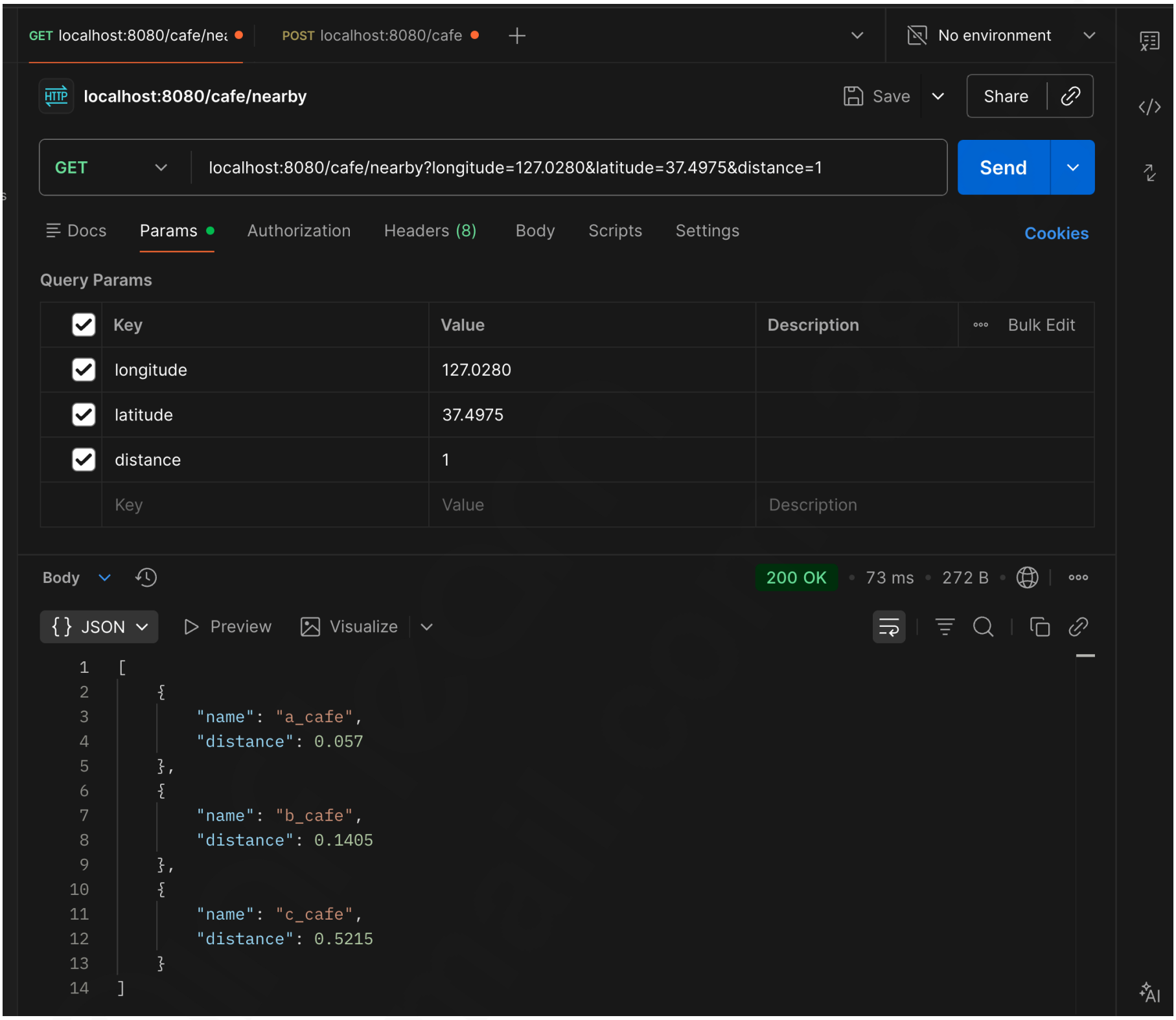
3. **Postman으로 카페 추가하기**

khb302500@gmail.com




```
// 1
{
    "name": "a_cafe",
    "longitude": 127.0276,
    "latitude": 37.4979
}

// 2
{
    "name": "b_cafe",
    "longitude": 127.0294,
    "latitude": 37.4981
}

// 3
{
    "name": "c_cafe",
    "longitude": 127.0330,
    "latitude": 37.4950
}
```

4. **Postman으로 특정 위치를 기준으로 1km 반경에 있는 카페 조회하기**



특정 위치를 기준으로 1km 반경에 있는 카페가 가까운 순으로 잘 조회되는 걸 확인할 수 있다.

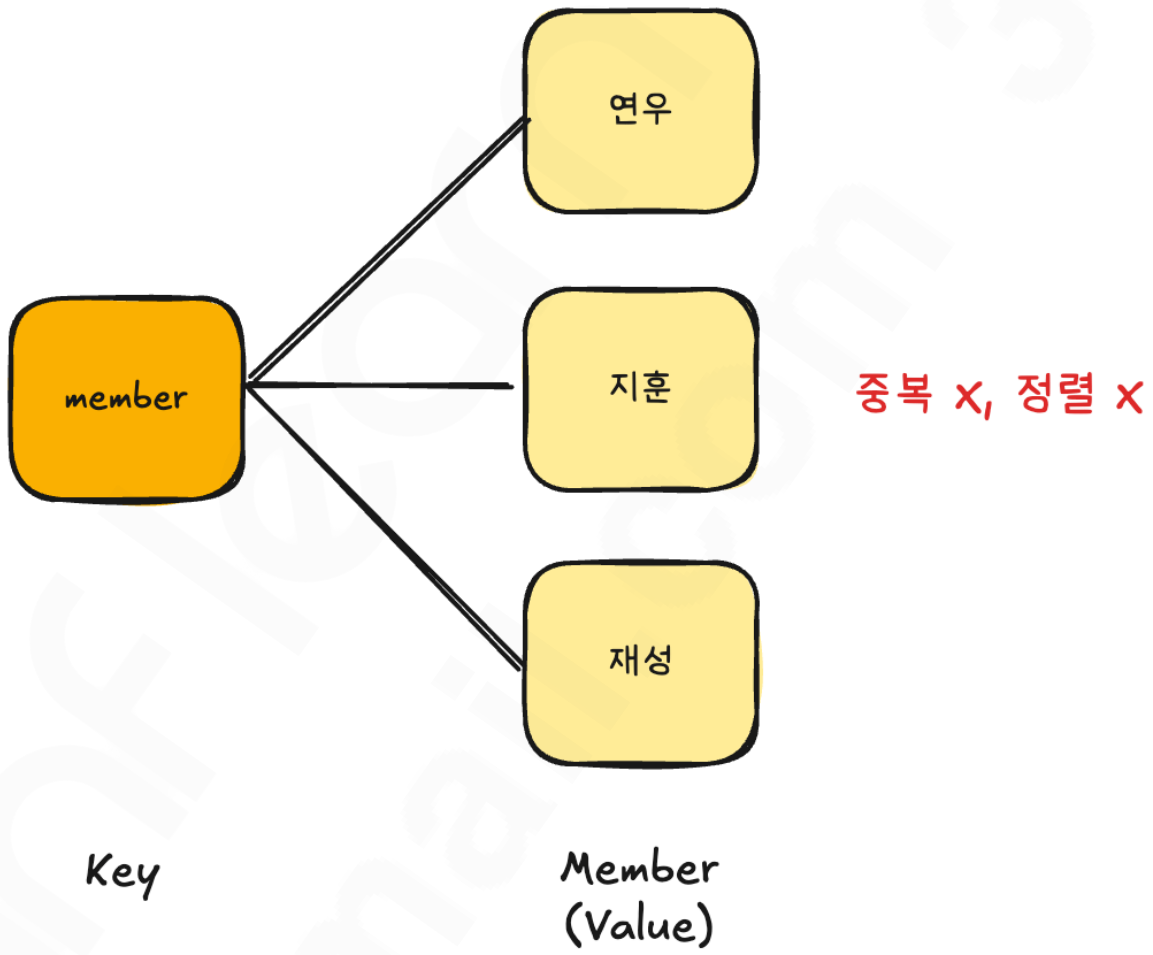
khb3025@gmail.com

📖

Set 자료 구조 (SADD, SMEMBERS, SCARD)

✅ Set

이전 강의에서 Redis의 Sorted Set 자료 구조를 먼저 배웠었다. Sorted Set은 score(점수)를 기준으로 자동 정렬되는 중복없는 자료 구조라고 설명했다. **Set**은 **정렬되지 않는 중복 없는 자료구조**를 뜻한다. 이런 특성 때문에 Set은 **'순서는 중요하지 않지만, 중복되면 안 되는 데이터'를 다룰 때 적합**하다.



✅ Set에 데이터 저장

```
# Set에 데이터 저장
# SADD [key] [member...]
$ SADD member yeonwoo
$ SADD member jihoon jaeseong # 한 번에 여러 개의 값을 넣을 수도 있음
$ SADD member jaeseong yeonwoo # 테스트를 위해 중복되는 값 넣어보기
```

✅ Set에 데이터 조회

```
$ keys * # Set이 잘 생성됐는 지 확인하기

# SET에 들어있는 모든 데이터 조회
# SMEMBERS [key]
$ SMEMBERS member # 중복이 제거된 채로 데이터가 조회되는 걸 확인하기 (정렬도 되어 있지 않음)

# SET에 들어있는 데이터 개수 조회
```

```
# SCARD [key]
$ SCARD member
```

@naver.com
khb3025

Bitmap 자료 구조 (SETBIT, GETBIT, BITCOUNT)

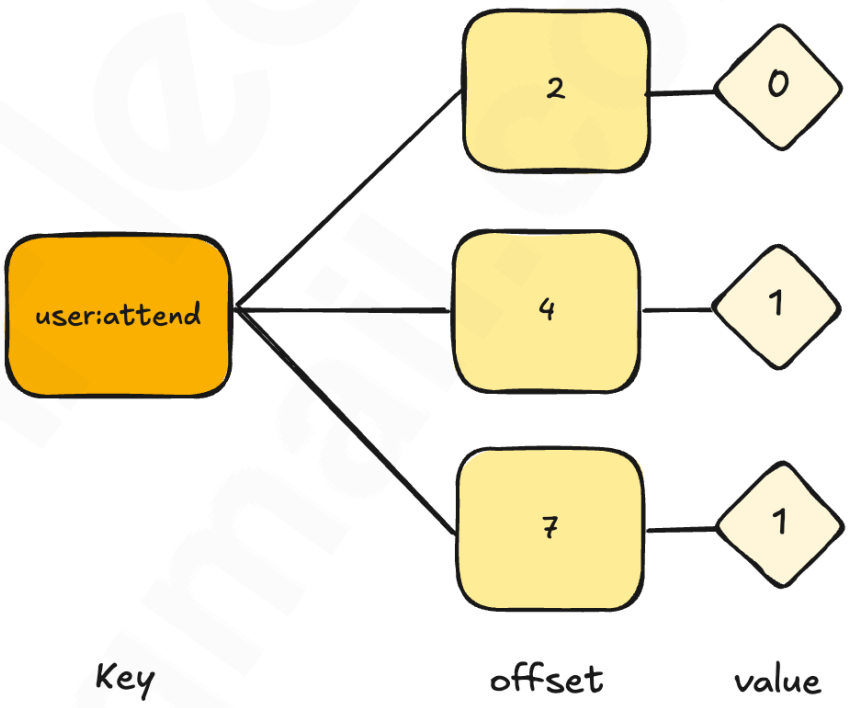
✅ Bitmap 자료 구조

Redis에서 **Bitmap**은 **아주 적은 메모리로 대량의 boolean 상태를 저장할 때 사용하는 자료구조**이다. Bitmap의 가장 큰 장점은 공간 효율성이다. 일반적으로 boolean 값을 하나 저장하려면 8 bit 이상이 필요하지만, Bitmap은 1 bit만 사용하기 때문에 수백만~수억 개의 상태값도 매우 적은 메모리로 관리할 수 있다.

이러한 특징 때문에 Bitmap은 아래와 같은 예시에서 많이 활용된다.

- 사용자별 일일 출석 여부 확인 (출석 : true / 결석 : false)
- 대규모 이벤트에서 사용자별 참여 여부 (참석 : true / 불참 : false)
- 하루 동안 서비스에 방문한 사용자 수인 DAU 구하기 (방문 : true / 방문 X : false)

또한 Bitmap 자료 구조의 특징은 다음과 같다.



- key, offset, value의 구성 요소를 가지고 있다.
- offset에는 0 이상의 정수만 넣을 수 있다. 인덱스(index)라고 생각하면 편하다.
- value에는 0 또는 1의 숫자만 들어갈 수 있다. 이 값을 활용해서 true(1), false(0)를 표현하면 된다.

✅ Bitmap 데이터 저장

```
# Bitmap 데이터 저장
# SETBIT [key] [offset] [value]
$ SETBIT user:attend 4 1
$ SETBIT user:attend 7 1
```

✅ Bitmap 데이터 조회

```
# Bitmap이 잘 저장됐는 지 key로 확인하기
$ keys *
```

```
# Bitmap에서 특정 offset의 value 확인하기
# GETBIT [key] [offset]
$ GETBIT user:attend 2 # offset에 아무값도 저장되어 있지 않을 때는 0으로 출력
$ GETBIT user:attend 4
$ GETBIT user:attend 7

# 특정 Bitmap에 value가 1로 저장된 모든 데이터의 개수를 카운팅하기
# BITCOUNT [key]
$ BITCOUNT user:attend
```

@naver.com
khb3025

📖

Set과 Bitmap의 공통점 / 차이점

✅ Set vs Bitmap

[공통점]

- Set과 Bitmap은 둘 다 **중복을 허용하지 않는 자료구조**라는 점이 공통점이다.

[차이점]

- Set은 실제 값을 있는 그대로 저장하는 자료구조이다. 문자열, 숫자, UUID 등 다양한 형태의 데이터를 그대로 담을 수 있다. 반면, Bitmap은 값을 있는 그대로 저장하지 않고, 정수 offset에 대응되는 bit(0또는 1)만 기록하는 구조이다. 그래서 문자열, UUID와 같은 값은 저장할 수 없다.
- Set은 member 하나하나가 메모리를 차지하기 때문에 데이터 수가 늘어날수록 메모리 사용량이 선형적으로 증가한다. 반면, Bitmap은 데이터 하나당 1bit만 사용하므로 대규모 데이터를 저장해야 하는 상황에서도 매우 적은 메모리로 데이터를 저장할 수 있다. 다만, Bitmap은 0과 1만 저장할 수 있고 문자열, UUID와 같은 값은 저장할 수 없다는 제약이 있다.
- Set은 데이터 목록 조회, 집합 연산 등 데이터 자체를 다룰 수 있는 명령어를 제공한다. 반면, Bitmap은 데이터 목록을 조회할 수 없고 bit로 저장되기 때문에 데이터를 사람이 읽기도 불편하지만, 대량의 boolean 상태를 적은 메모리 용량으로 처리하는 데 특화되어 있다.

✅ 정리

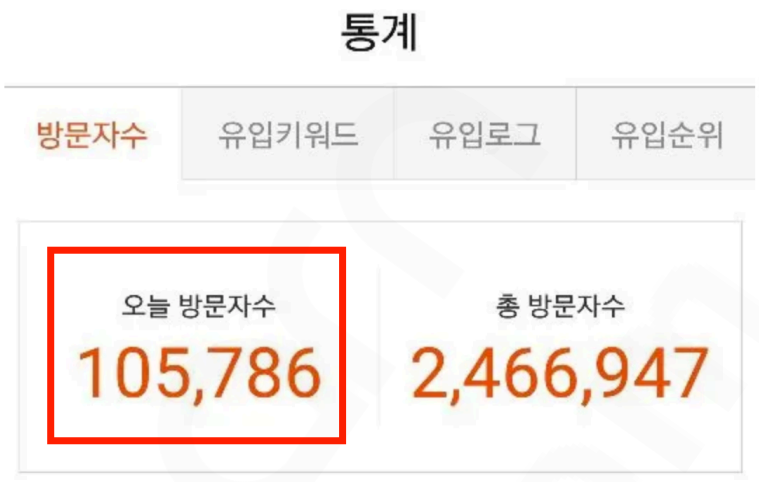
Set은 **"중복 없는 데이터의 집합을 관리하고 싶을 때"** 적합한 자료구조이다.

Bitmap은 **"수많은 데이터에 대해 true/false 상태만 효율적으로 기록하고 싶을 때"** 적합한 자료구조이다.

khb3025@gmail.com

[실습] 하루 동안 서비스에 방문한 사용자 수(DAU) 카운팅하기 - 1

✅ 문제 상황



서비스를 운영하다보면 **하루 동안 서비스에 방문한 사용자 수(DAU, Daily Active Users)**를 파악해야 할 때가 많다. 그런데 하루 방문자 수를 카운팅하고 조회할 때마다 RDB(ex. MySQL)로 처리하면 트래픽이 많아질수록 성능 부담이 커진다. 정말 그런지 테스트해보자.

✅ 실습

1. **코드 살펴보기**

> 실습의 편의를 위해 일부 코드를 작성해두었다.

DailyActiveUser

```
@Entity(name = "daily_active_users")
@Getter
@NoArgsConstructor
public class DailyActiveUser {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private Long userId;

  private LocalDate activeDate;

  public DailyActiveUser(Long userId, LocalDate activeDate) {
    this.userId = userId;
    this.activeDate = activeDate;
  }
}
```

DailyActiveUserController

```
@RestController
@RequestMapping("/dau")
```

```
@RequiredArgsConstructor
public class DailyActiveUserController {

  private final DailyActiveUserService dailyActiveUserService;

  // 활동 유저 기록 API
  @PostMapping("/record")
  public void recordActiveUser(
      @RequestBody RecordActiveUserRequestDto recordActiveUserRequestDto
  ) {
    dailyActiveUserService.recordActiveUser(recordActiveUserRequestDto.getUserId());
  }

  // DAU 조회 API
  @GetMapping("/count")
  public long getDau(
      @RequestParam
      @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
      LocalDate date
  ) {
    return dailyActiveUserService.getDau(date);
  }
}
```

DailyActiveUserRequestDto

```
@Getter
@NoArgsConstructor
public class RecordActiveUserRequestDto {
    private Long userId;
}
```

DailyActiveUserService

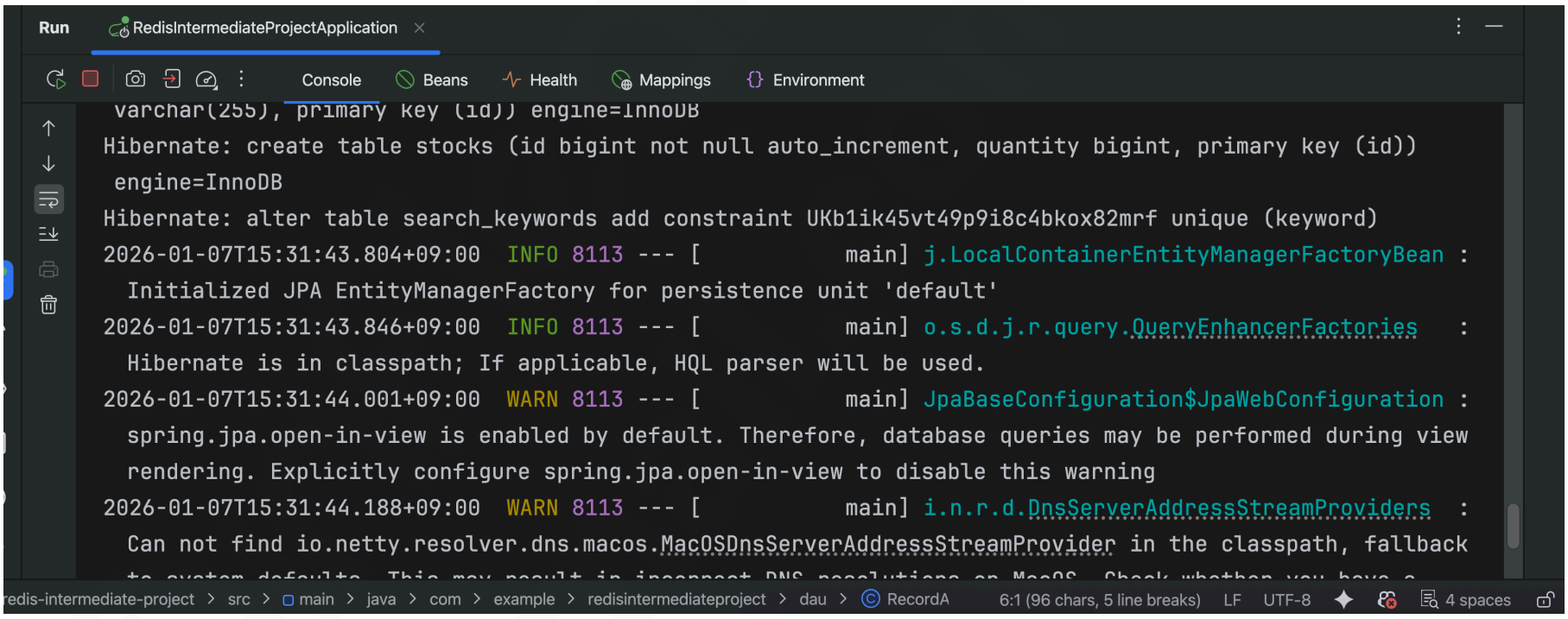
```
@Service
@RequiredArgsConstructor
public class DailyActiveUserService {

private final DailyActiveUserRepository dailyActiveUserRepository;

@Transactional
public void recordActiveUser(Long userId) {
  LocalDate today = LocalDate.now();
  // 이미 활동(방문) 기록이 있는 유저라면 저장하지 않는다.
  if (!dailyActiveUserRepository.existsByUserIdAndActiveDate(userId, today)) {
    // 활동 기록 저장
    dailyActiveUserRepository.save(new DailyActiveUser(userId, today));
  }
}

@Transactional(readOnly = true)
public long getDau(LocalDate date) {
  // 특정 날짜의 DAU를 SQL문의 COUNT를 활용해 계산
  return dailyActiveUserRepository.countByActiveDate(date);
```

```
  }
}
```

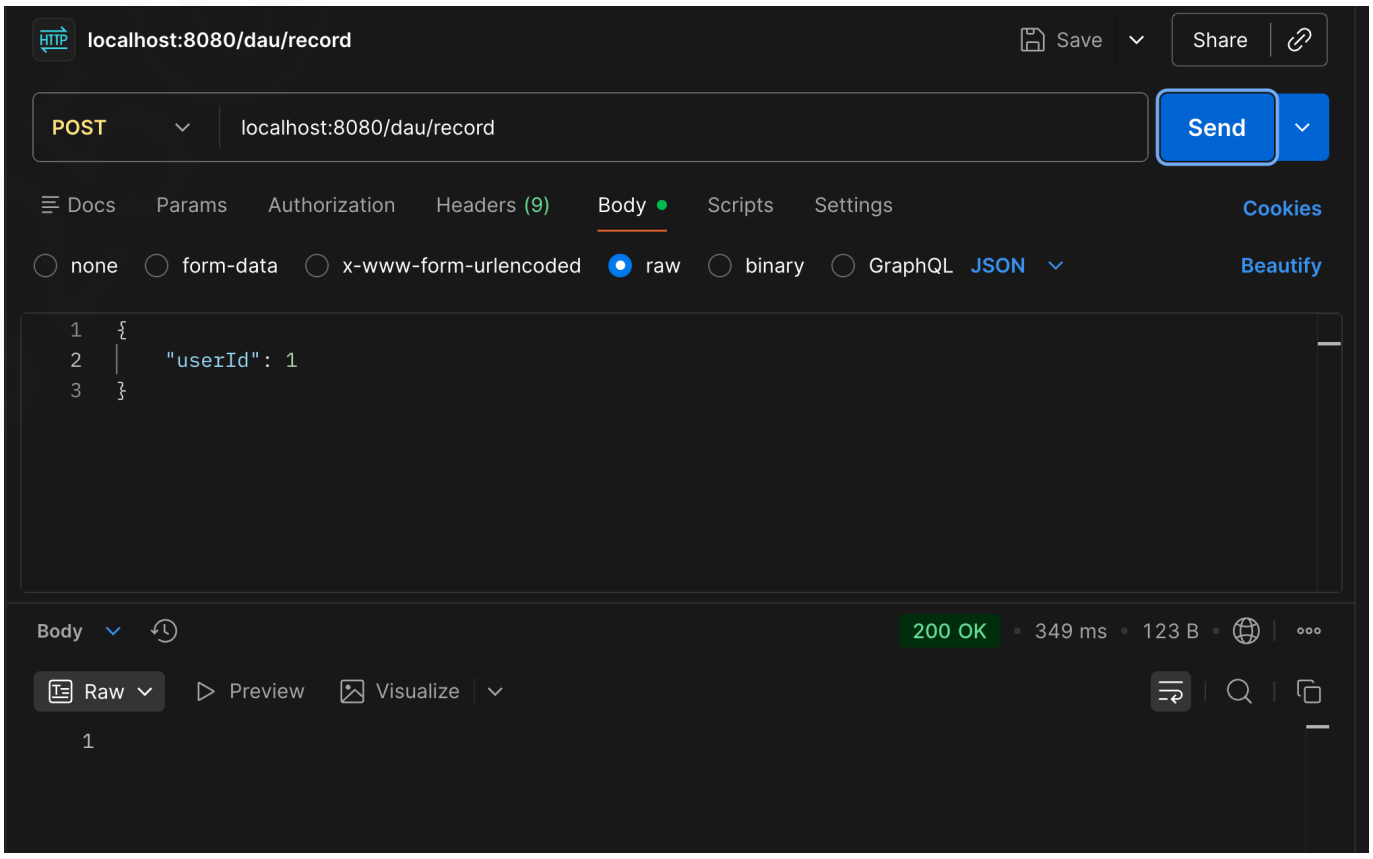
DailyActiveUserRepository

```
public interface DailyActiveUserRepository extends JpaRepository<DailyActiveUser, Long> {
    // 특정 사용자가 특정 날짜에 활동(방문)했는지 확인
    boolean existsByUserIdAndActiveDate(Long userId, LocalDate activeDate);

    // 특정 날짜의 DAU를 계산
    long countByActiveDate(LocalDate activeDate);
}
```

2. **Spring Boot 서버 실행시키기**

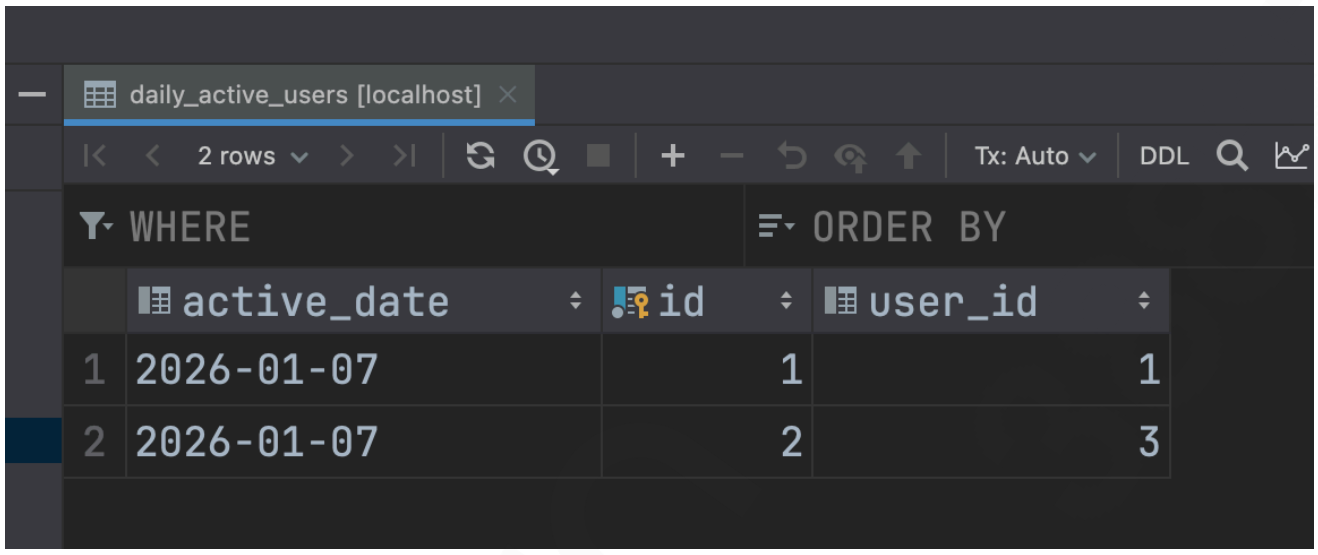


3. **Postman으로 잘 작동하는 지 확인하기**
    a. **활동 유저 기록 API**

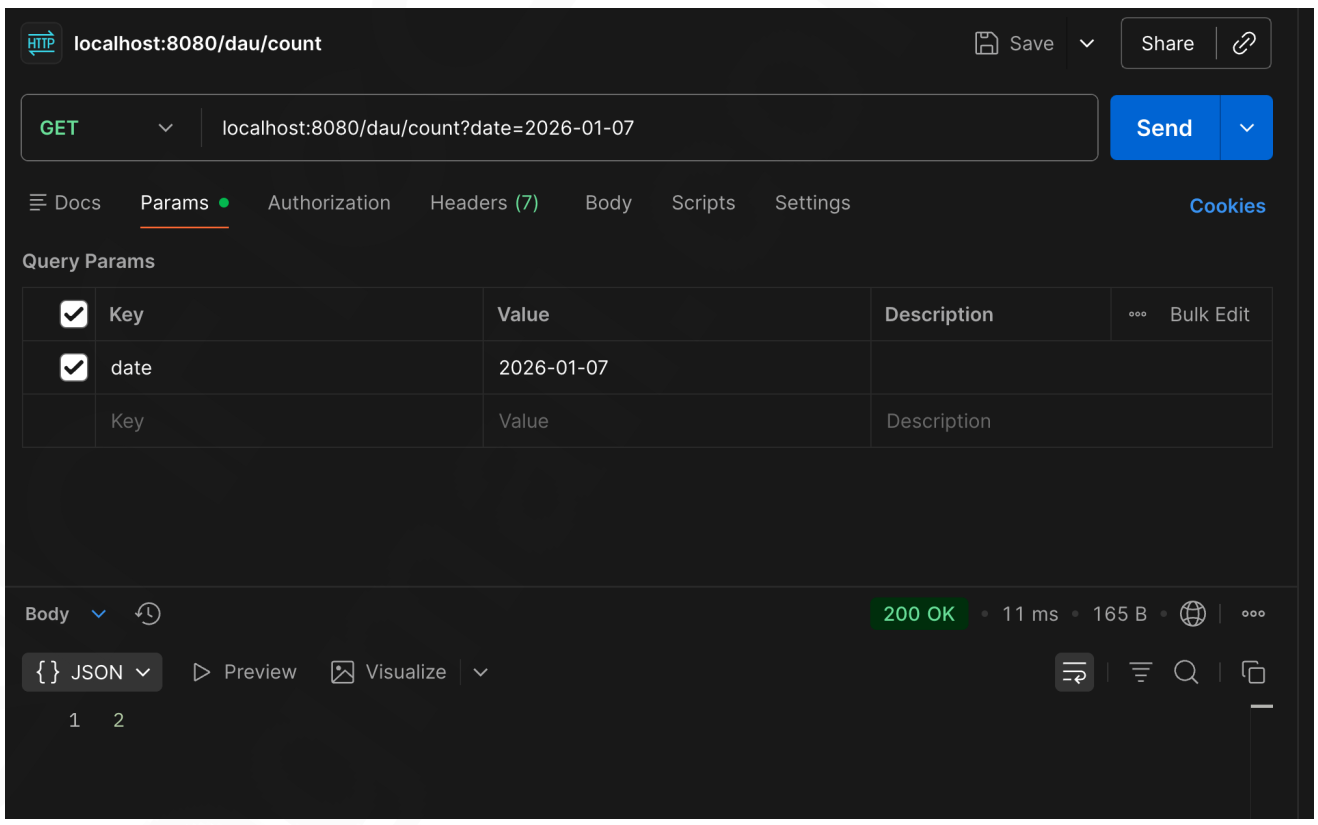


→ 데이터를 2개 넣어주자.

b. **DB 확인하기**



c. **DAU 조회 API**



4. 부하 테스트 스크립트 살펴보기

scripts/script_4-1.js

```
import http from 'k6/http';
import {check} from 'k6';

export const options = {
    // 가상 유저(VUs) 1000명으로 설정
  vus: 1000,

  // 테스트를 10초 동안 진행
  duration: '10s',
};

export default function () {
  const userId = Math.floor(Math.random() * 100000) + 1; // 1 ~ 100000 사이의 랜덤 userId
  const payload = JSON.stringify({
    userId: userId,
  });
```

```
const params = {
  headers: {
    'Content-Type': 'application/json',
  },
};

  // 1. 유저 활동 기록 API 호출
const recordRes = http.post('http://localhost:8080/dau/record', payload, params);

check(recordRes, {
  'record status is 200': (r) => r.status === 200,
});

// 오늘 날짜를 YYYY-MM-DD 형식으로 가져온다.
const today = new Date().toISOString().split('T')[0];

// 2. DAU 조회 API 호출
  const countRes = http.get(`http://localhost:8080/dau/count?date=${today}`);

  check(countRes, {
    'count status is 200': (r) => r.status === 200,
  });
}
```

4. **부하 테스트 진행하기**

```
$ k6 run scripts/script_4-1.js
```

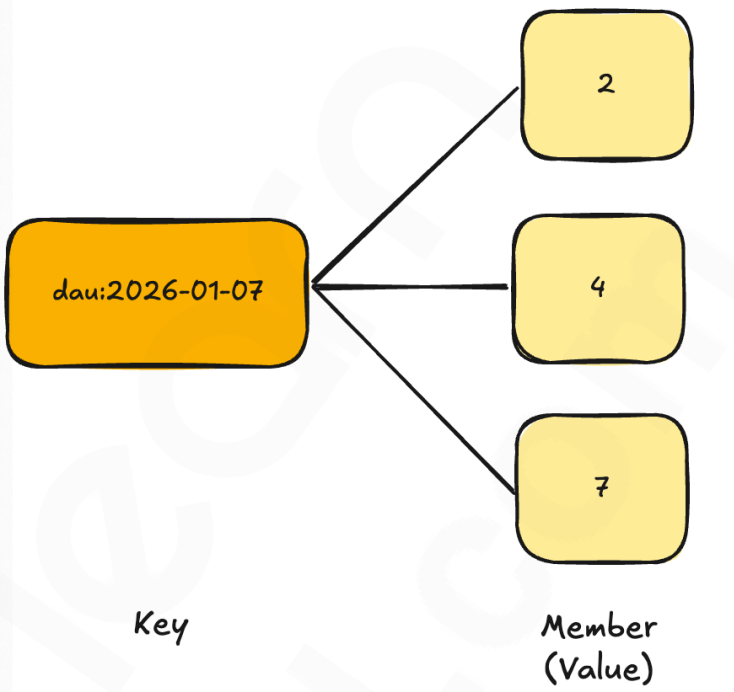
```
checks.........................: 100.00% ✓ 2386       ✗ 0
data_received..................: 240 kB  13 kB/s
data_sent......................: 315 kB  17 kB/s
http_req_blocked...............: avg=24.89ms  min=0s     med=6µs    max=114.53ms p(90)=97.45ms p(95)=103.09ms
http_req_connecting............: avg=24.67ms  min=0s     med=0s     max=112.01ms p(90)=97.42ms p(95)=103.01ms
http_req_duration..............: avg=5.9s     min=5.67ms med=7s     max=12.7s    p(90)=8.68s   p(95)=9.19s
  { expected_response:true }...: avg=5.9s     min=5.67ms med=7s     max=12.7s    p(90)=8.68s   p(95)=9.19s
http_req_failed................: 0.00%   ✓ 0          ✗ 2386
http_req_receiving.............: avg=59.48µs  min=9µs    med=45µs   max=4.49ms   p(90)=110µs   p(95)=134.49µs
http_req_sending...............: avg=905.69µs min=4µs    med=26µs   max=19.03ms  p(90)=2.17ms  p(95)=6.25ms
http_req_tls_handshaking.......: avg=0s       min=0s     med=0s     max=0s       p(90)=0s      p(95)=0s
http_req_waiting...............: avg=5.9s     min=5.33ms med=7s     max=12.7s    p(90)=8.68s   p(95)=9.19s
http_reqs......................: 2386    125.094594/s
iteration_duration.............: avg=11.86s   min=1.61s  med=12.17s max=18.45s   p(90)=14.97s  p(95)=15.75s
iterations.....................: 1193    62.547297/s
vus............................: 29      min=29       max=1000
vus_max........................: 1000    min=1000     max=1000
```

총 2386번의 요청을 처리했음을 알 수 있고, 유저 활동 기록 API와 DAU 조회 API의 평균 TPS가 125 정도가 나온 걸 확인할 수 있다.

현재 TPS로는 성능이 부족한 상황이라고 가정하고, 다음 강의에서는 Redis를 활용해 성능을 개선해보자.

[실습] 하루 동안 서비스에 방문한 사용자 수(DAU) 카운팅하기 - 2

✅ 구현 방법



- 하루동안 서비스에 방문한 사용자 수를 카운팅하려면, 하루에 어떤 사용자가 방문했는 지 데이터로 저장해두어야 한다. 이 때, 한 명의 사용자가 여러 번 방문하더라도 한 번으로 카운팅해야 하기 때문에 **중복없이** 저장해야 한다. 또한 **순서와는 상관없이** 데이터를 저장해도 된다. 이에 가장 적절한 Redis의 자료구조는 **Set**이다.
- 하루 단위로 방문한 사용자 수를 기록해야 하므로, 하루 단위로 구분할 수 있게 `key`에는 날짜값을 포함시킨다.
- 방문한 사용자를 기록하기 위해 `value`에는 `사용자 id`를 저장한다.
- 하루동안 서비스에 방문한 사용자 수를 카운팅할 때는 Set에 저장된 데이터 개수를 세알리면 된다.

✅ 실습

1. **코드 작성하기**

DailyActiveUserService

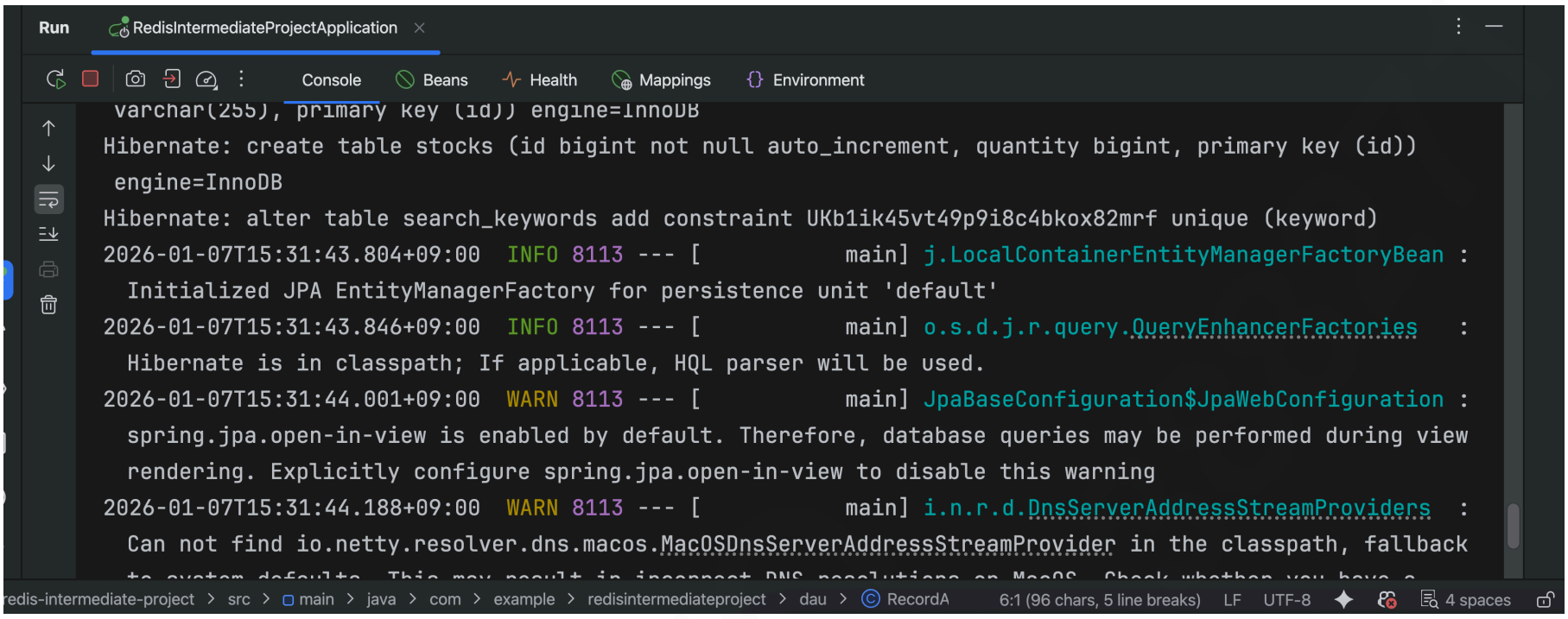
```
@Service
@RequiredArgsConstructor
public class DailyActiveUserService {

  private final DailyActiveUserRepository dailyActiveUserRepository;
  private final RedisTemplate<String, String> redisTemplate;

  ...

  public void recordActiveUserWithRedis(Long userId) {
    LocalDate today = LocalDate.now();
    String key = "dau:" + today.toString(); // dau:YYYY-MM-DD
    // Redis에 userId를 Set 자료구조로 저장
    // Redis 명령어의 'SADD [key] [value]'와 동일하다.
    redisTemplate.opsForSet().add(key, userId.toString());
```

```
  }

  public long getDauWithRedis(LocalDate date) {
    String key = "dau:" + date.toString();
    // Redis에 저장된 Set을 활용하여 DAU를 계산
    // Redis 명령어의 'SCARD [key]'와 동일하다.
    return redisTemplate.opsForSet().size(key);
  }
}
```

DailyActiveUserController

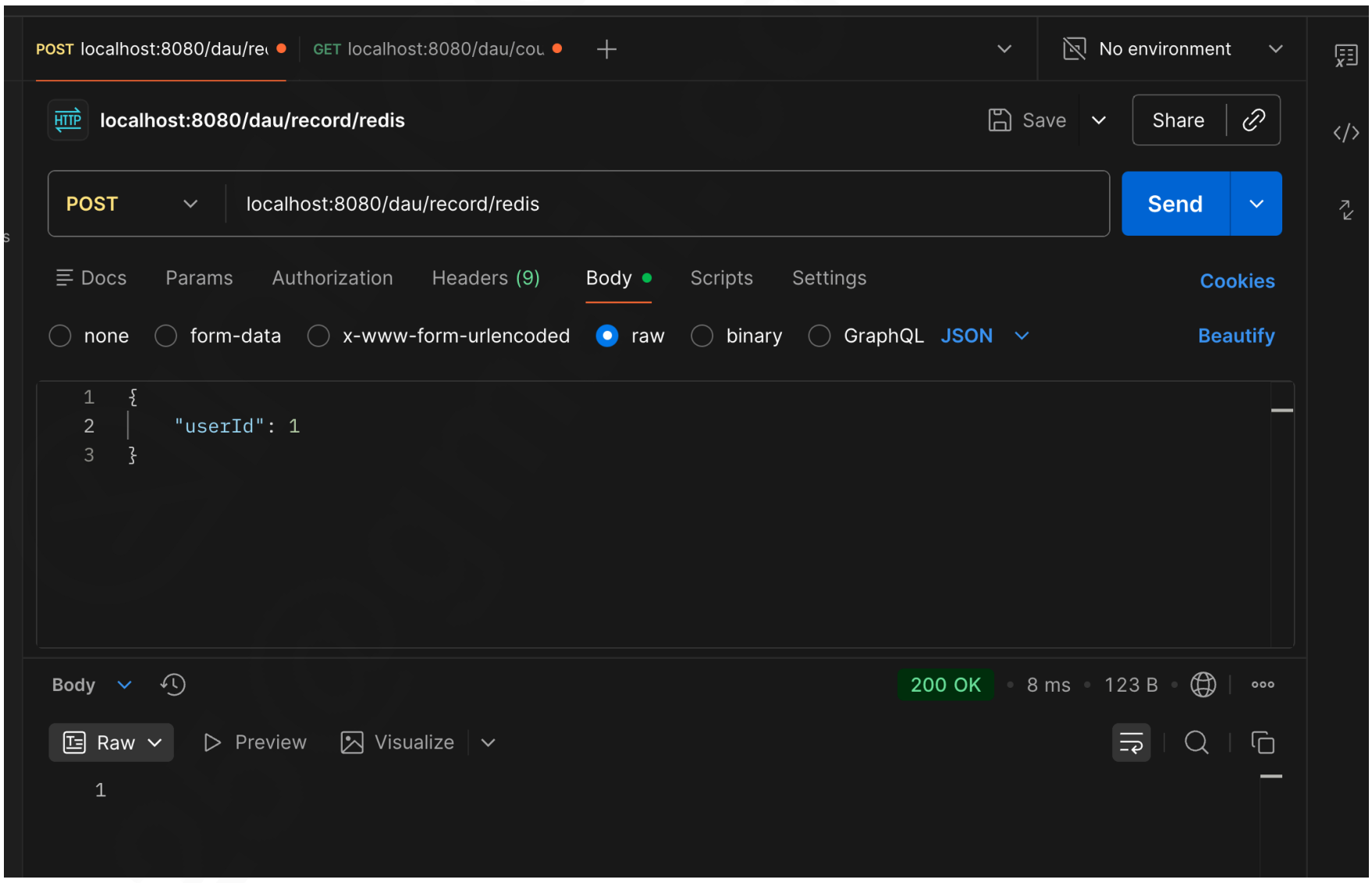
```
@RestController
@RequestMapping("/dau")
@RequiredArgsConstructor
public class DailyActiveUserController {

  private final DailyActiveUserService dailyActiveUserService;

  ...

  // 활동 유저 기록 API
  @PostMapping("/record/redis")
  public void recordActiveUserWithRedis(
      @RequestBody RecordActiveUserRequestDto recordActiveUserRequestDto
  ) {
    dailyActiveUserService.recordActiveUserWithRedis(recordActiveUserRequestDto.getUserId
());
  }

  // DAU 조회 API
  @GetMapping("/count/redis")
  public long getDauWithRedis(
      @RequestParam
      @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
      LocalDate date
  ) {
    return dailyActiveUserService.getDauWithRedis(date);
  }
}
```

2. **Spring Boot 서버 실행시키기**



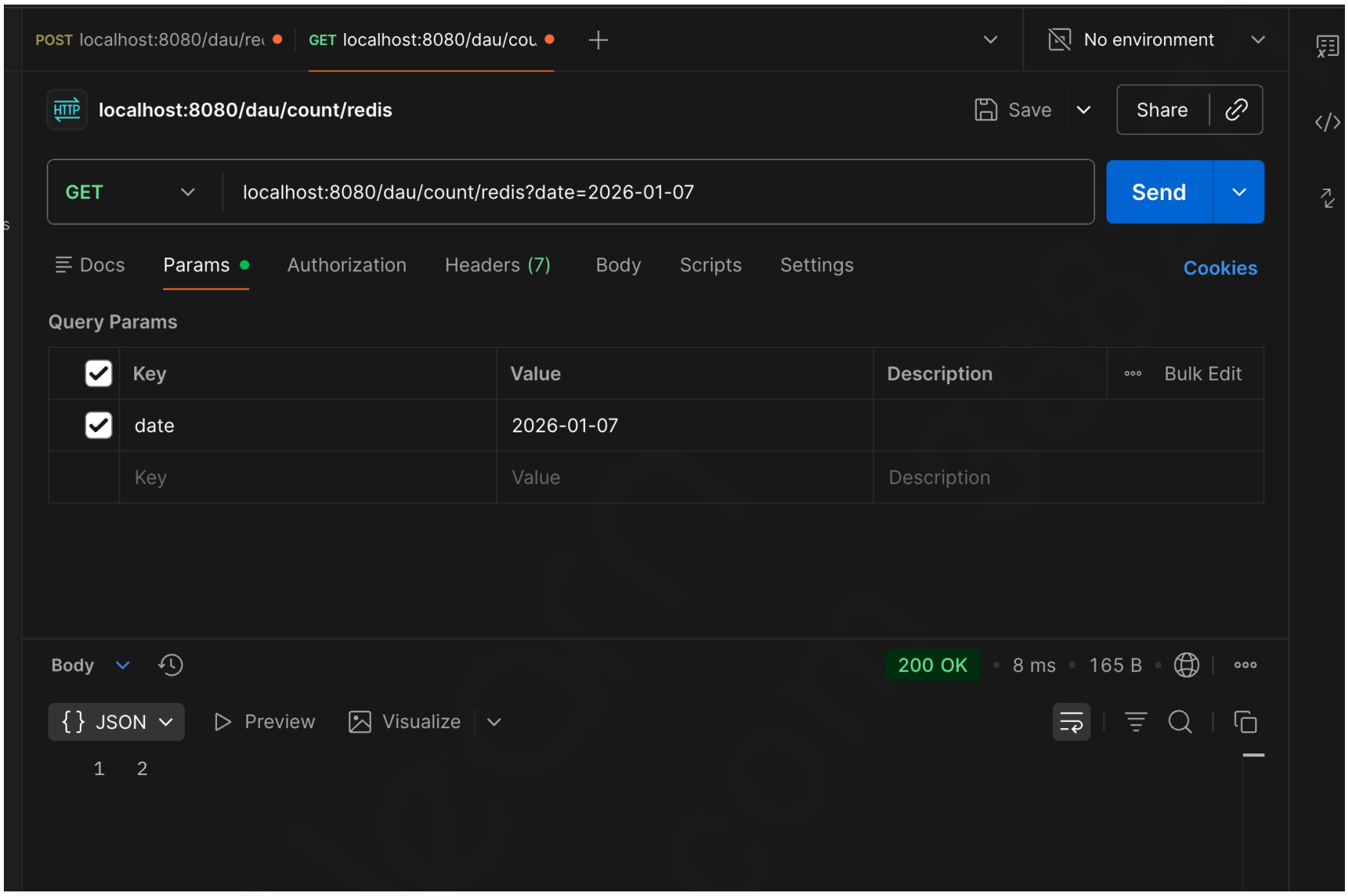
3. **Postman으로 잘 작동하는 지 확인하기**

a. **활동 유저 기록 API**



→ 데이터를 2개 넣어주자.

b. **DAU 조회 API**



4. **부하 테스트 스크립트 살펴보기**

   scripts/script_4-2.js

```
import http from 'k6/http';
import {check} from 'k6';

export const options = {
  vus: 1000,
  duration: '10s',
};

export default function () {
  // 1. 유저 활동 기록
const userId = Math.floor(Math.random() * 100000) + 1; // 1 ~ 100000 사이의 랜덤 userId
const payload = JSON.stringify({
  userId: userId,
});

const params = {
  headers: {
    'Content-Type': 'application/json',
  },
};

const recordRes = http.post('http://localhost:8080/dau/record/redis', payload, params);

check(recordRes, {
  'record status is 200': (r) => r.status === 200,
});

// 2. DAU 조회
```

```
  // 오늘 날짜를 YYYY-MM-DD 형식으로 가져온다.
  const today = new Date().toISOString().split('T')[0];
  const countRes = http.get(`http://localhost:8080/dau/count/redis?date=${today}`);

  check(countRes, {
    'count status is 200': (r) => r.status === 200,
  });
}
```

이전 부하 테스트 스크립트와 비교했을 때 API 주소만 바뀌었다.

4. **부하 테스트 진행하기**

```
$ k6 run scripts/script_4-2.js
```

```
checks.........................: 100.00% ✓ 338192       ✗ 0
data_received..................: 34 MB   3.4 MB/s
data_sent......................: 47 MB   4.6 MB/s
http_req_blocked...............: avg=215.78µs min=0s       med=1µs     max=229.21ms p(90)=1µs     p(95)=2µs
http_req_connecting............: avg=213.86µs min=0s       med=0s      max=229.17ms p(90)=0s      p(95)=0s
http_req_duration..............: avg=29.35ms  min=272µs    med=26.82ms max=133.23ms p(90)=38.61ms p(95)=51.75ms
  { expected_response:true }...: avg=29.35ms  min=272µs    med=26.82ms max=133.23ms p(90)=38.61ms p(95)=51.75ms
http_req_failed................: 0.00%   ✓ 0            ✗ 338192
http_req_receiving.............: avg=21.18µs  min=-46000ns med=7µs     max=40.7ms   p(90)=36µs    p(95)=59µs
http_req_sending...............: avg=7.2µs    min=1µs      med=2µs     max=8.81ms   p(90)=6µs     p(95)=8µs
http_req_tls_handshaking.......: avg=0s       min=0s       med=0s      max=0s       p(90)=0s      p(95)=0s
http_req_waiting...............: avg=29.32ms  min=268µs    med=26.8ms  max=133.22ms p(90)=38.58ms p(95)=51.69ms
http_reqs......................: 338192  33699.307972/s
iteration_duration.............: avg=59.2ms   min=1.05ms   med=54.51ms max=400.05ms p(90)=69.41ms p(95)=107.34ms
iterations.....................: 169096  16849.653986/s
vus............................: 1000    min=1000       max=1000
vus_max........................: 1000    min=1000       max=1000
```

부하테스트를 해보니 평균 TPS가 33699으로 성능이 눈에 띄게 향상된 걸 확인할 수 있다.

6. **Set에 저장된 데이터가 차지하는 메모리량 체크하기**

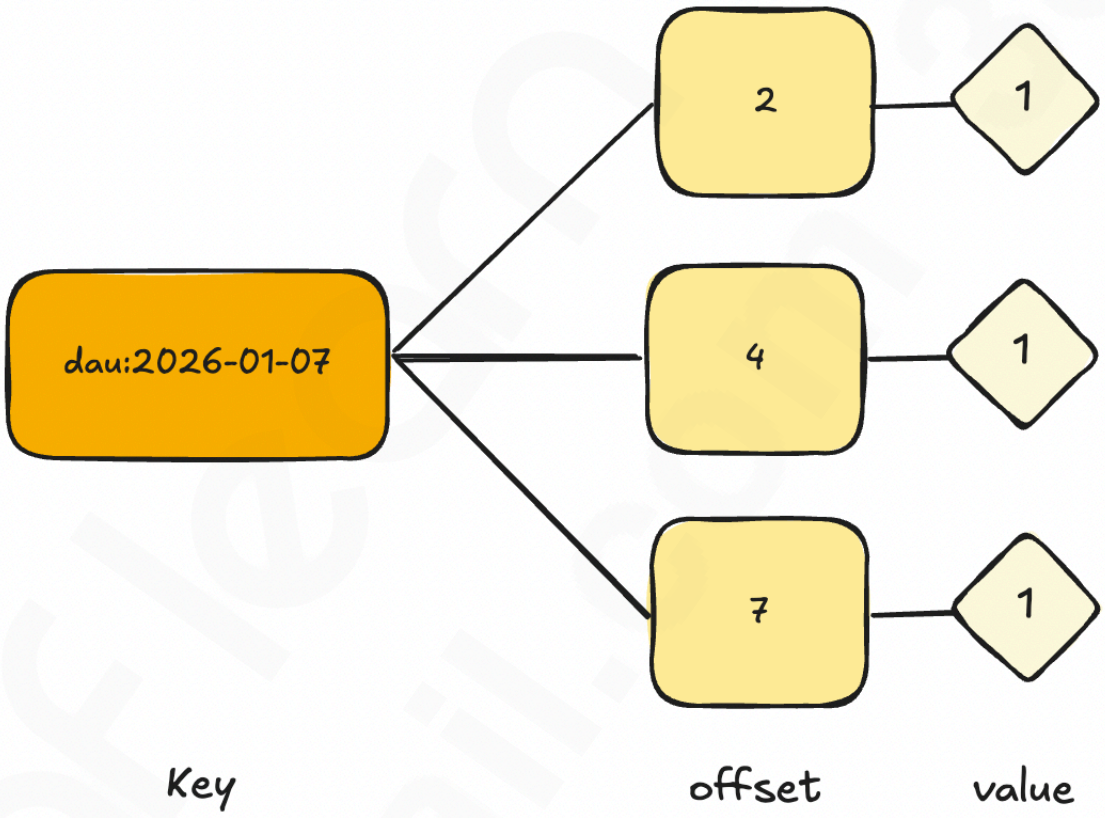
```
$ keys * # 데이터가 저장된 key 확인하기

$ SCARD dau:2026-01-08 # 해당 Set에 저장된 데이터 개수 확인하기

# MEMORY USAGE [key]
$ MEMORY USAGE dau:2026-01-08 # 특정 key가 차지하고 있는 메모리 확인
```

```
jaeseong ~/Documents/Develop/redis-intermediate-project main ± redis-cli
127.0.0.1:6379> keys *
1) "user:attend"
2) "member"
3) "dau:2026-01-08"
127.0.0.1:6379> SCARD dau:2026-01-08
(integer) 81548
127.0.0.1:6379> memory usage dau:2026-01-08
(integer) 3567613
```

부하 테스트를 통해 저장된 데이터의 총 개수는 81548개이고, 해당 key 하나가 차지하는 메모리는 3567613 Byte인 걸 알 수 있다. 다음 강의 때 이 값들을 비교할 예정이기 때문에 이 값을 어딘가에 잘 적어두자.

[실습] 하루 동안 서비스에 방문한 사용자 수(DAU) 카운팅하기 - 3

✅ 구현 방법



- 하루동안 서비스에 방문한 사용자 수를 카운팅하려면 사용자 별로 방문을 했는 지 여부를 기록으로 남겨야 한다. 방문을 했는 지 여부는 `true` / `false` 로 표현할 수 있기 때문에 Redis의 자료구조 중에서 Bitmap을 활용한다.
- 하루 단위로 방문한 사용자 수를 기록해야 하므로, 하루 단위로 구분할 수 있게 `key` 에는 날짜값을 포함시킨다.
- `offset` 은 사용자의 id로 기록을 하고, `value` 에는 방문을 했다면 `1` 로 기록을 한다.
- 하루동안 서비스에 방문한 사용자 수를 카운팅할 때는 특정 Bitmap에 저장된 `offset` 의 모든 개수를 세알리면 된다.

✅ 실습

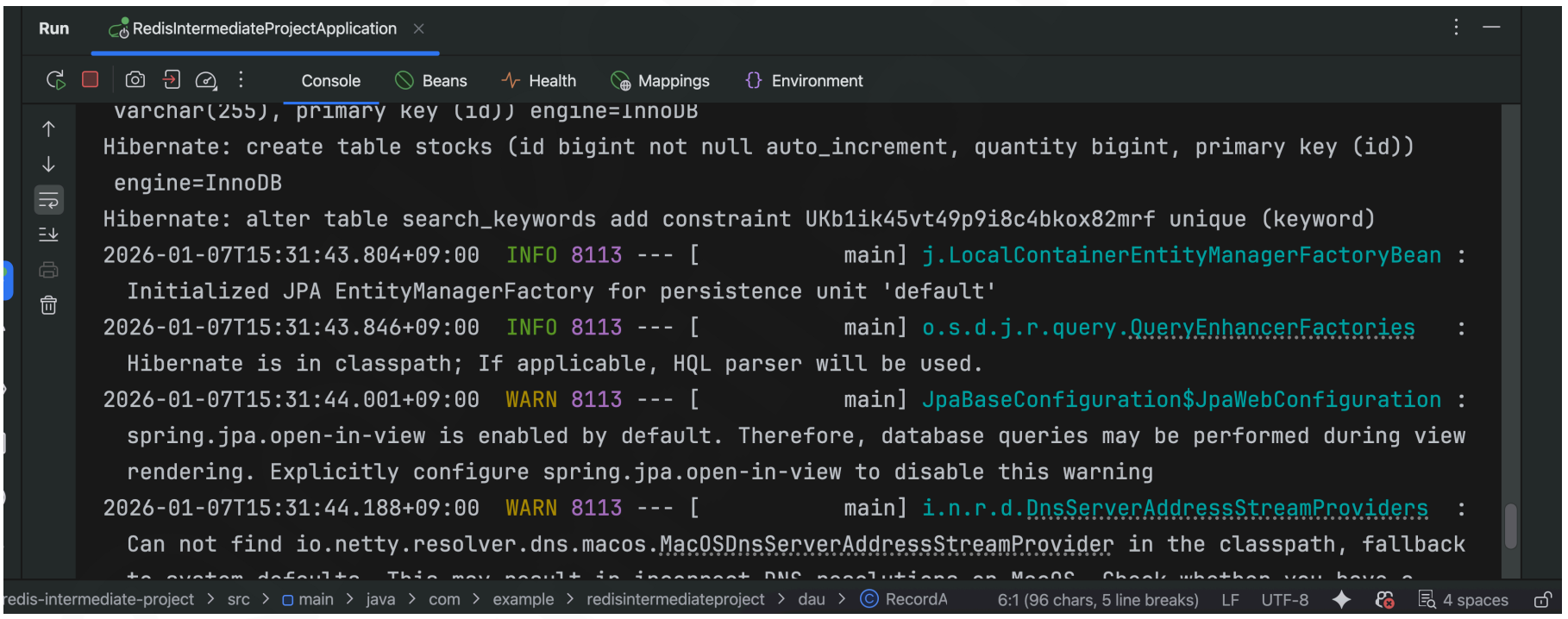
1. **코드 작성하기**

   DailyActiveUserService

```
@Service
@RequiredArgsConstructor
public class DailyActiveUserService {

  private final DailyActiveUserRepository dailyActiveUserRepository;
  private final RedisTemplate<String, String> redisTemplate;

  ...

  public void recordActiveUserWithRedis(Long userId) {
    LocalDate today = LocalDate.now();
    String key = "dau:" + today.toString(); // dau:YYYY-MM-DD
```

```
    // Redis에 userId를 Bitmap 자료구조로 저장
    // Redis 명령어의 'SETBIT [key] [offset] [value]'와 동일하다.
    redisTemplate.opsForValue().setBit(key, userId, true);
  }

  public long getDauWithRedis(LocalDate date) {
    String key = "dau:" + date.toString();
    // Redis에 저장된 비트맵을 활용하여 DAU를 계산
    // Redis 명령어의 'BITCOUNT [key]'와 동일하다.
    return redisTemplate.execute(
        (RedisCallback<Long>) (connection) -> connection.bitCount(key.getBytes())
    );
  }
}
```

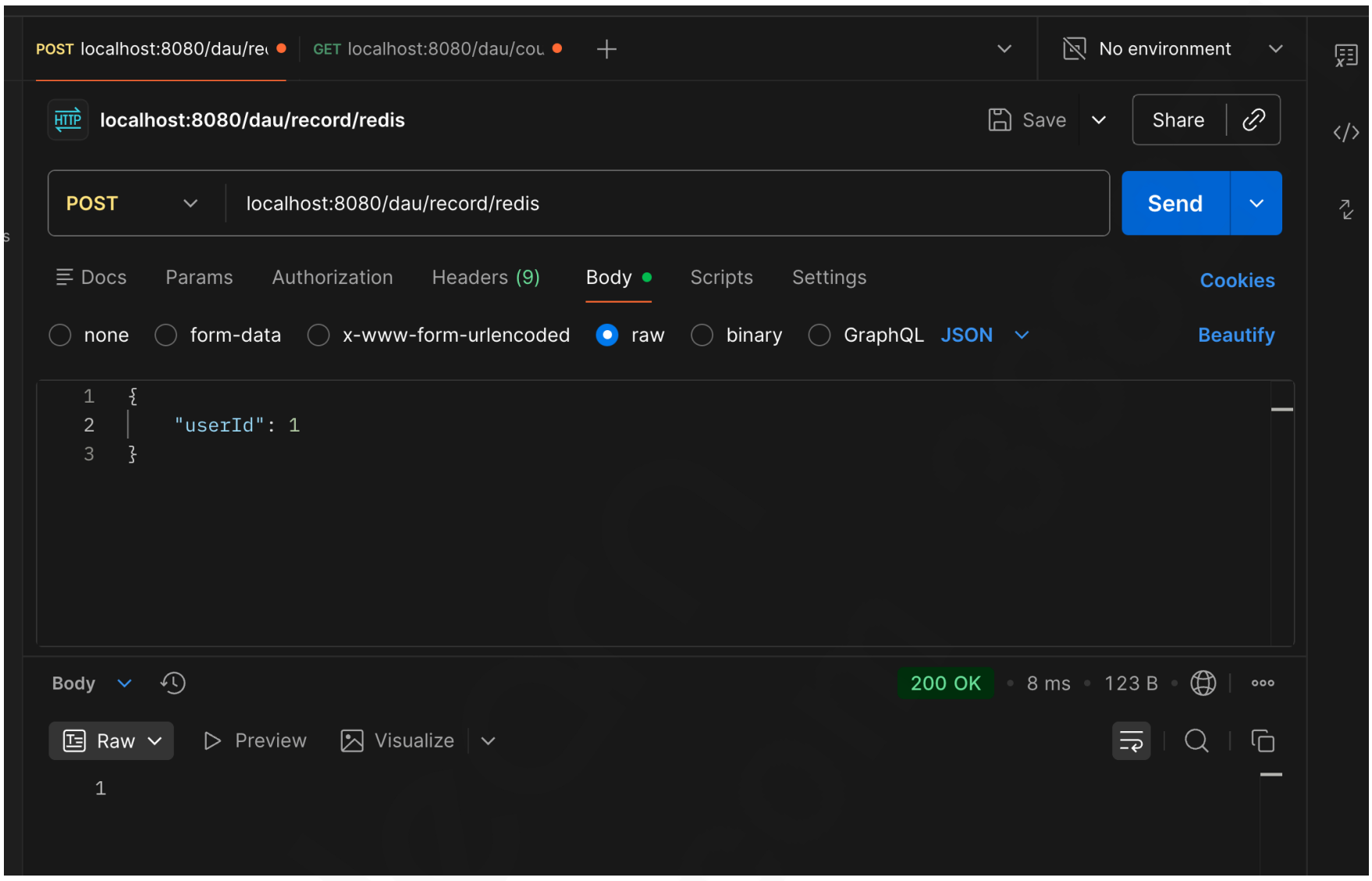
2. **Spring Boot 서버 실행시키기**



3. **기존 Redis 데이터 삭제하기**

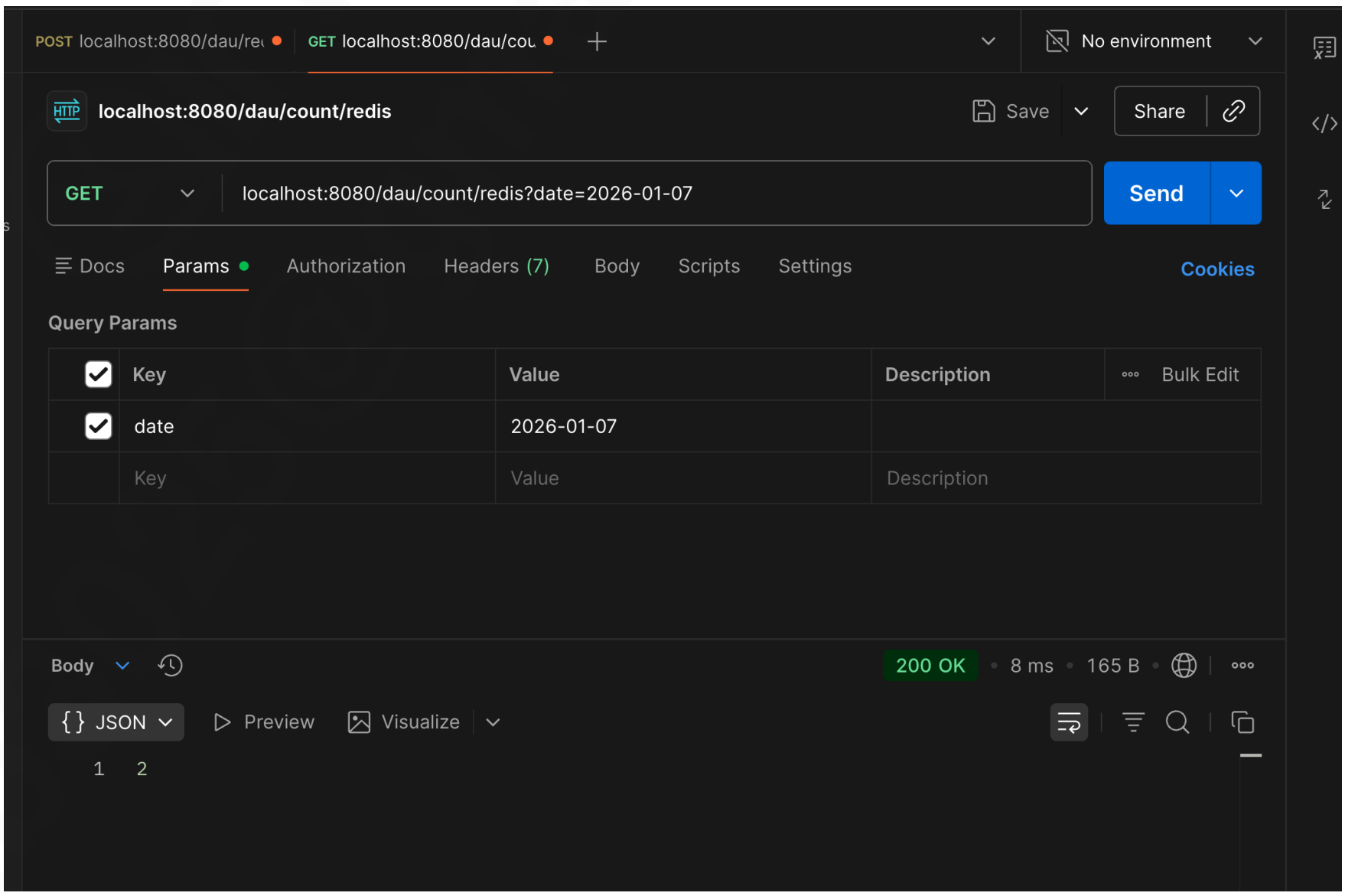
```
$ keys *
$ del dau:2026-01-08
```

4. **Postman으로 잘 작동하는 지 확인하기**
    a. **활동 유저 기록 API**



→ 데이터를 2개 넣어주자.

b. **DAU 조회 API**



4. 부하 테스트 진행하기

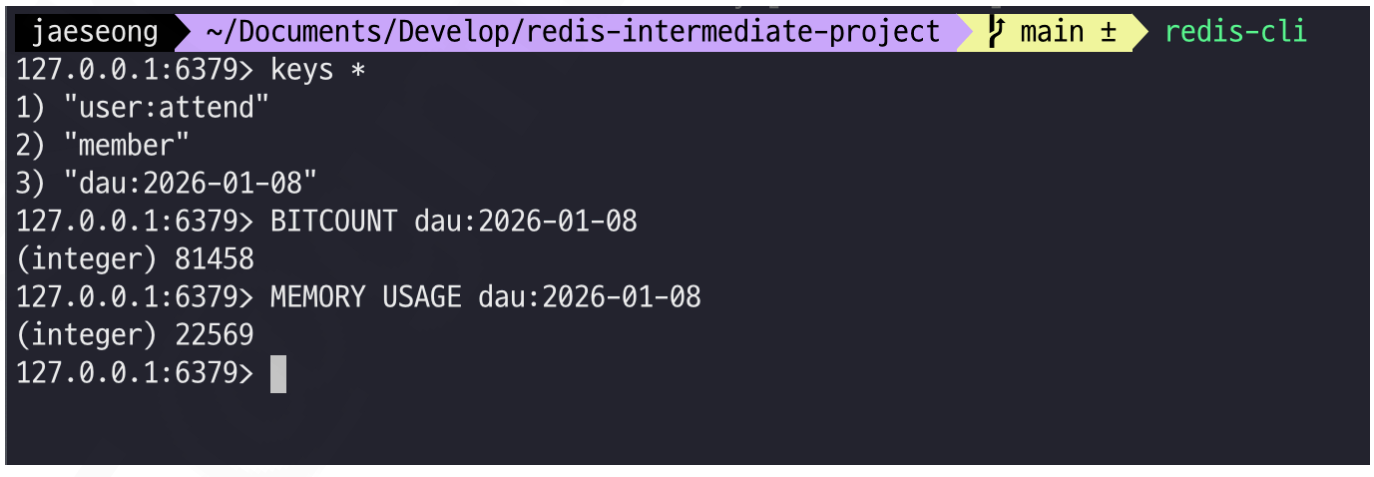
```
$ k6 run scripts/script_4-2.js
```

```
checks.........................: 100.00% ✓ 303096       ✗ 0
data_received..................: 31 MB   3.1 MB/s
data_sent......................: 42 MB   4.2 MB/s
http_req_blocked...............: avg=851.88µs min=0s       med=1µs     max=1.13s    p(90)=1µs     p(95)=2µs
http_req_connecting............: avg=848.57µs min=0s       med=0s      max=1.13s    p(90)=0s      p(95)=0s
http_req_duration..............: avg=32.14ms  min=207µs    med=29.22ms max=388.6ms  p(90)=42.11ms p(95)=50.58ms
  { expected_response:true }...: avg=32.14ms  min=207µs    med=29.22ms max=388.6ms  p(90)=42.11ms p(95)=50.58ms
http_req_failed................: 0.00%   ✓ 0            ✗ 303096
http_req_receiving.............: avg=22.74µs  min=2µs      med=8µs     max=42.68ms  p(90)=36µs    p(95)=61µs
http_req_sending...............: avg=4.78µs   min=-81000ns med=2µs     max=16.05ms  p(90)=6µs     p(95)=9µs
http_req_tls_handshaking.......: avg=0s       min=0s       med=0s      max=0s       p(90)=0s      p(95)=0s
http_req_waiting...............: avg=32.12ms  min=193µs    med=29.19ms max=388.29ms p(90)=42.08ms p(95)=50.51ms
http_reqs......................: 303096  30213.623404/s
iteration_duration.............: avg=66.06ms  min=1.41ms   med=59.24ms max=1.2s     p(90)=81.58ms p(95)=95.1ms
iterations.....................: 151548  15106.811702/s
vus............................: 1000    min=1000       max=1000
vus_max........................: 1000    min=1000       max=1000
```

부하테스트를 해보니 평균 TPS가 30213으로 Set 자료구조를 사용했을 때와 크게 차이가 나진 않는다.

6. **Bitmap에 저장된 데이터가 차지하는 메모리량 체크하기**

```
$ keys * # 데이터가 저장된 key 확인하기

$ BITCOUNT dau:2026-01-08 # 해당 Bitmap에 저장된 데이터 개수 확인하기

# MEMORY USAGE [key]
$ MEMORY USAGE dau:2026-01-08 # 특정 key가 차지하고 있는 메모리 확인
```



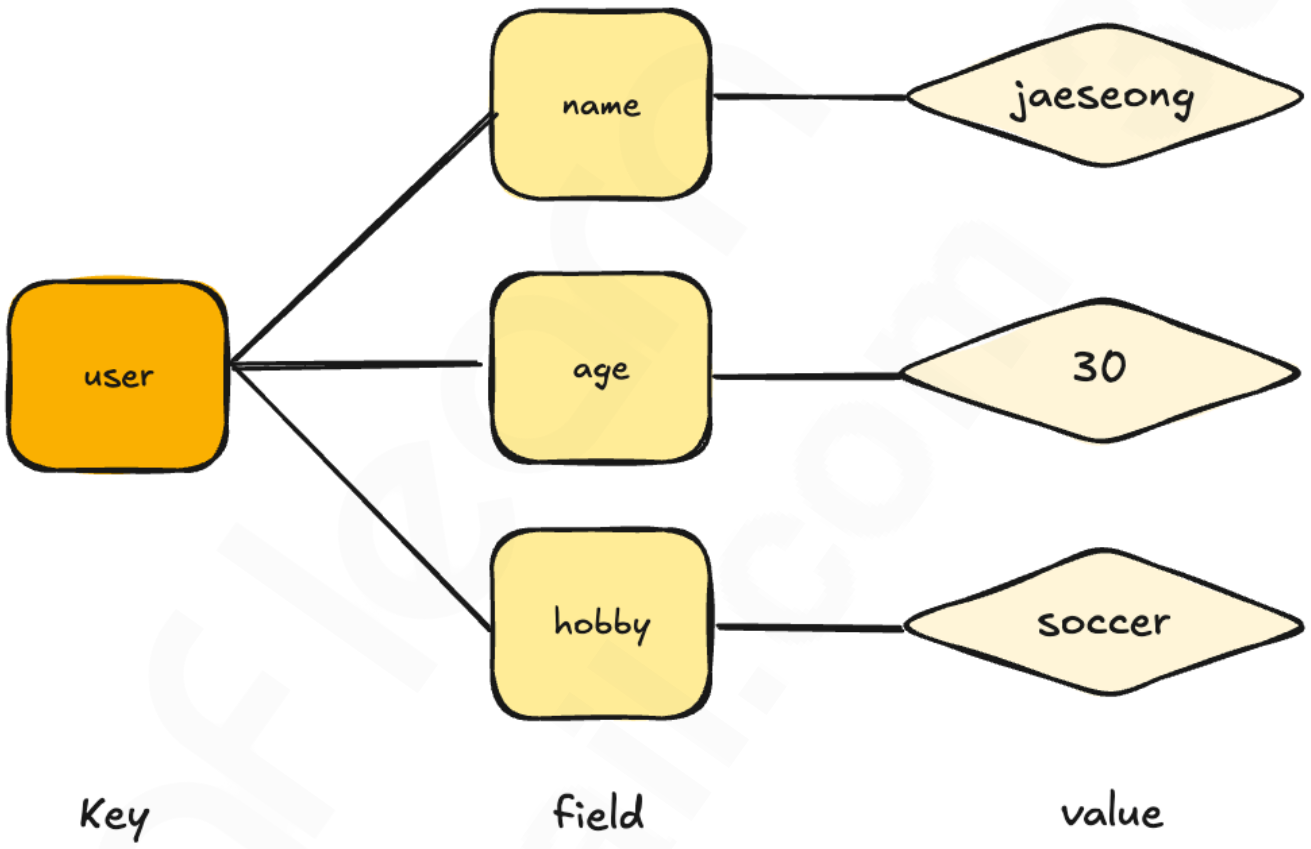
부하 테스트를 통해 저장된 데이터의 총 개수는 81458개로 Set으로 부하테스트를 진행했을 때와 개수는 비슷하다. 그리고 Bitmap에 저장된 데이터들이 차지하는 메모리는 22569 Byte이다. Set에 데이터를 저장했을 때보다 훨씬 메모리 양을 적게 차지하고 있다는 걸 확인할 수 있다.

📖

Hash 자료 구조 (HSET, HGET, HGETALL)

✅ Hash

Redis에서 **Hash**는 **하나의 key 안에 여러 개의 field-value 쌍을 저장할 수 있는 자료구조**이다. Hash는 json 객체의 구성과 유사해서, 여러 속성들을 묶어서 저장할 때 적합하다.



✅ Hash에 데이터 저장

```
# Hash에 데이터 저장
# HSET [key] [field] [value]
$ HSET user name jaeseong
$ HSET user age 30
$ HSET user hobby soccer
```

✅ Hash에 데이터 조회

```
$ keys * # Hash가 잘 생성됐는 지 확인하기

# Hash에 들어있는 모든 field, value를 조회
# HGETALL [key]
$ HGETALL user

# Hash에 들어있는 특정 field의 value를 조회
# HGET [key] [field]
$ HGET user name
```

```
$ HGET user age
$ HGET user hobby
```

@naver.com
khb3025

📖

서버가 2대 이상일 경우, Redis로 세션 스토리지를 구축해야 하는 이유

✅ 세션(Session)

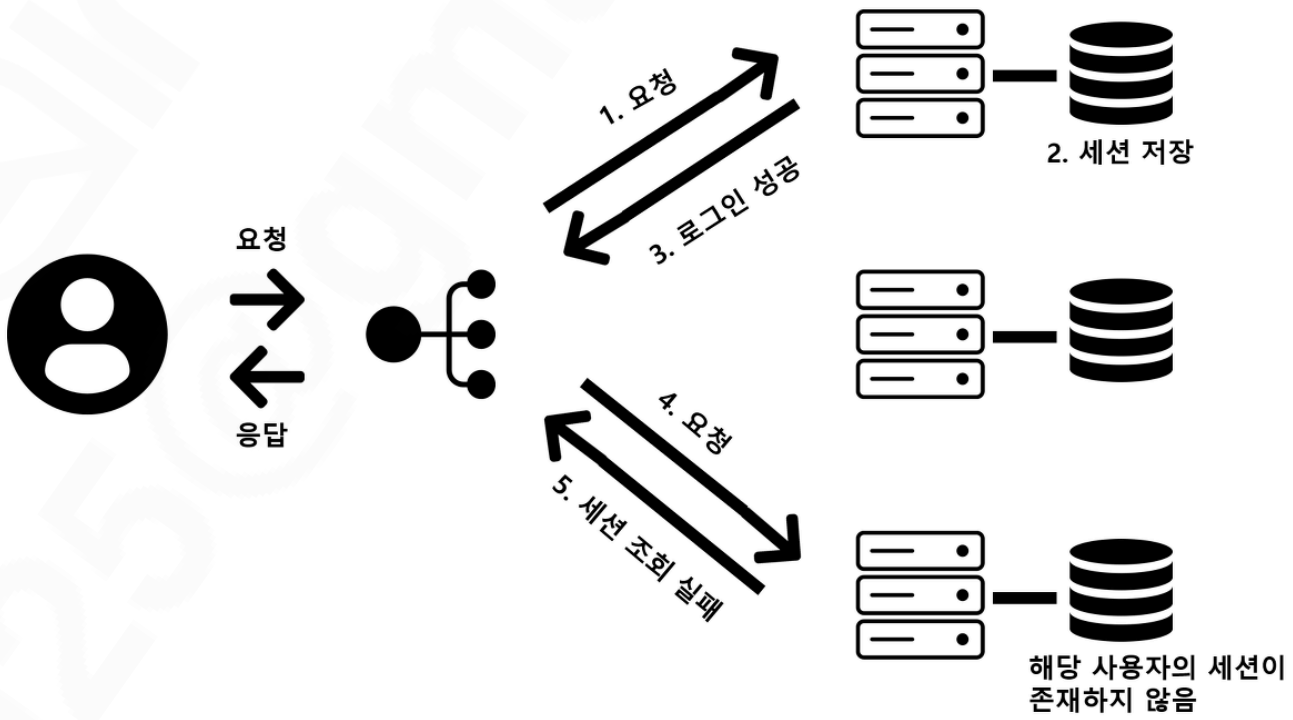
서비스에서 로그인 기능을 구현할 때 크게 2가지 방식으로 구현한다. JWT 방식과 세션(Session) 방식이 있다. 그 중에서 세션 방식은 다음과 같은 흐름으로 동작한다.

1. 사용자가 ID / 비밀번호로 로그인 요청
2. 서버는 인증이 성공하면 세션 ID를 생성한 뒤, 해당 세션 ID에 사용자 정보를 매핑하여 **세션 저장소**에 저장
3. 서버는 응답으로 세션 ID를 쿠키에 담아 클라이언트에게 전달
4. 이후 클라이언트는 요청마다 쿠키에 포함된 세션 ID를 함께 전송
5. 서버는 세션 ID를 기준으로 **세션 저장소**에서 사용자 정보를 조회해서 로그인을 처리

✅ 서버가 2대 이상일 경우, 로컬 세션 스토리지의 문제점

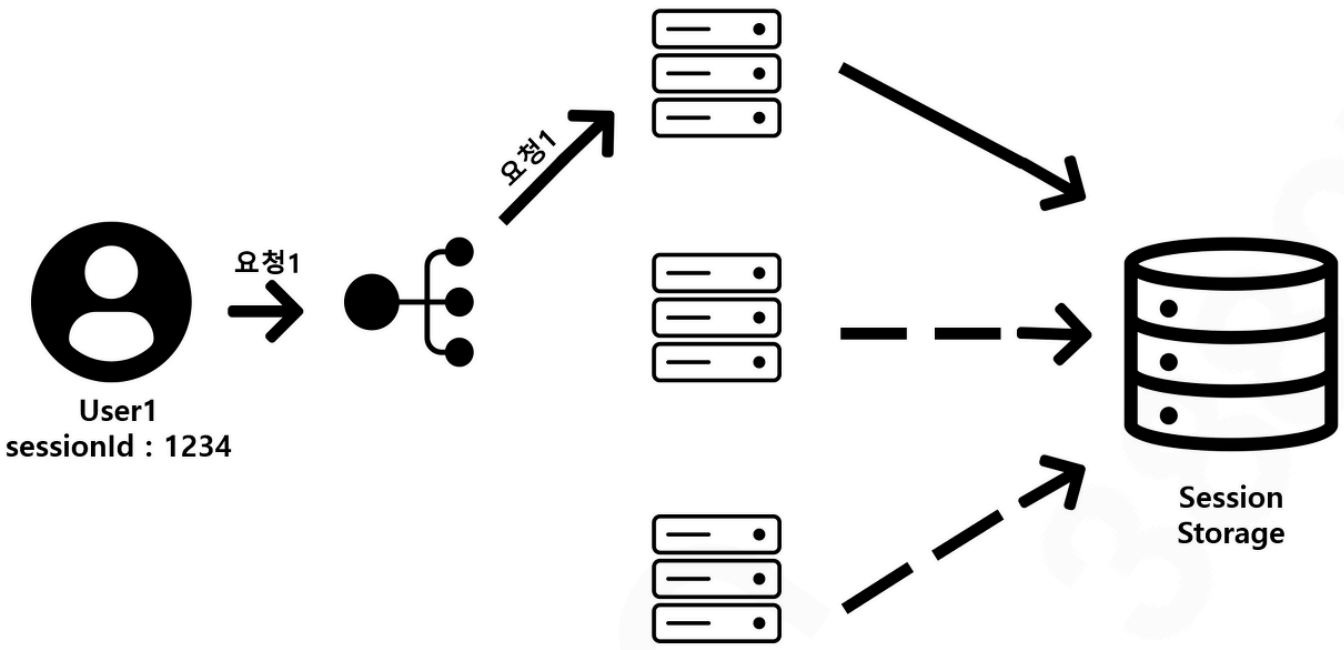
우리가 일반적으로 사용하는 프레임워크(ex. Spring Boot, Nest.js 등)를 살펴보면 각 프레임워크에 내장된 세션 저장소(= 로컬 세션 스토리지)가 존재한다. 그래서 Spring Boot로 예를 들자면, Spring Boot 서버의 일부 메모리 공간에 세션 정보가 저장된다.

이 로컬 스토리지 세션은 서버가 한 대일 경우에는 문제가 안 된다. 하지만 서버가 여러 대 이상일 때는 문제가 발생한다. 왜냐하면 각 서버가 가지고 있는 세션 정보를 서버들끼리 공유하지 않기 때문이다. 그렇게 되면 한 명의 사용자가 로드밸런서에 의해 여러 서버에 요청을 골고루 보내게 될텐데, 어떤 요청에서는 인증이 성공하지만 어떤 요청에서는 인증이 실패하는 상황이 발생한다.



출처 : https://hstory0208.tistory.com/

따라서 이를 해결하기 위해 세션 정보를 공유하기 위한 기존 서버와 별개인 외부 세션 스토리지가 필요하다. 외부 세션 스토리지를 사용하게 되면 세션 정보를 외부 세션 스토리지 한 곳에 저장해두면 되기 때문에, 세션 정보를 서로 공유할 수 있게 된다.



여기서 외부 세션 스토리지로 많이 활용하는 툴 중 하나가 Redis이다.

👨‍🏫 다음 강의에서는 로컬 세션 스토리지의 문제점을 직접 눈으로 확인해보고, 문제점을 개선하기 위해 Redis를 도입하는 것까지 실습으로 진행해보자.

kb302500@gmail.com

[실습] 인증 기능을 로컬 세션 스토리지로 구현했을 때 발생하는 문제점 - 1

👨‍🏫 Redis로 세션 스토리지를 구축하기 전에, 로컬 세션 스토리지 방식으로 구현한 코드를 먼저 살펴보자.

✅ 실습

1. **코드 살펴보기**

> 실습의 편의를 위해 일부 코드를 작성해두었다.

AuthController

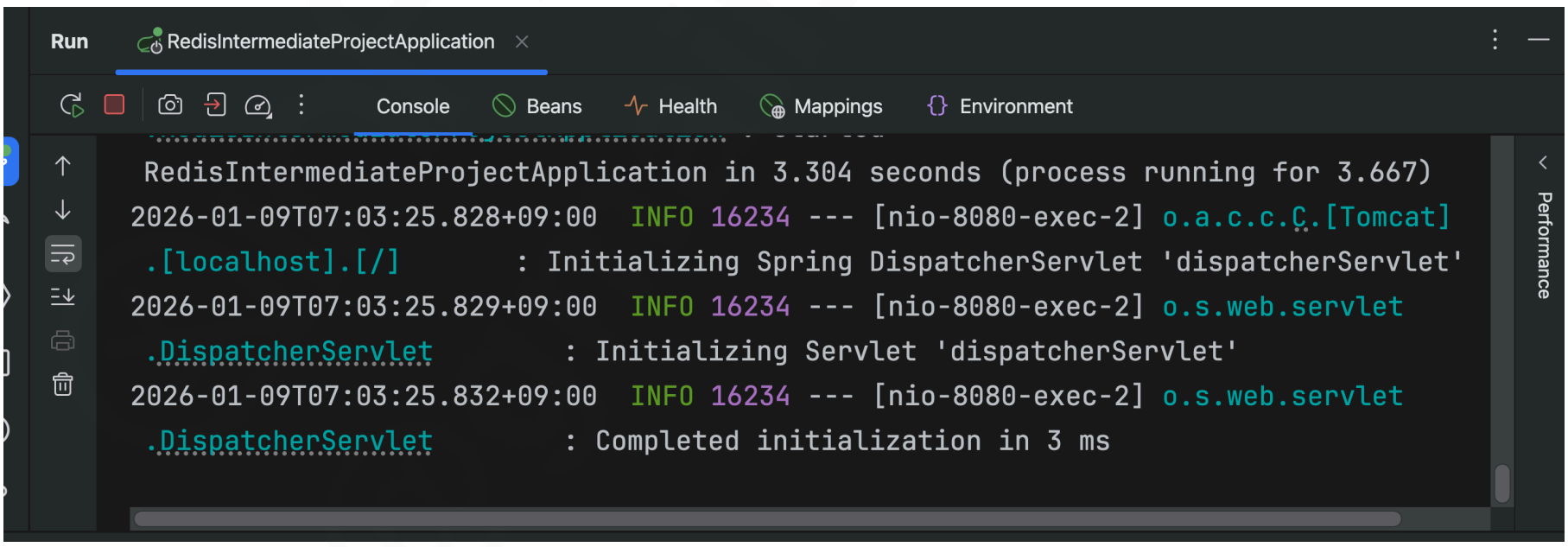
```
@RestController
@RequestMapping("/auth")
public class AuthController {

    @PostMapping("/login")
    public String login(
        @RequestBody LoginRequestDto loginRequest,
        HttpSession session
    ) {
        // 간단한 예제로 실습하기 위해, 로그인 할 사용자 정보 조회를 하드코딩으로 대체.
        // 실제 구현에서는 로그인을 하기 위한 정보를 DB에서 조회해야 함.
        String account = "jscode";
        String password = "password";
        String name = "박재성";
        long userId = 1;

        if (account.equals(loginRequest.getAccount()) && password.equals(loginRequest.getP
assword())) {
            // 로그인 성공 시 세션에 사용자 정보(userId, name 등) 저장한다.
            // 세션 정보는 Spring Boot에 내장된 톰캣의 인메모리에 저장된다.
            session.setAttribute("userId", userId);
            session.setAttribute("name", name);

            // 간단한 실습을 위해 String으로 심플하게 응답.
            return "로그인 성공";
        } else {
            return "로그인 실패";
        }
    }

    @PostMapping("/logout")
    public String logout(HttpSession session) {
        session.invalidate();
        // 간단한 실습을 위해 String으로 심플하게 응답.
        return "로그아웃 성공";
    }

    // 내 정보 조회 API
    @GetMapping("/me")
```

```
    public String status(HttpSession session) {
        Long userId = (Long) session.getAttribute("userId");
        String name = (String) session.getAttribute("username");

        // 간단한 실습을 위해 String으로 심플하게 응답.
        if (userId == null) {
            return "로그인이 필요합니다.";
        } else {
            return "userId : " + userId + ", " + "name : " + name;
        }
    }
}
```

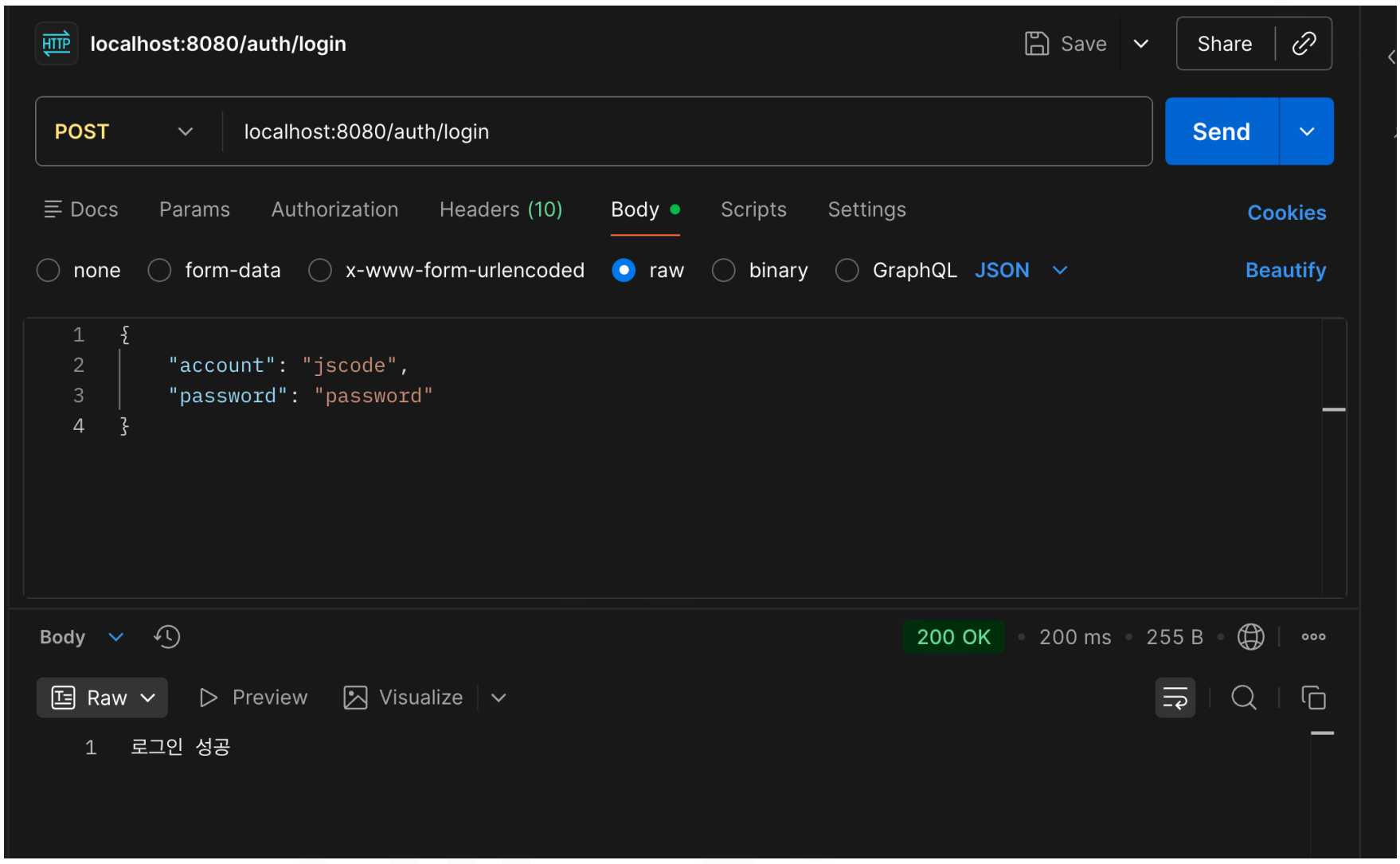
LoginRequestDto

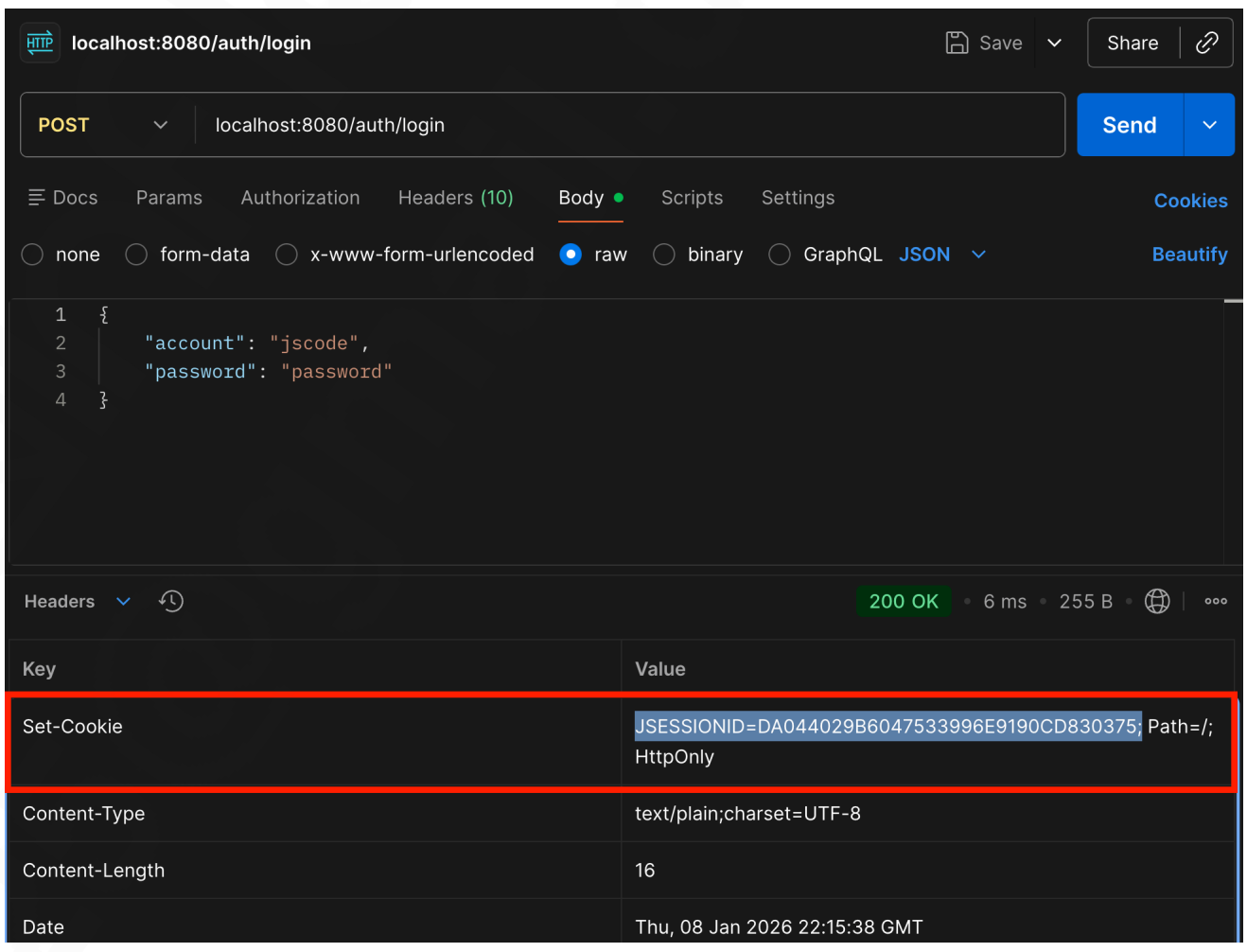
```
@Getter
public class LoginRequestDto {
    private String account;
    private String password;
}
```

2. **Spring Boot 서버 실행하기**



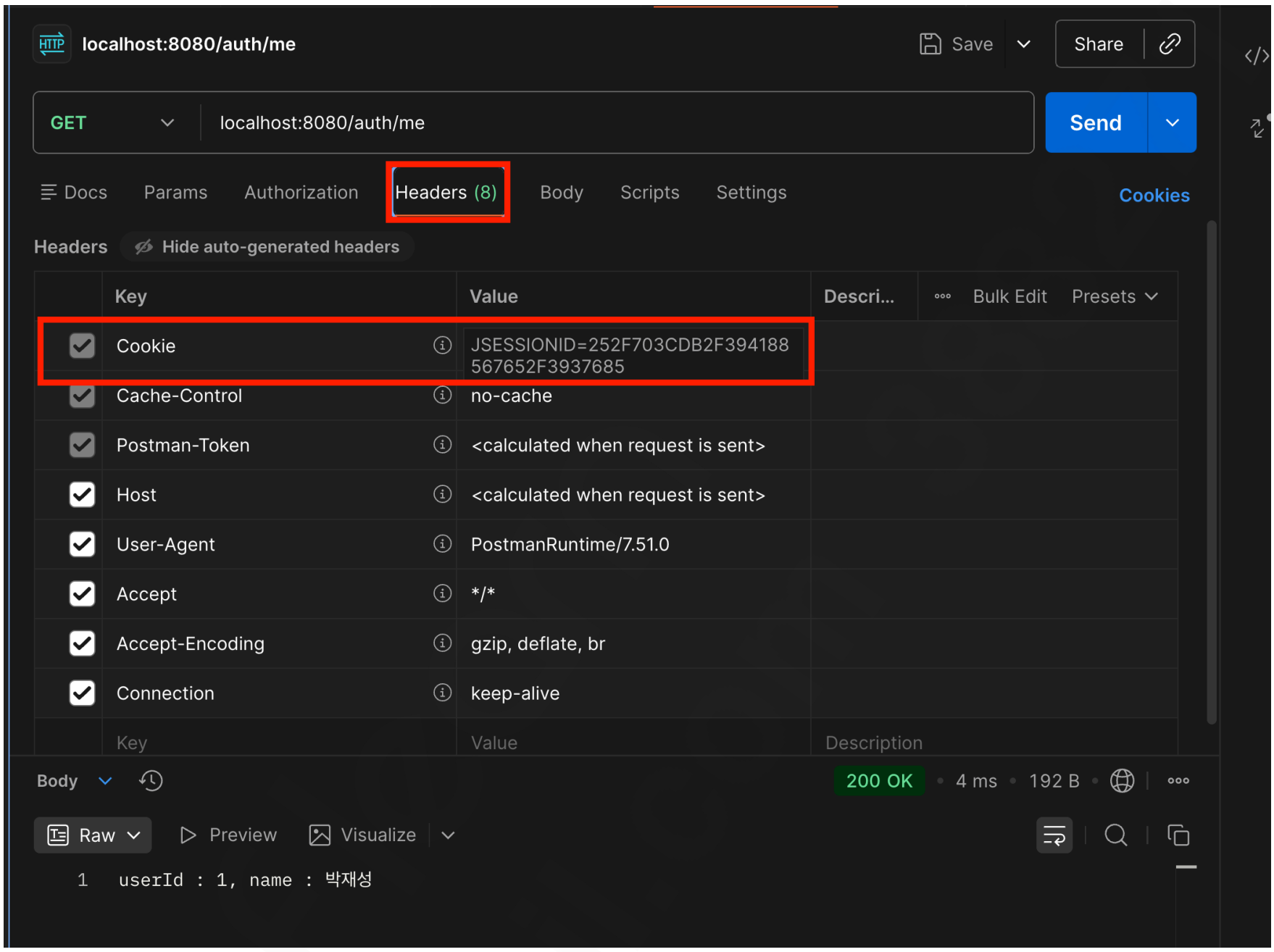
3. **Postman으로 API 잘 작동하는 지 테스트해보기**
    a. **로그인 API**





로그인 API로 요청을 보내면 Response Header에 Set-Cookie의 값이 날라오기 때문에 Postman에 Cookie가 자동으로 저장이 된다. 그래서 다음 요청을 보낼 때 해당 Cookie 값을 포함시켜서 요청을 보내게 된다.

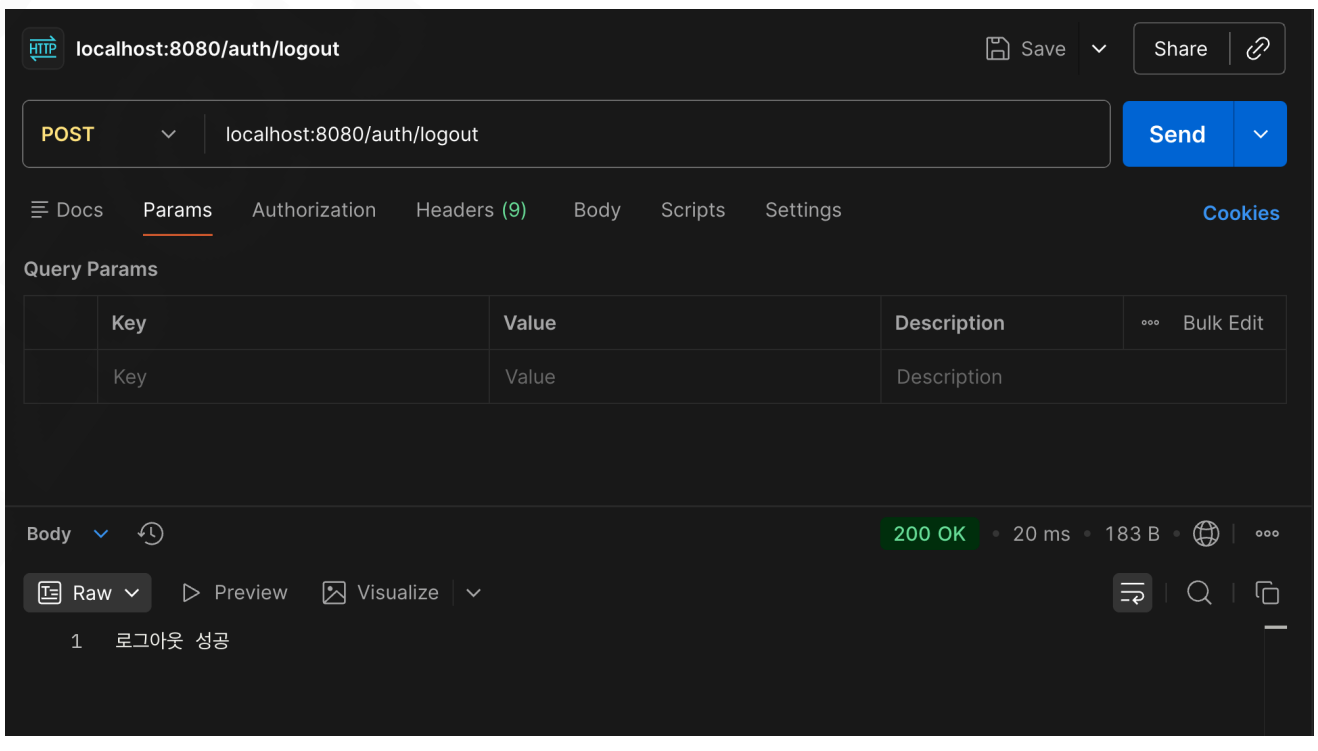
b. **내 정보 조회 API**

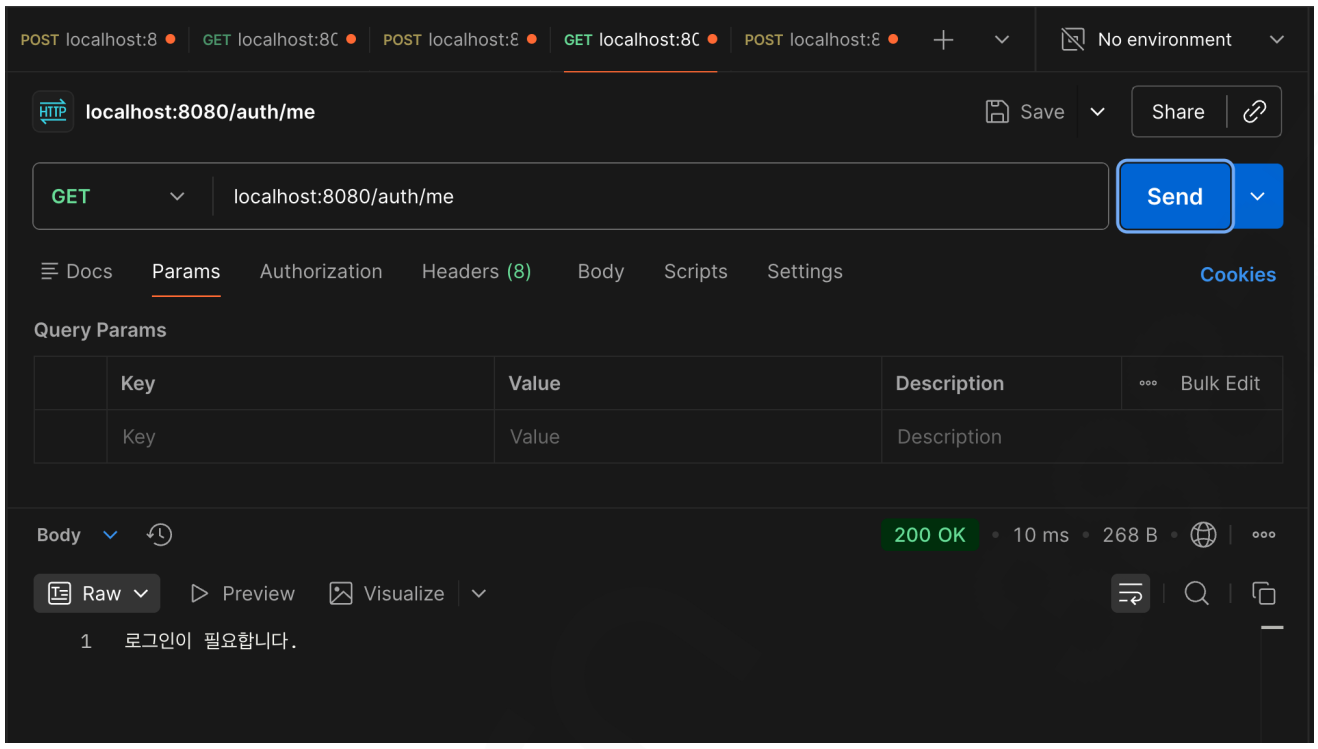


Request를 보내기 전에 Header를 열어보면 Cookie 값이 자동으로 담겨져있는 것을 확인할 수 있다. 그러고 내 정보 조회 API에 요청을 보내보면 정상적으로 userId와 name이 응답값으로 날라오는 걸 볼 수 있다.

c. **로그아웃 API → 내 정보 조회 API**

'로그아웃 API'를 호출한 뒤에 '내 정보 조회 API'를 호출해보면 정상적으로 로그아웃이 된 걸 확인할 수 있다.

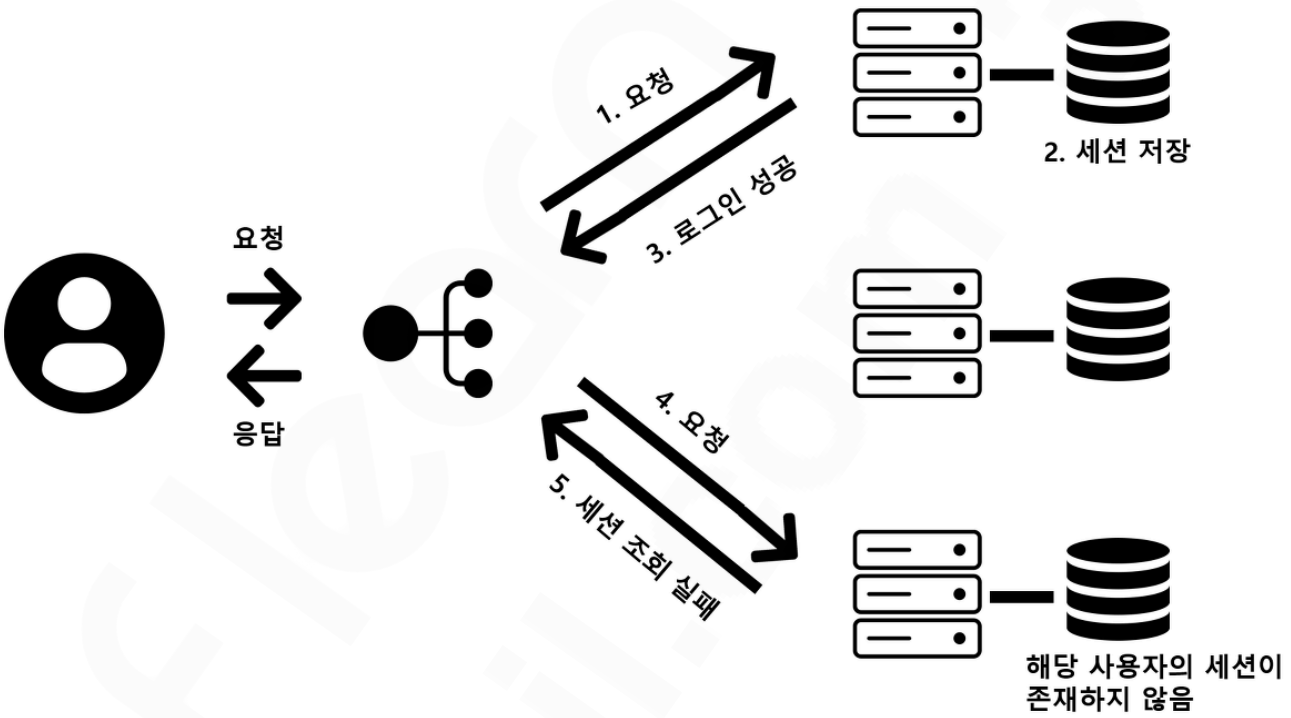




@gmail.com
khb302

[실습] 인증 기능을 로컬 세션 스토리지로 구현했을 때 발생하는 문제점 - 2

👨‍🏫 이번에는 Spring Boot 서버를 2대 실행시켜서, 이전 강의에서 이론으로 배웠던 로컬 세션 스토리지의 문제점을 눈으로 직접 확인해보자.



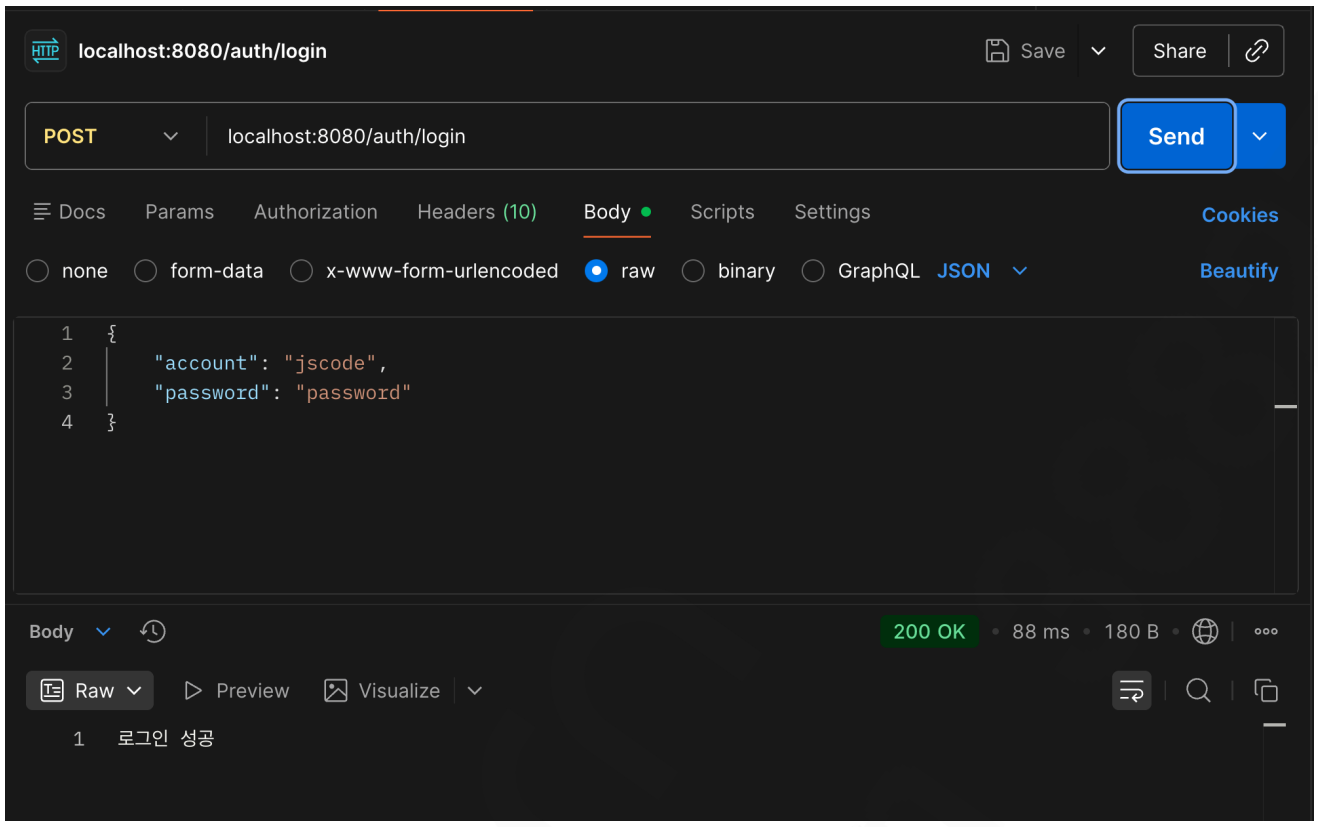
✅ 실습

1. **Spring Boot 서버 두 대 띄우기**

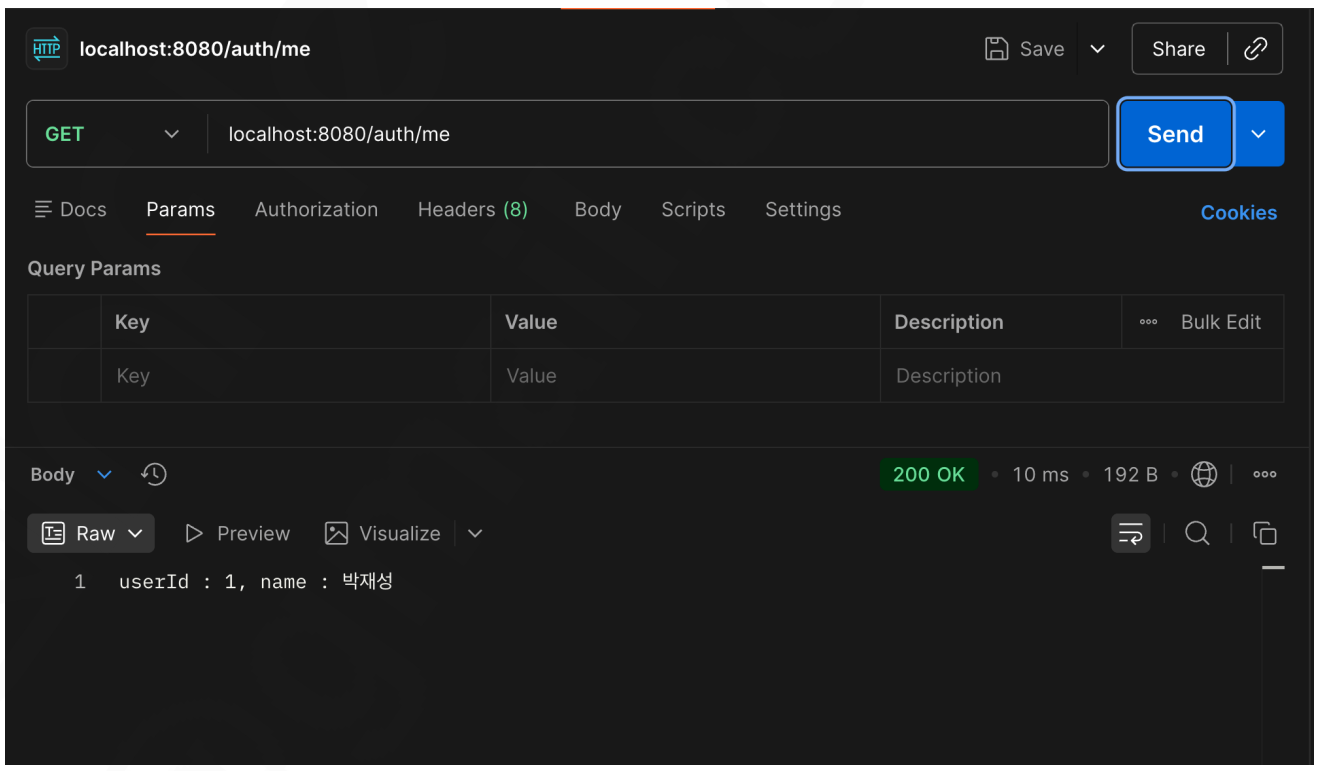
   > 기존에 실행시켜놨던 Spring Boot 서버가 있다면 종료하고 아래 명령어를 입력하자.

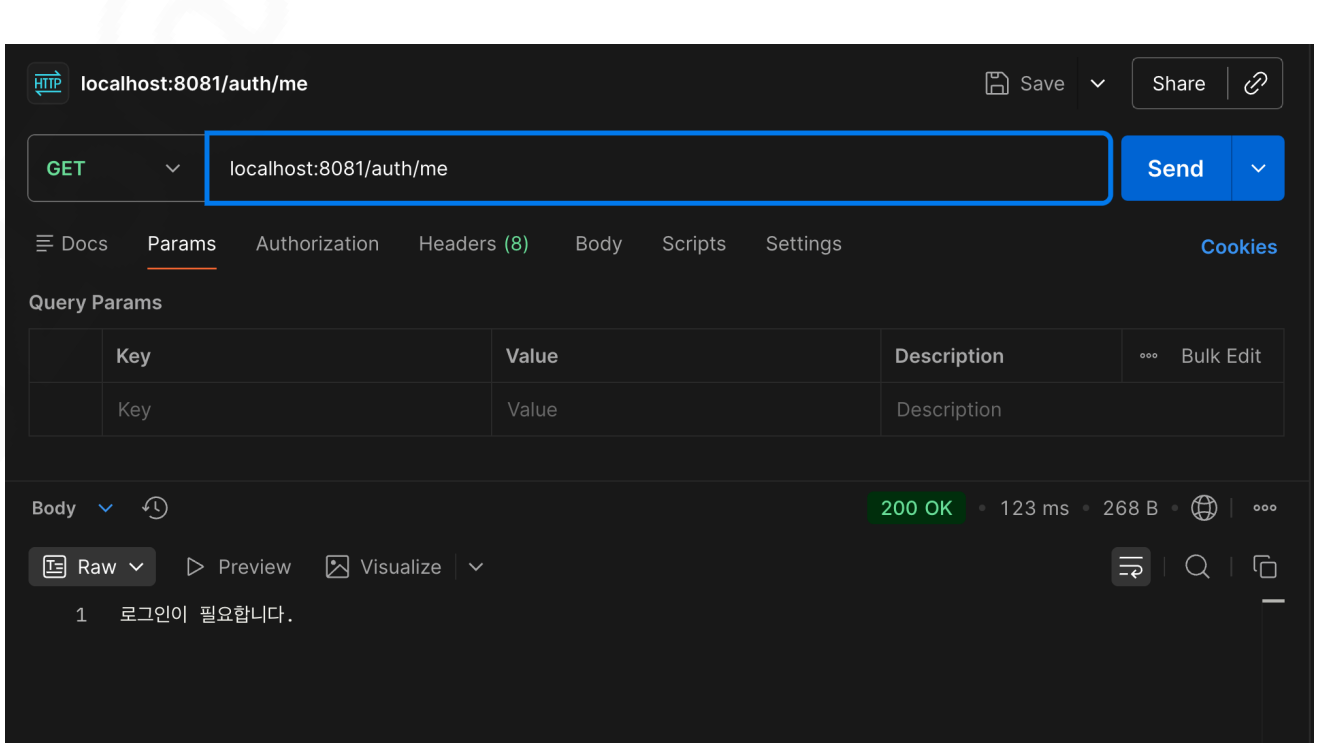
   ```
   $ ./gradlew bootRun --args='--server.port=8080'
   $ ./gradlew bootRun --args='--server.port=8081'
   ```

2. **Postman으로 API 테스트해보기**
   a. **8080번 포트로 로그인 API 요청 보내기**



b. **8080번, 8081번 포트에서 사용자 정보 조회 API 요청 보내기**





세션 정보가 저장된 8080번 포트의 서버에서는 사용자 정보 조회 API가 정상적으로 잘 되지만, 세션 정보가 저장되어 있지 않은 8081번 포트의 서버에서는 로그인이 필요하다고 뜬다. 각 서버의 인메모리에서 세션 정보를 관리하고 있기 때문에 발생하는 문제이다.

다음 강의에서는 Redis를 활용해 이 문제를 해결해볼 것이다. Redis라는 외부 세션 스토리지를 구축하게 되면 세션 정보를 공유할 수 있게 되기 때문에 서버가 여러 대이더라도 이런 문제가 발생하지 않을 것이다.

@gmail.com
khb302

🎯 [실습] Redis로 세션 스토리지 구축하기

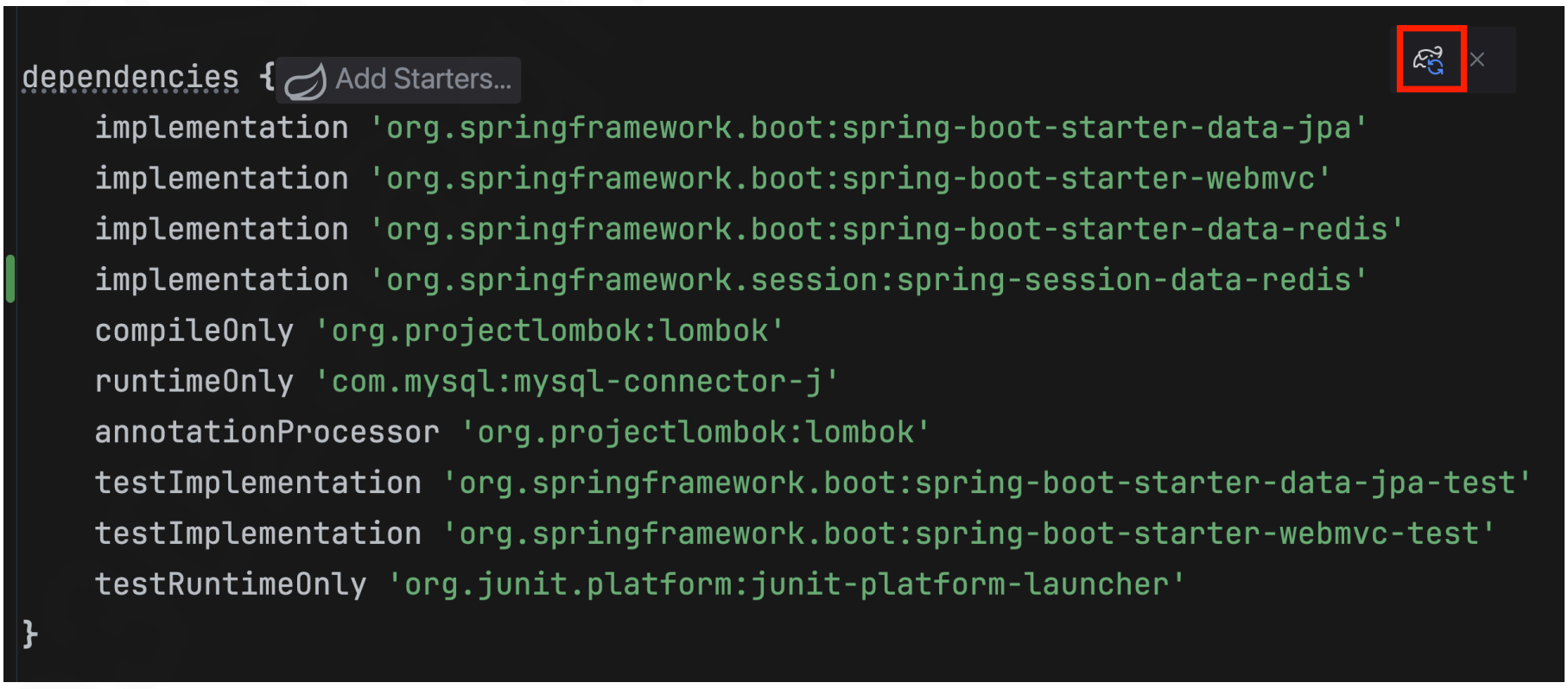
👨🏻‍💻 Spring Boot에는 `spring-session-data-redis`라는 라이브러리가 있어서 Redis로 세션 스토리지를 쉽게 구축할 수 있다. 이 라이브러리를 활용해서 세션 스토리지를 구축해보자.

✅ 실습

1. **의존성 추가하기**

   build.gradle

```
...

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-data-redis'
    implementation 'org.springframework.session:spring-session-data-redis'
    compileOnly 'org.projectlombok:lombok'
    runtimeOnly 'com.mysql:mysql-connector-j'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-data-jpa-test'
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

...
```



2. **application.yml 수정하기**

   application.yml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb?rewriteBatchedInserts=true
```

```
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  data:
    redis:
      host: localhost
      port: 6379
  jpa:
    hibernate:
      ddl-auto: create
    show-sql: true
    session:
    store-type: redis
```

3. **RedisSessionConfig 파일 만들기**

   Spring Boot에서 Redis 기반으로 세션 관리를 활성화하려면 아래 파일을 추가해주어야 한다.

   RedisSessionConfig

```
@Configuration
@EnableRedisHttpSession
public class RedisSessionConfig {

}
```

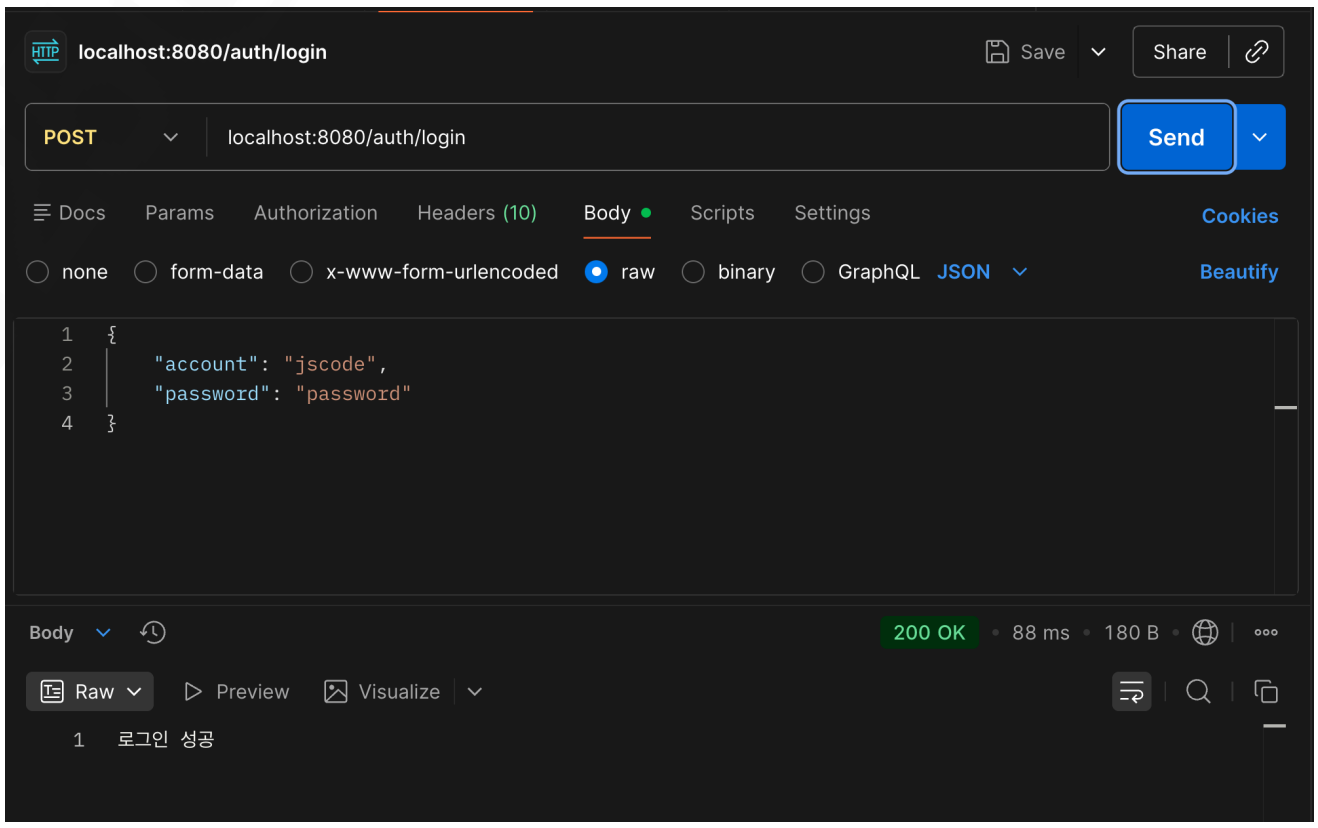
4. **Spring Boot 서버 두 대 띄우기**

   > 기존에 실행시켜놨던 Spring Boot 서버가 있다면 종료하고 아래 명령어를 입력하자.

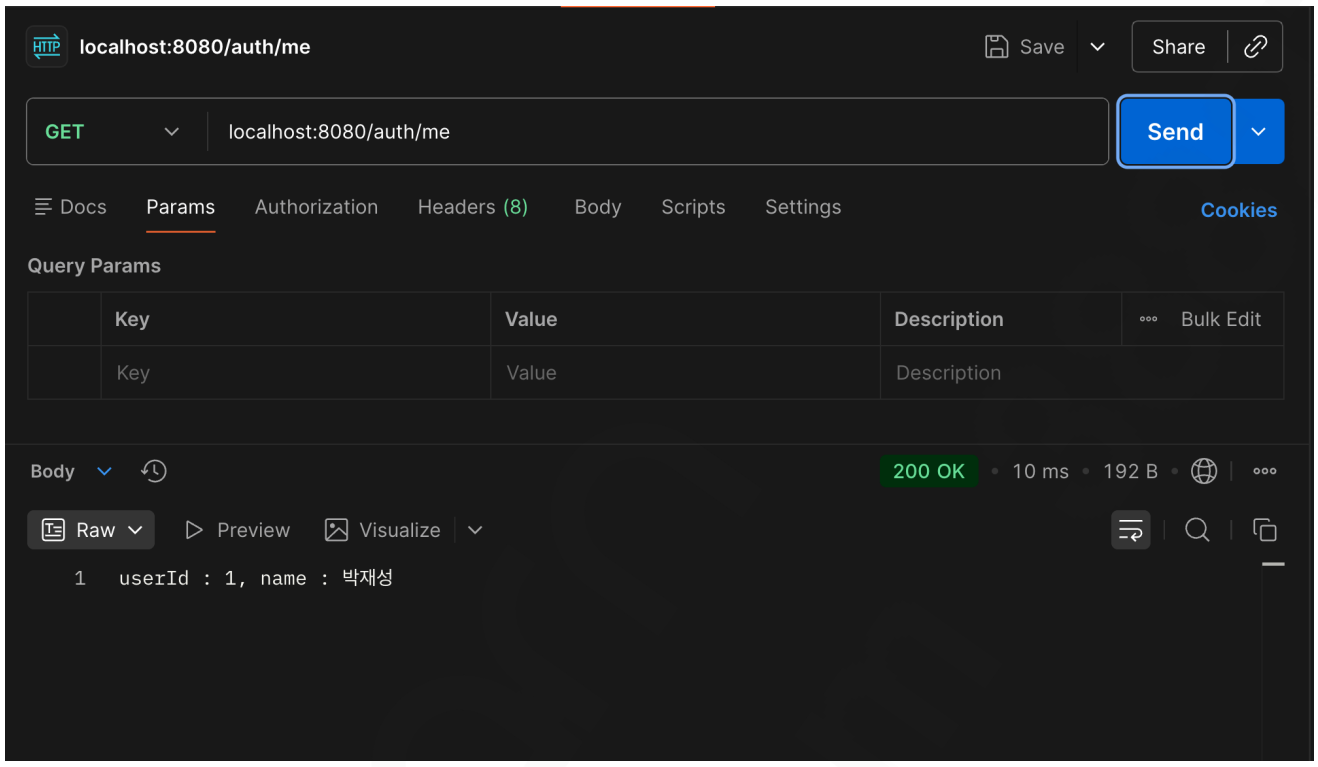
```
$ ./gradlew bootRun --args='--server.port=8080'
$ ./gradlew bootRun --args='--server.port=8081'
```

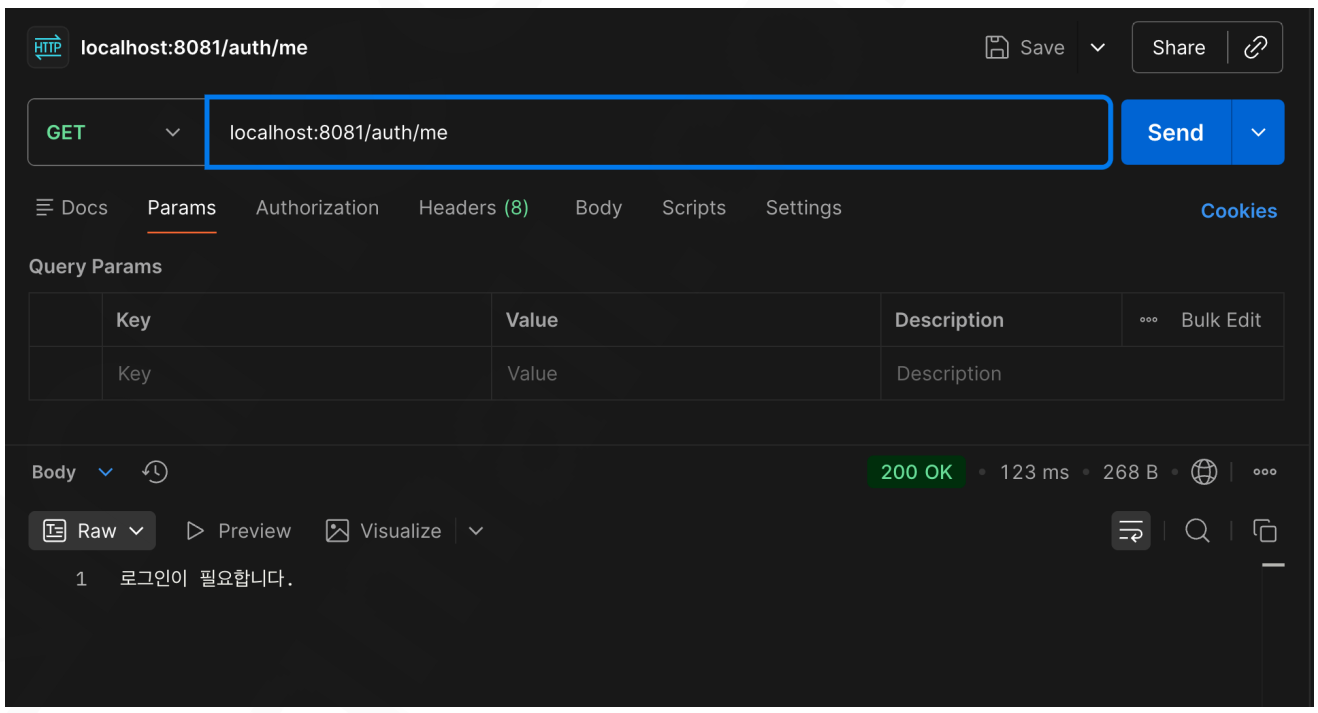
5. **Postman으로 API 테스트해보기**

   a. **8080번 포트로 로그인 API 요청 보내기**



b. **8080번, 8081번 포트에서 사용자 정보 조회 API 요청 보내기**





8080번, 8081번 포트의 서버 둘 다에서 사용자 정보 조회 API가 잘 작동한다. Redis를 통해 세션 정보를 공유하고 있기 때문에 어떤 서버로 요청을 보내든 잘 작동하는 것이다.

6. **Redis에 세션 정보 저장됐는 지 확인하기**

```
$ keys * # key 값 확인하기
$ type [세션이 저장된 key] # type 확인하기 -> hash

# (값이 깨져서 나올 것이다. 왜냐면 Spring Session이 세션 값을 Binary 형태로 Redis에 저장하기 때문이다.)
$ HGETALL [세션이 저장된 key] # 해당 hash에 저장된 모든 field, value 불러오기

$ HGET [세션이 저장된 key] sessionAttr:userId
$ HGET [세션이 저장된 key] sessionAttr:name
```